



UNIVPM

Università Politecnica delle Marche



Architettura Hardware

Luca Spalazzi

l.spalazzi@staff.univpm.it

www.univpm.it/luca.spalazzi



Dipartimento di Ingegneria
dell'Informazione



cini

Cyber Security Lab@UnivPM

Gruppo Ingegneria Informatica
@UnivPM

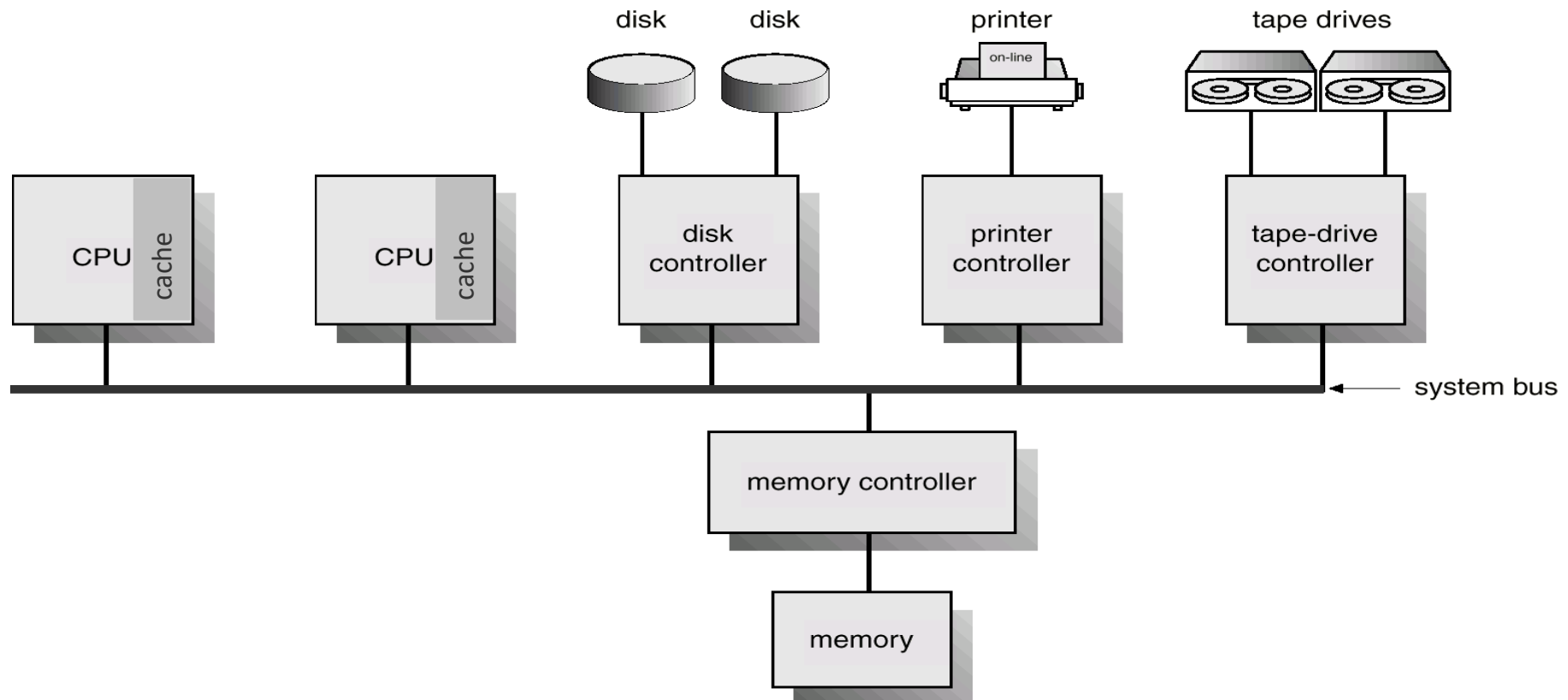
GII





- CPU e ciclo Fetch-Decode-Execute
- Organizzazione della Memoria
- Organizzazione dei Dispositivi di I/O
- Protezione dell'Hardware

Architettura di un Computer

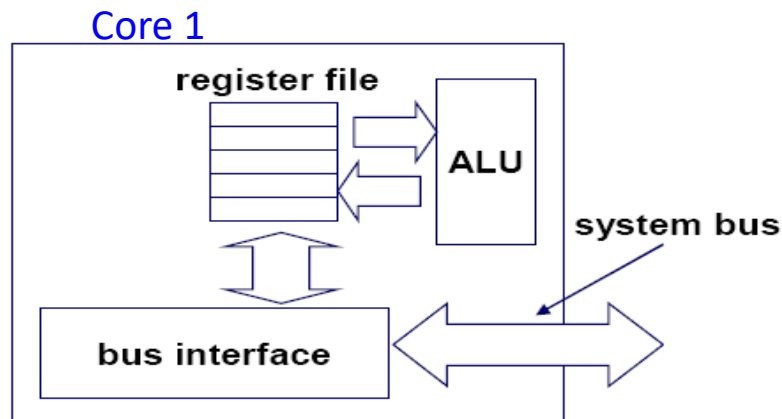


Operazioni di un Computer

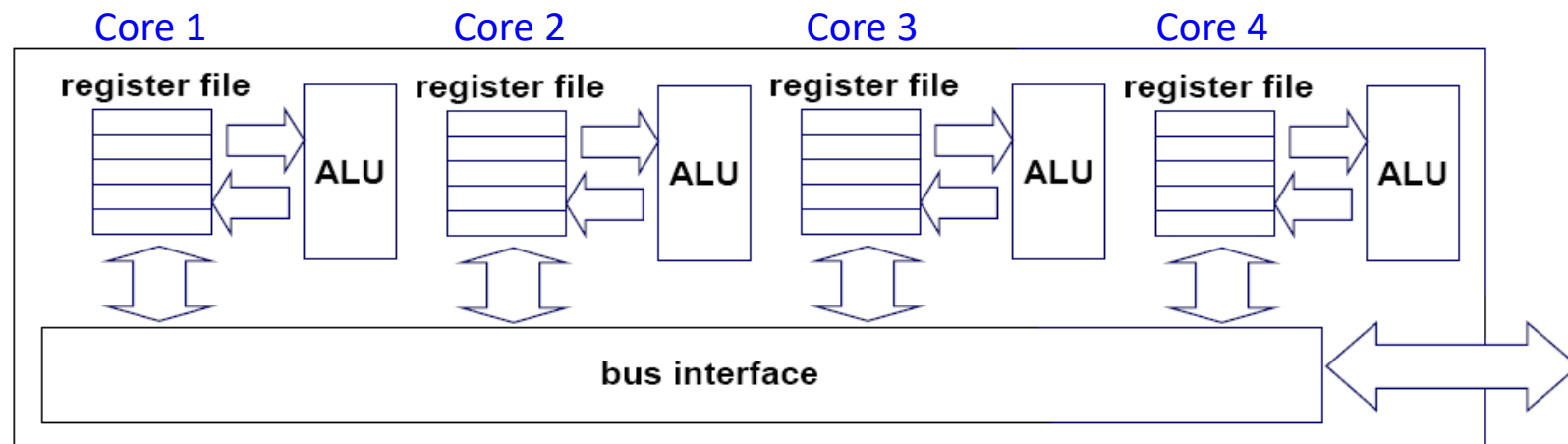
- I dispositivi di I/O e le CPU funzionano concorrentemente (architettura **master-slave**).
- Ogni controller si occupa solo di un particolare tipo di dispositivo.
- Ogni controller ha un buffer locale..
- La CPU trasferisce dati dalla memoria centrale ai buffer locali e viceversa.
- Una operazione di I/O consiste nel trasferire dati dal dispositivo al buffer locale e/o viceversa.
- Controller dei dispositivi e CPU devono essere sincronizzati tra loro.

Single Core vs. Multi Core

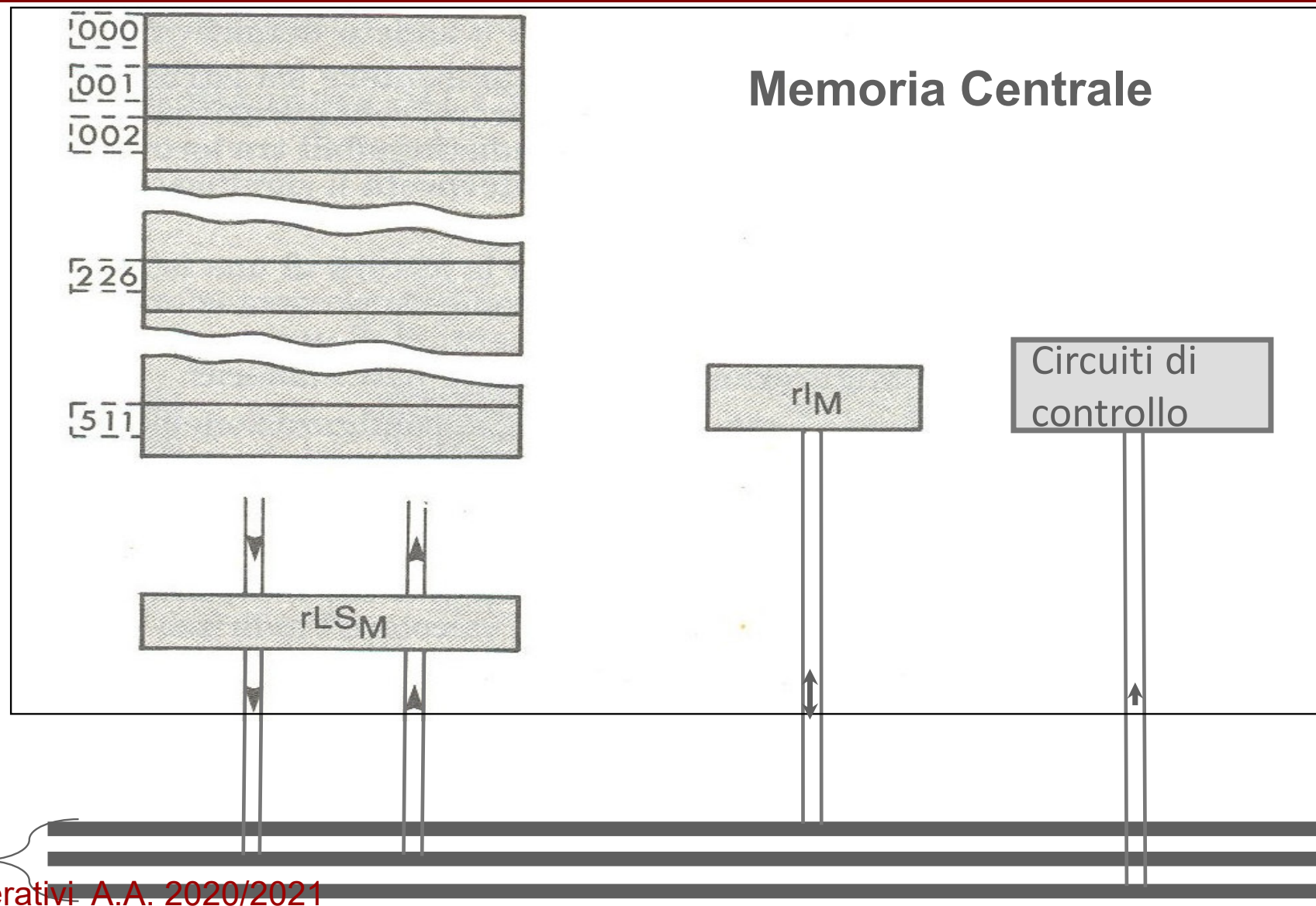
Single-core CPU chip



Multi-core CPU chip



Architettura della Memoria Centrale



Bus

Indirizzi
Dati
Controllo

Ciclo di funzionamento

```
while (not halt ) do
    { /* fetch istruzione */
        RCP -> RIp;
        RIp -> RIm;
        M[RIm] ->RLSm;
        RLSm -> RLSp;
        RLSp -> RIC;
        /* decodifica istruzione */
        "attiva rete di decodifica";
        /* esegui istruzione */
        RCP + 1 -> RCP
        . . . .
    }
```


Ciclo di funzionamento: un esempio

Esempio di programma in
Assembly Language.

Ogni istruzione è di solito
divisa in due parti:

OPCODE	OPERAND
--------	---------

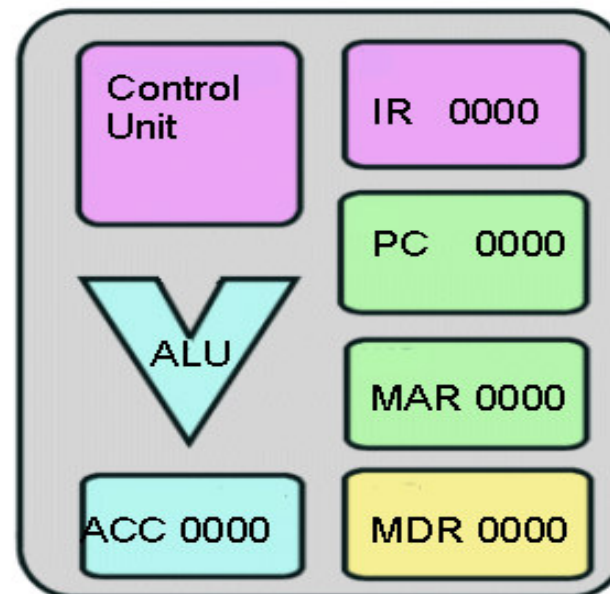
```
#assembly code program
```

```
0000 LDA-24    #load on ACC
```

```
0001 ADD-25    #add to ACC
```

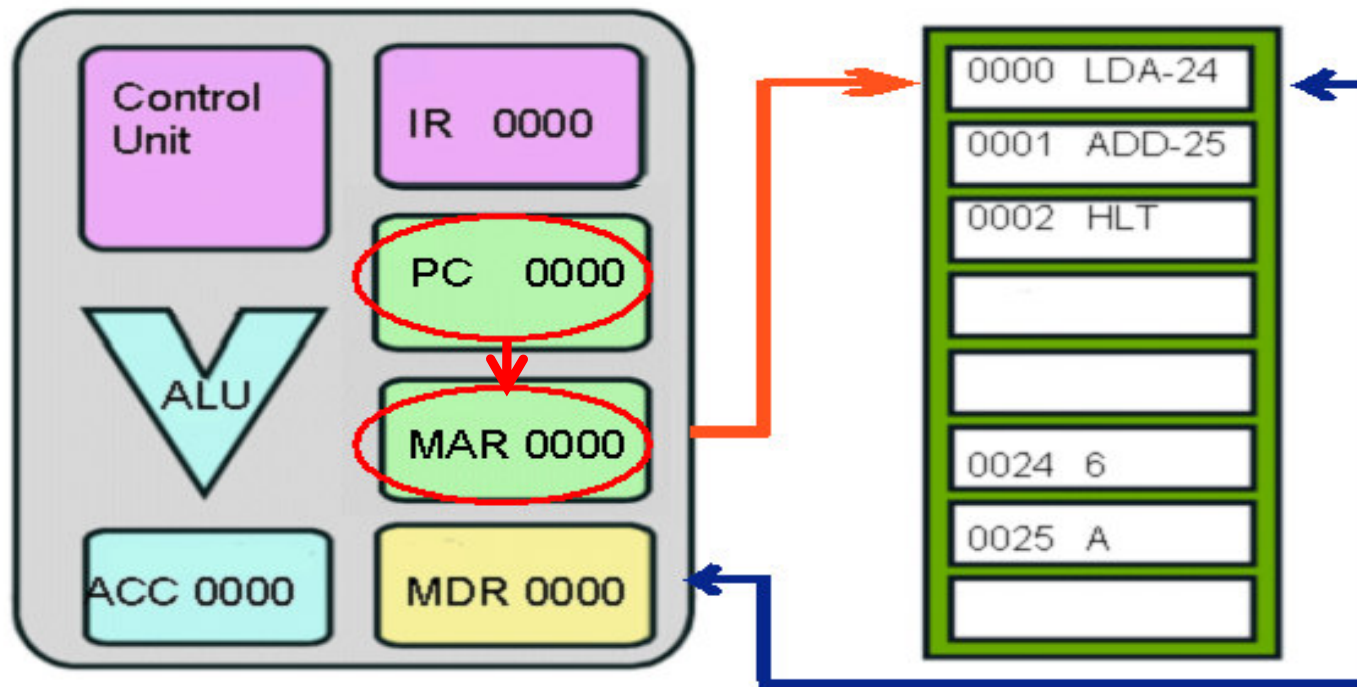
```
0002 HLT      #stop
```

```
#end of program
```

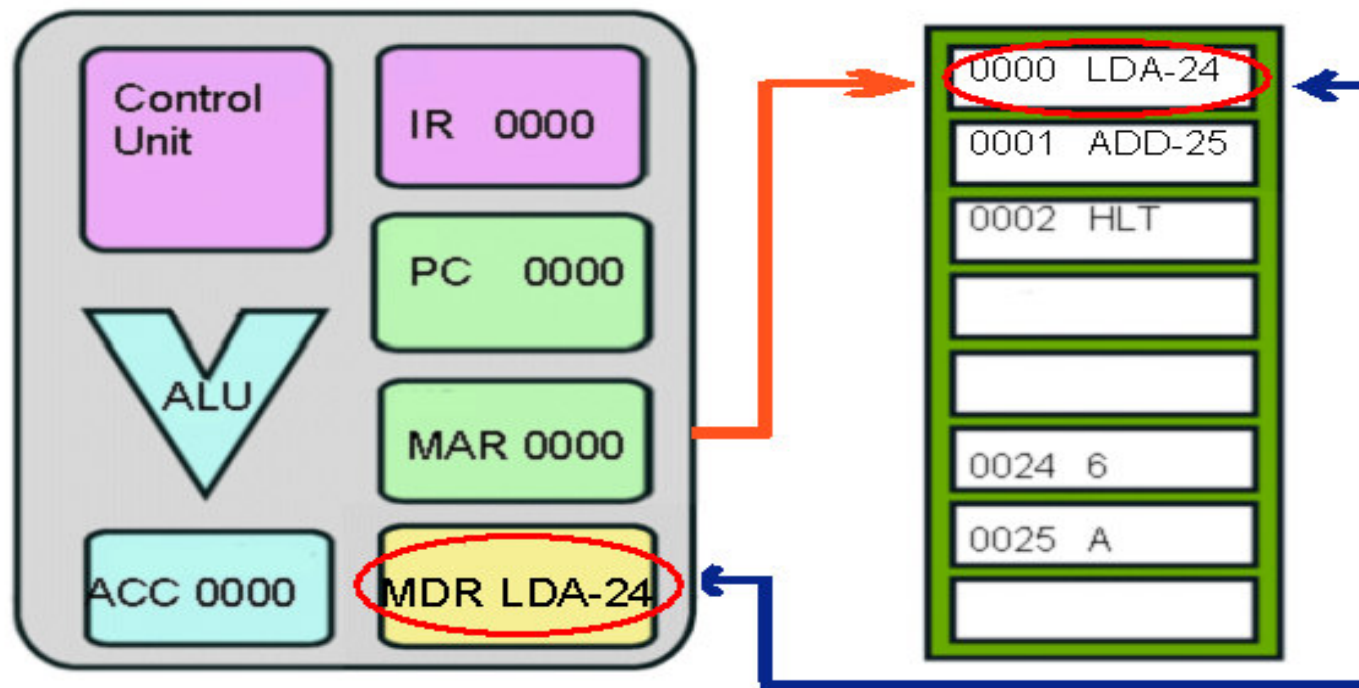


Ciclo di funzionamento: un esempio

fetch/1

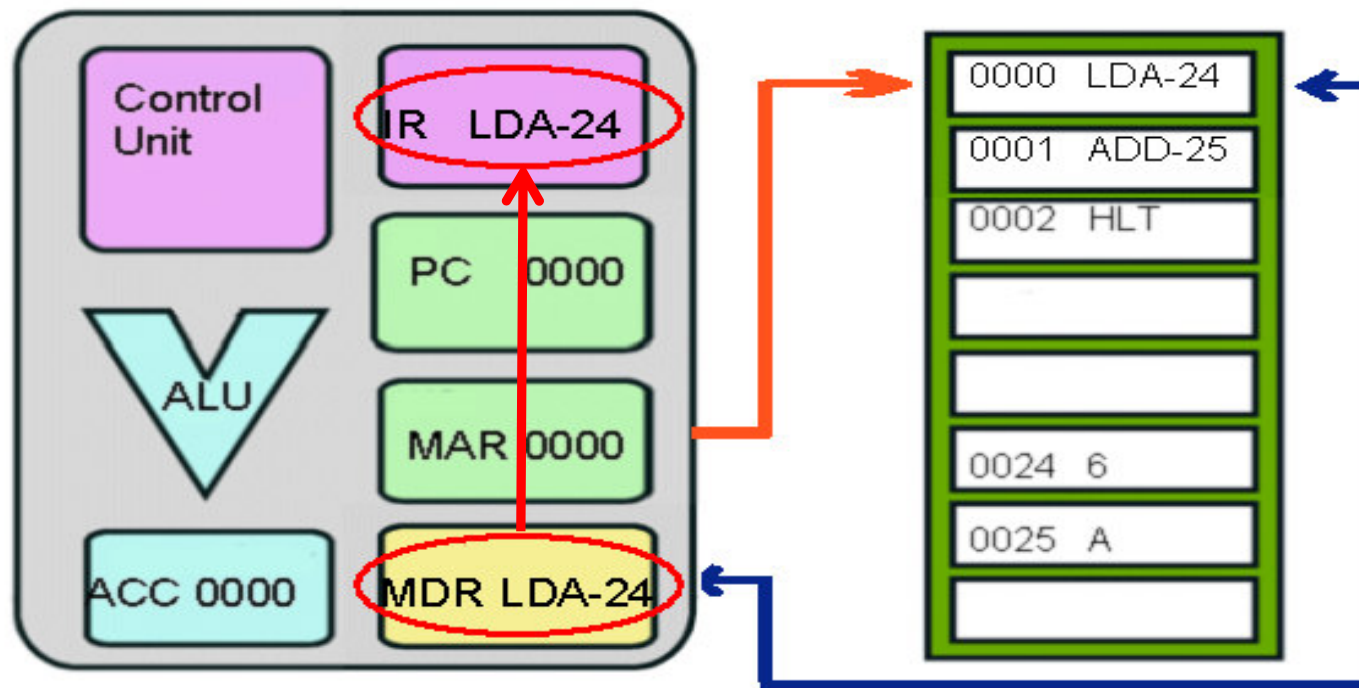


Ciclo di funzionamento: un esempio fetch/2



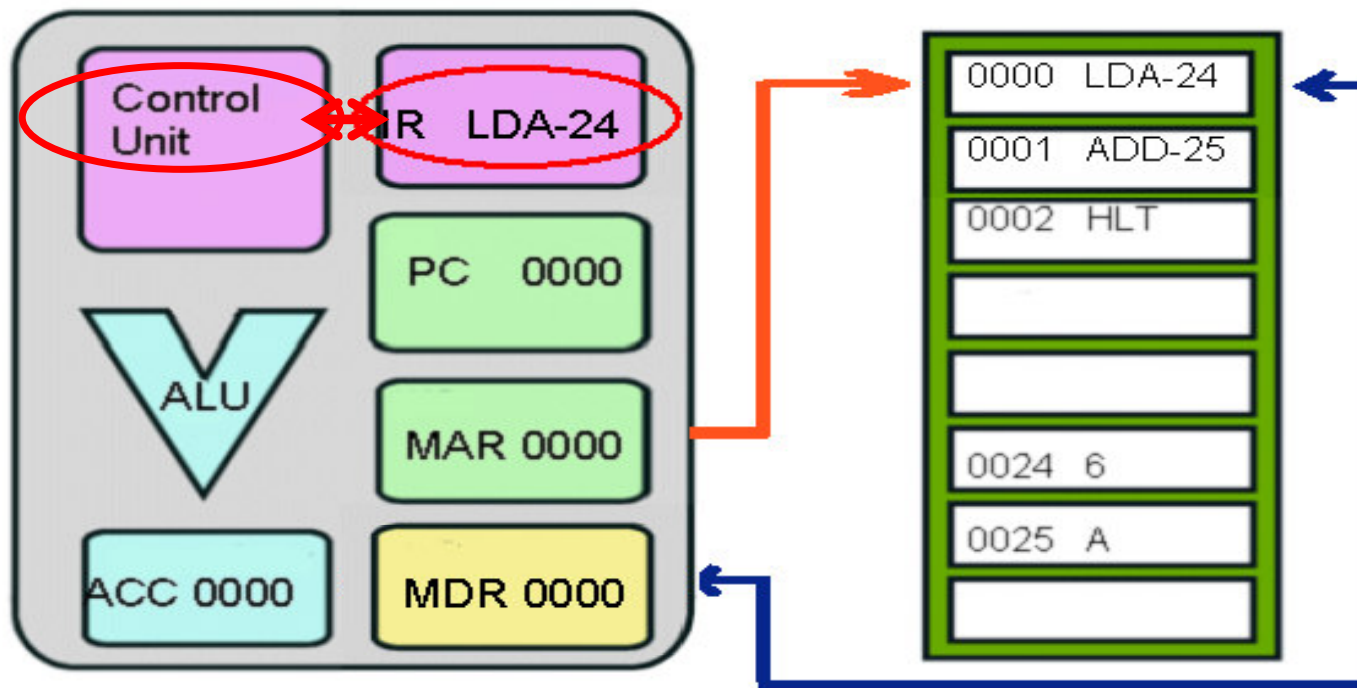
Ciclo di funzionamento: un esempio

fetch/3



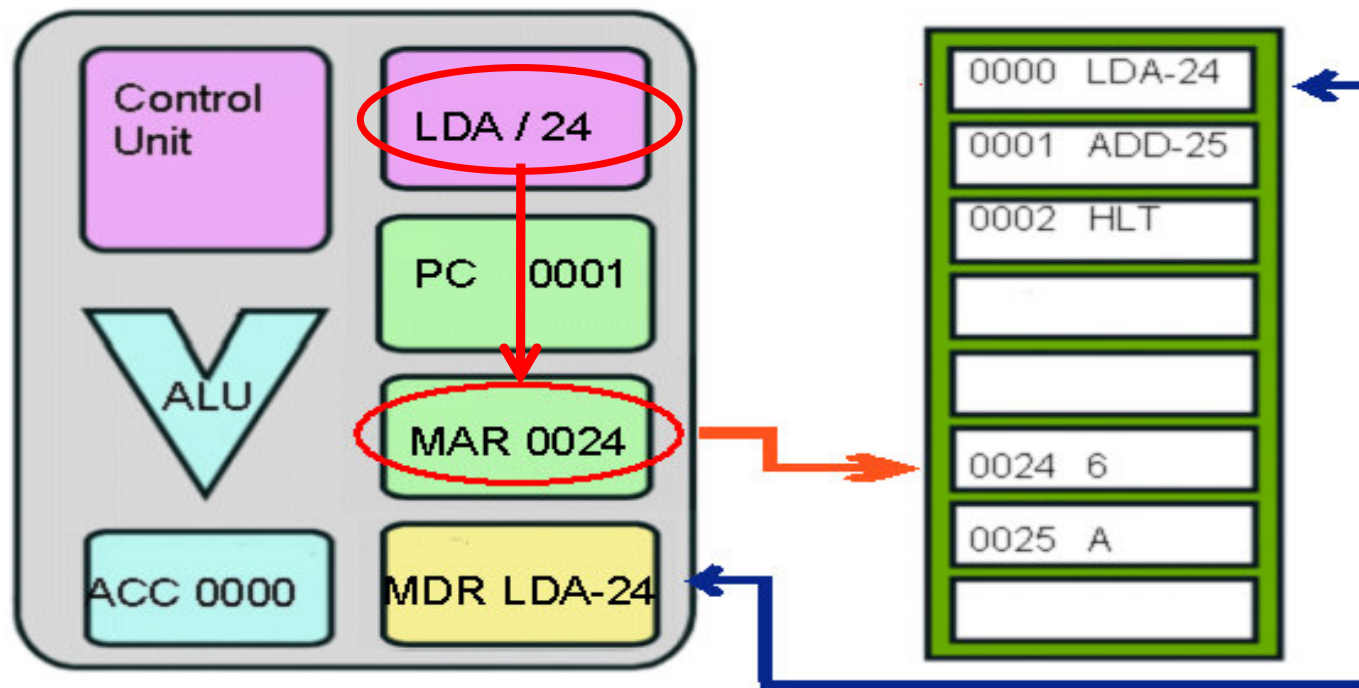
Ciclo di funzionamento: un esempio

decode/4



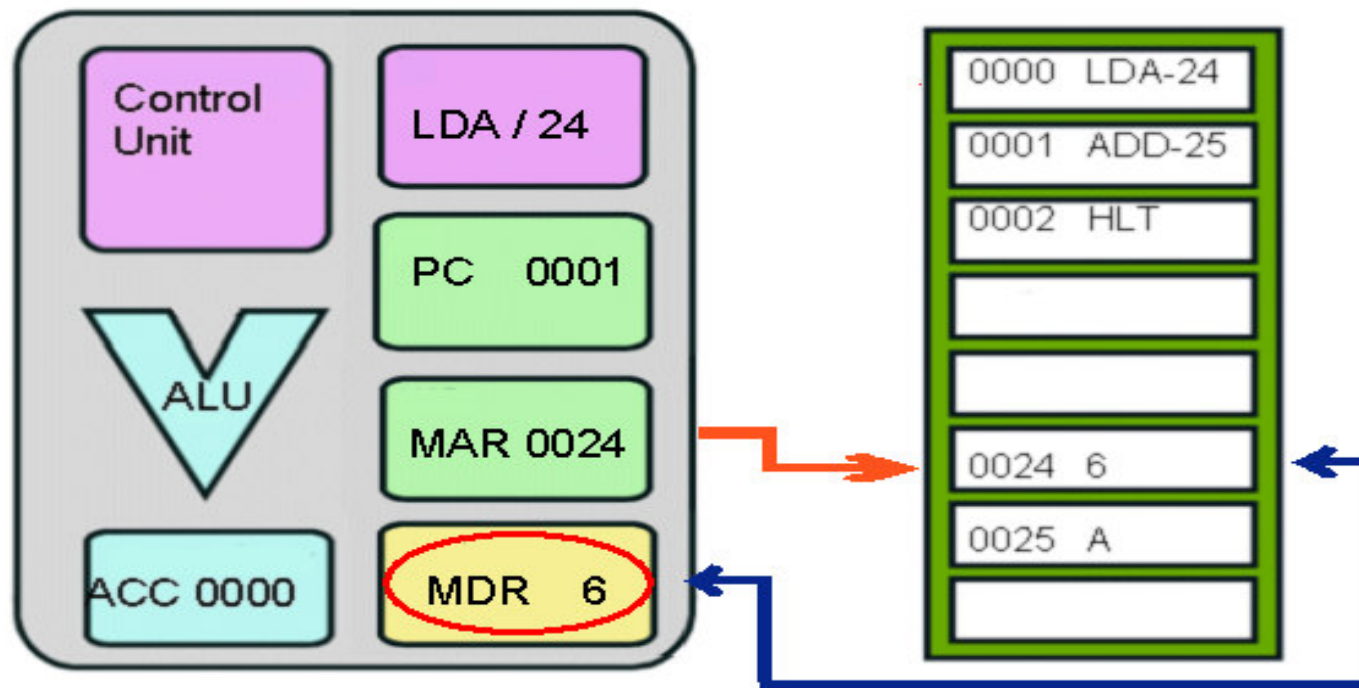
Ciclo di funzionamento: un esempio

execute/5



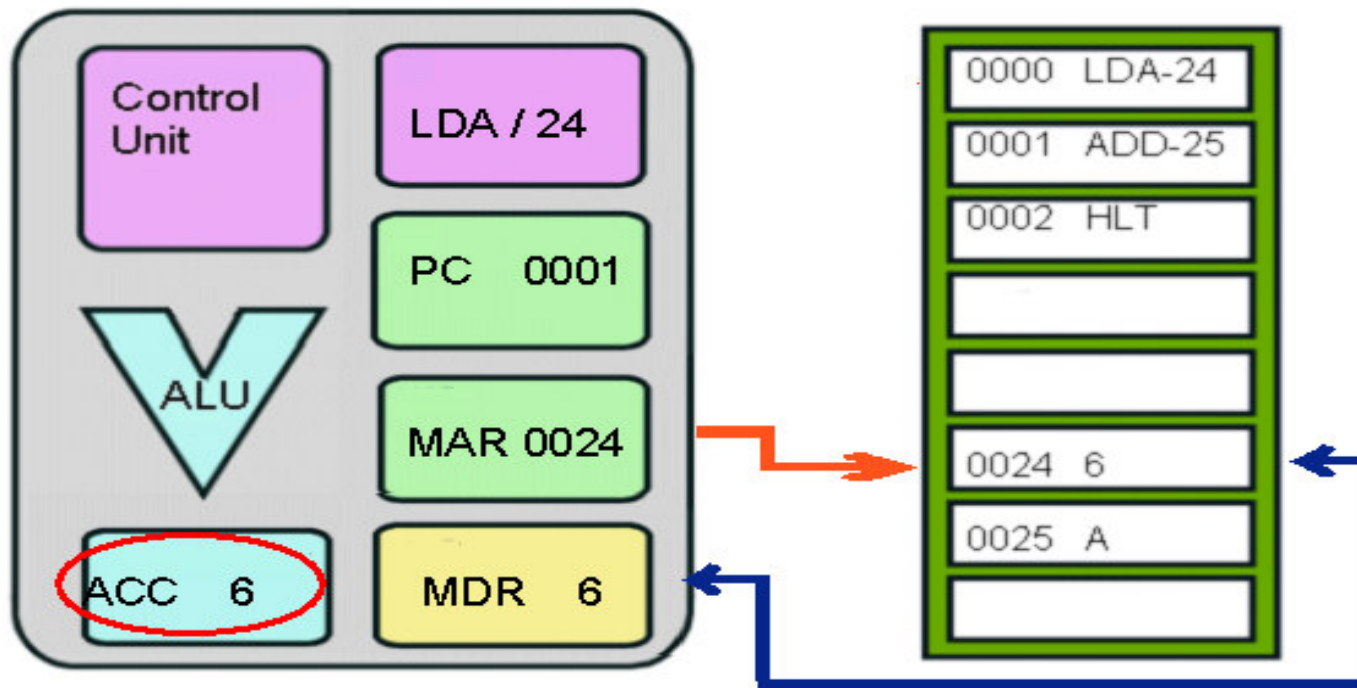
Ciclo di funzionamento: un esempio

execute/6



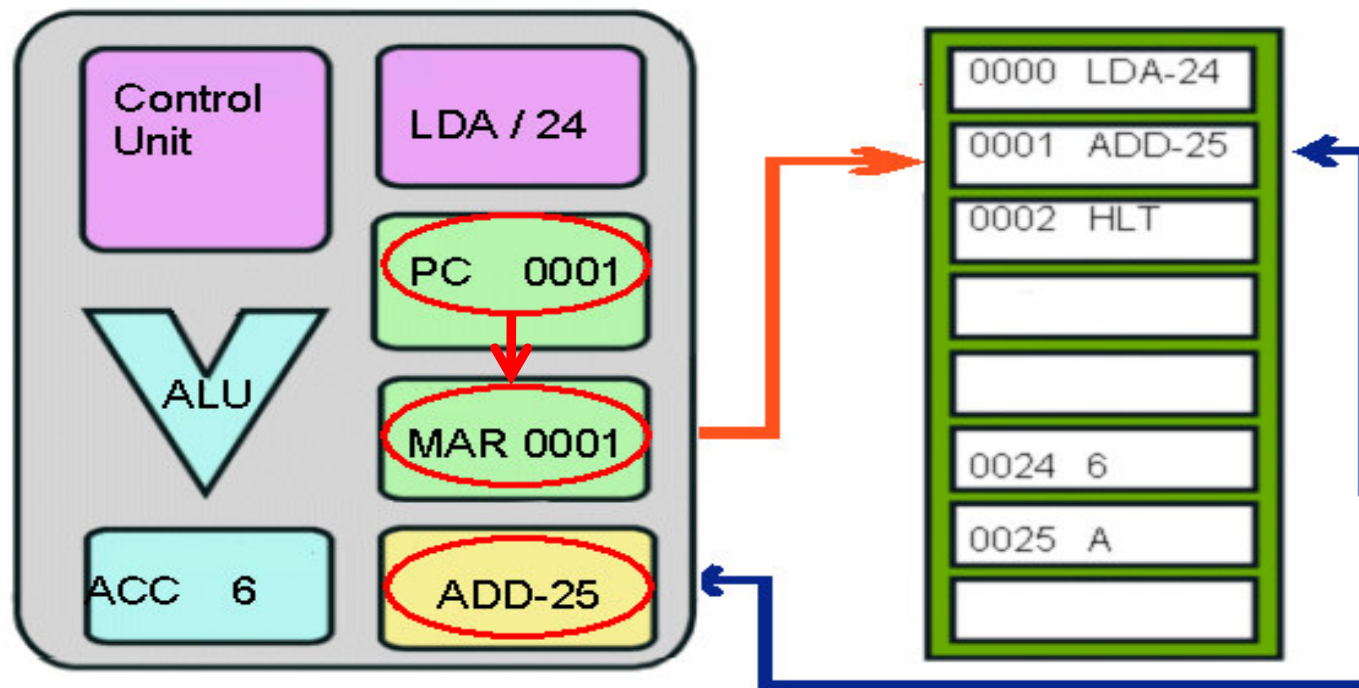
Ciclo di funzionamento: un esempio

execute/7



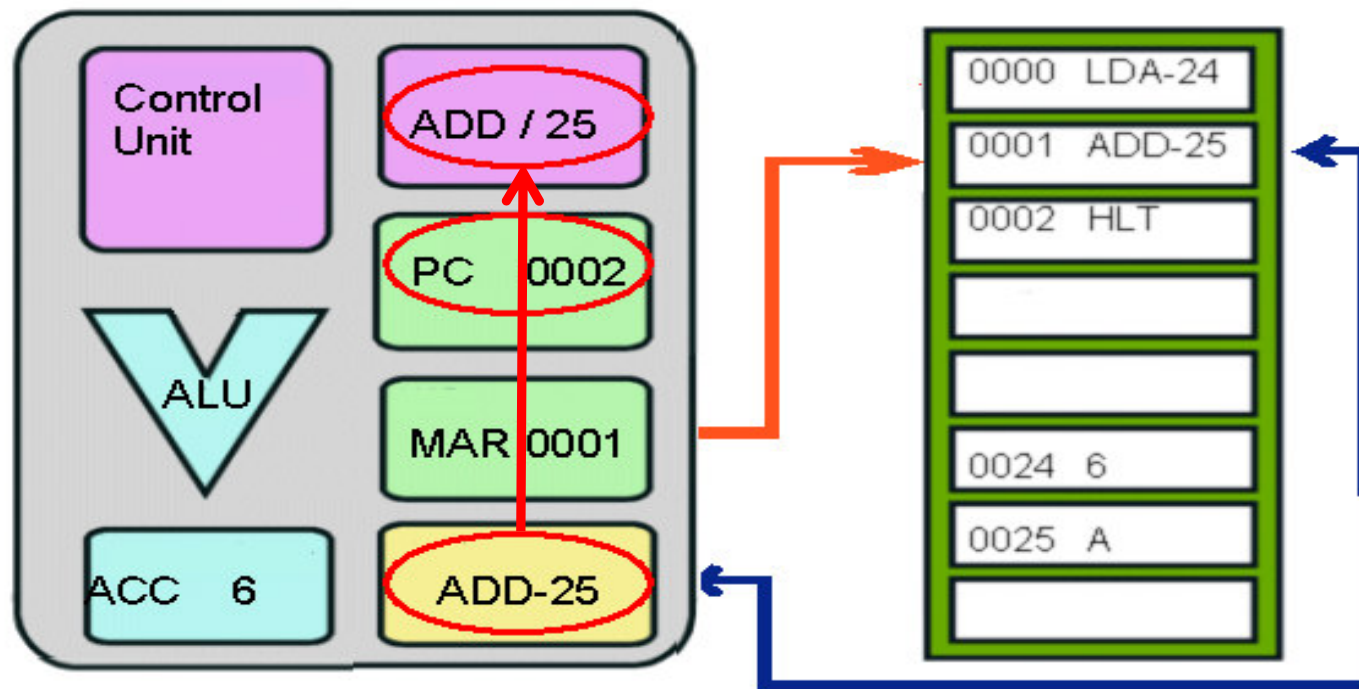
Ciclo di funzionamento: un esempio

fetch/8



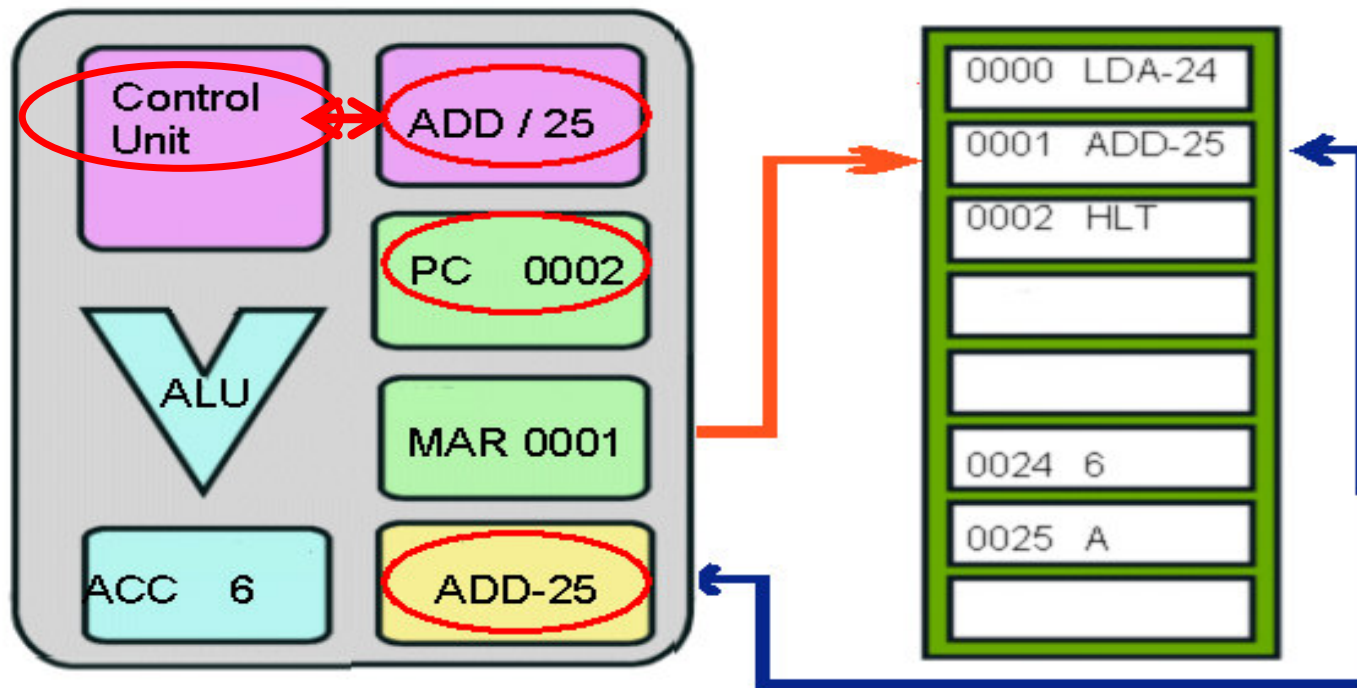
Ciclo di funzionamento: un esempio

fetch/9



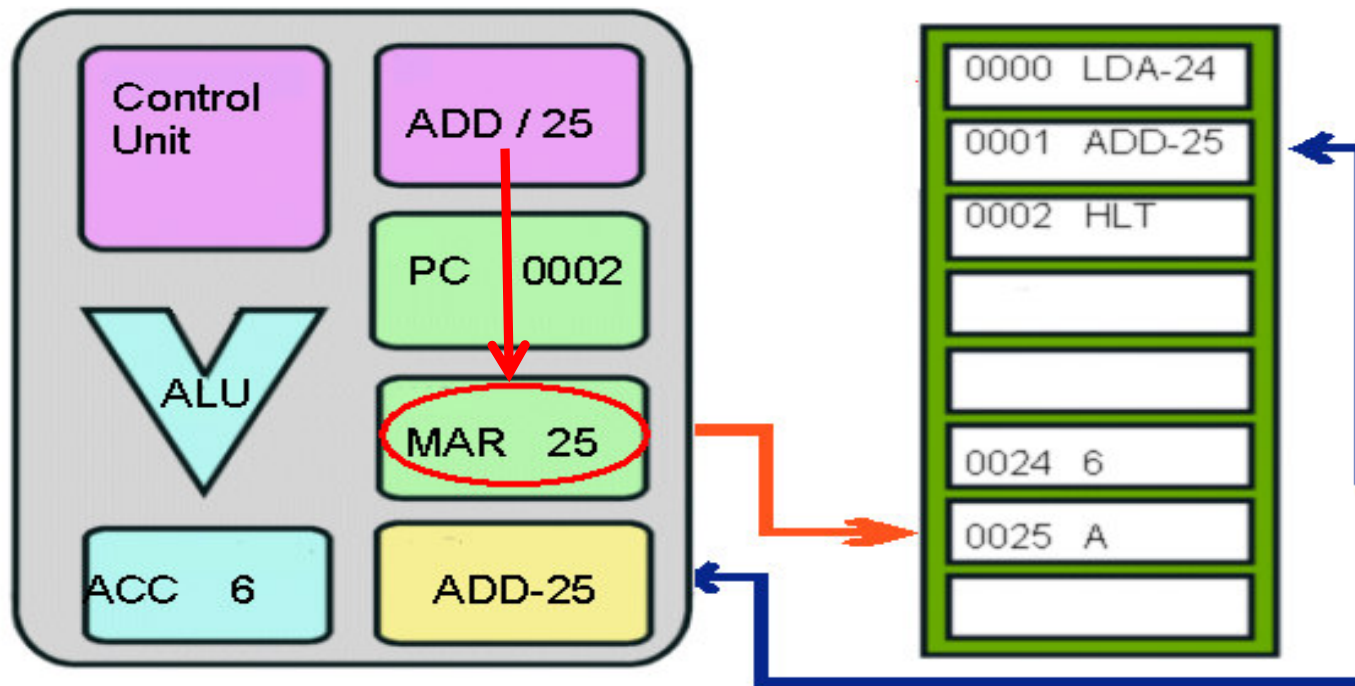
Ciclo di funzionamento: un esempio

decode/10



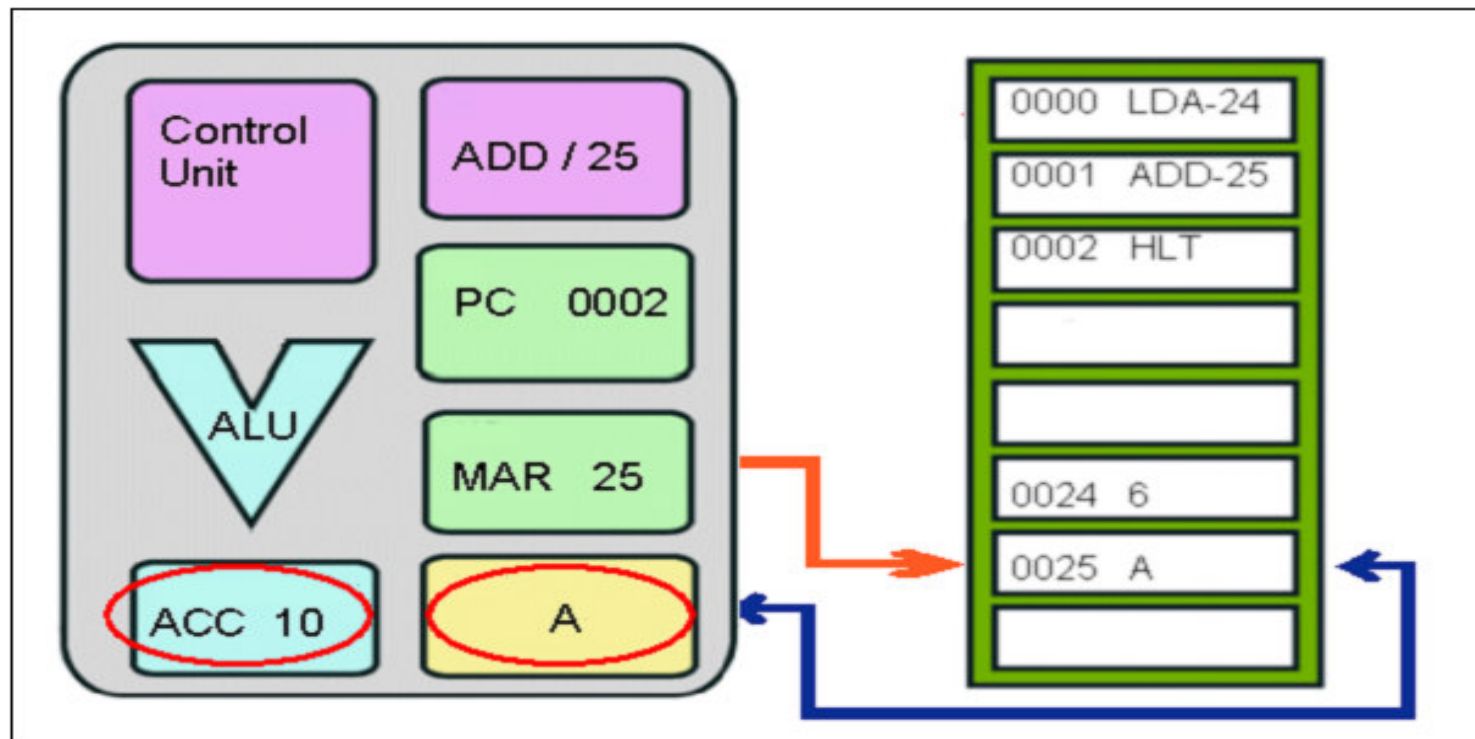
Ciclo di funzionamento: un esempio

execute/11

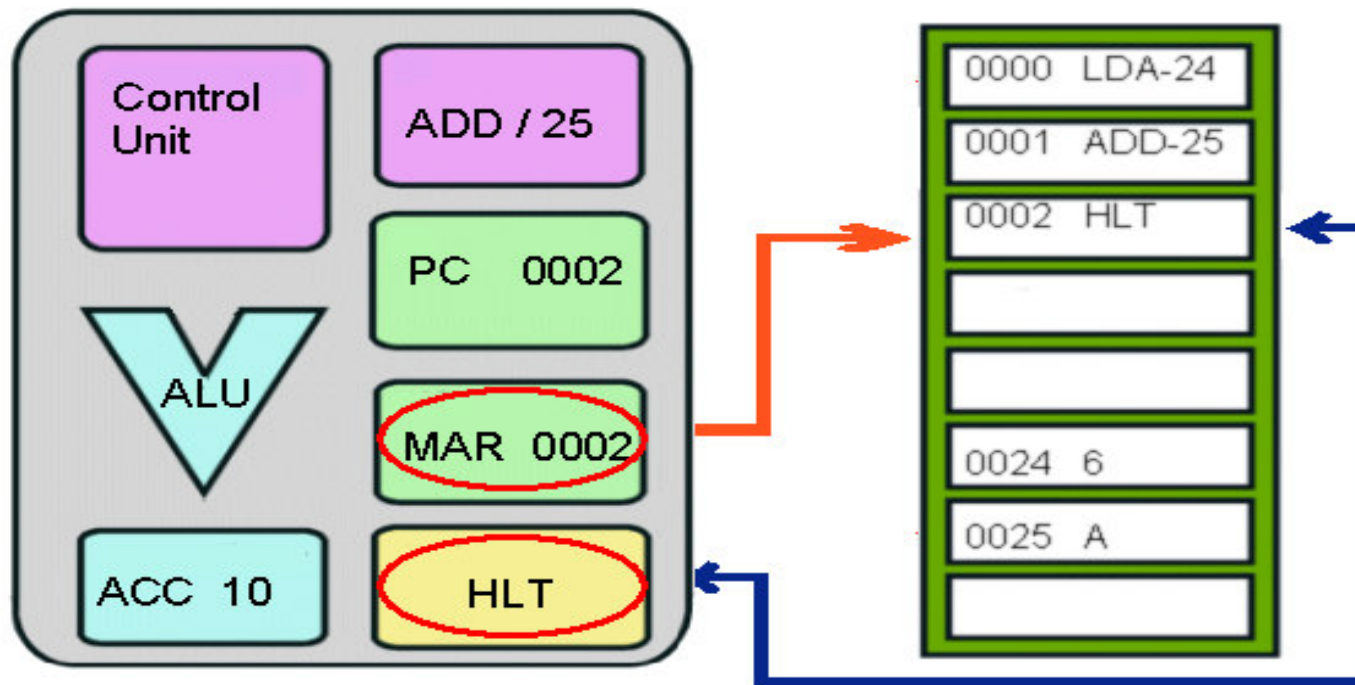


Ciclo di funzionamento: un esempio

execute/12

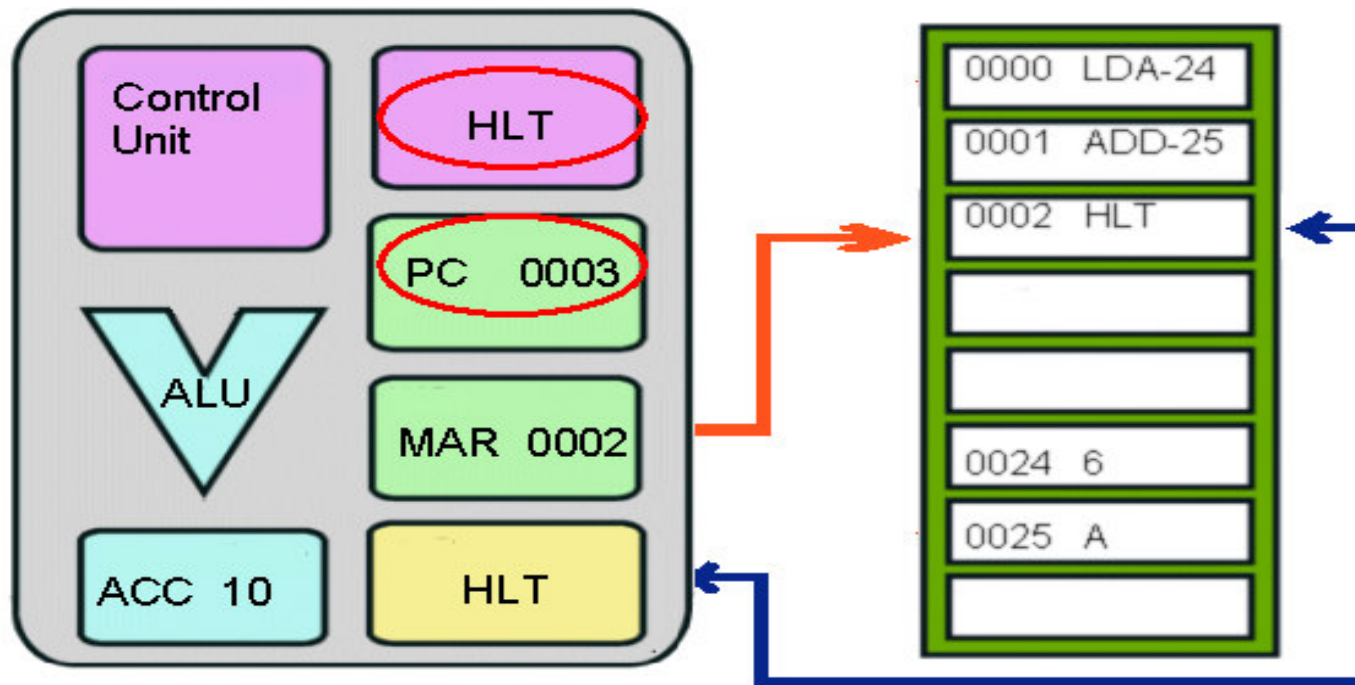


Ciclo di funzionamento: un esempio fetch/13



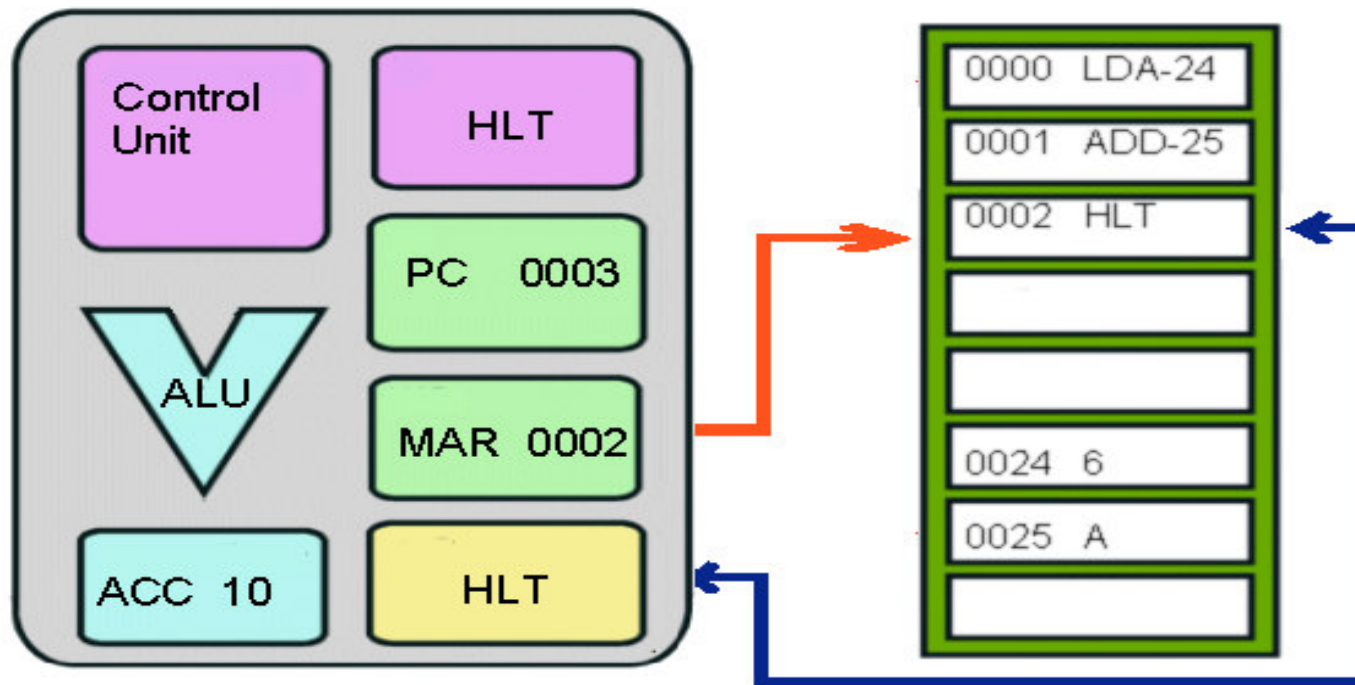
Ciclo di funzionamento: un esempio

decode/14



Ciclo di funzionamento: un esempio

execute/15





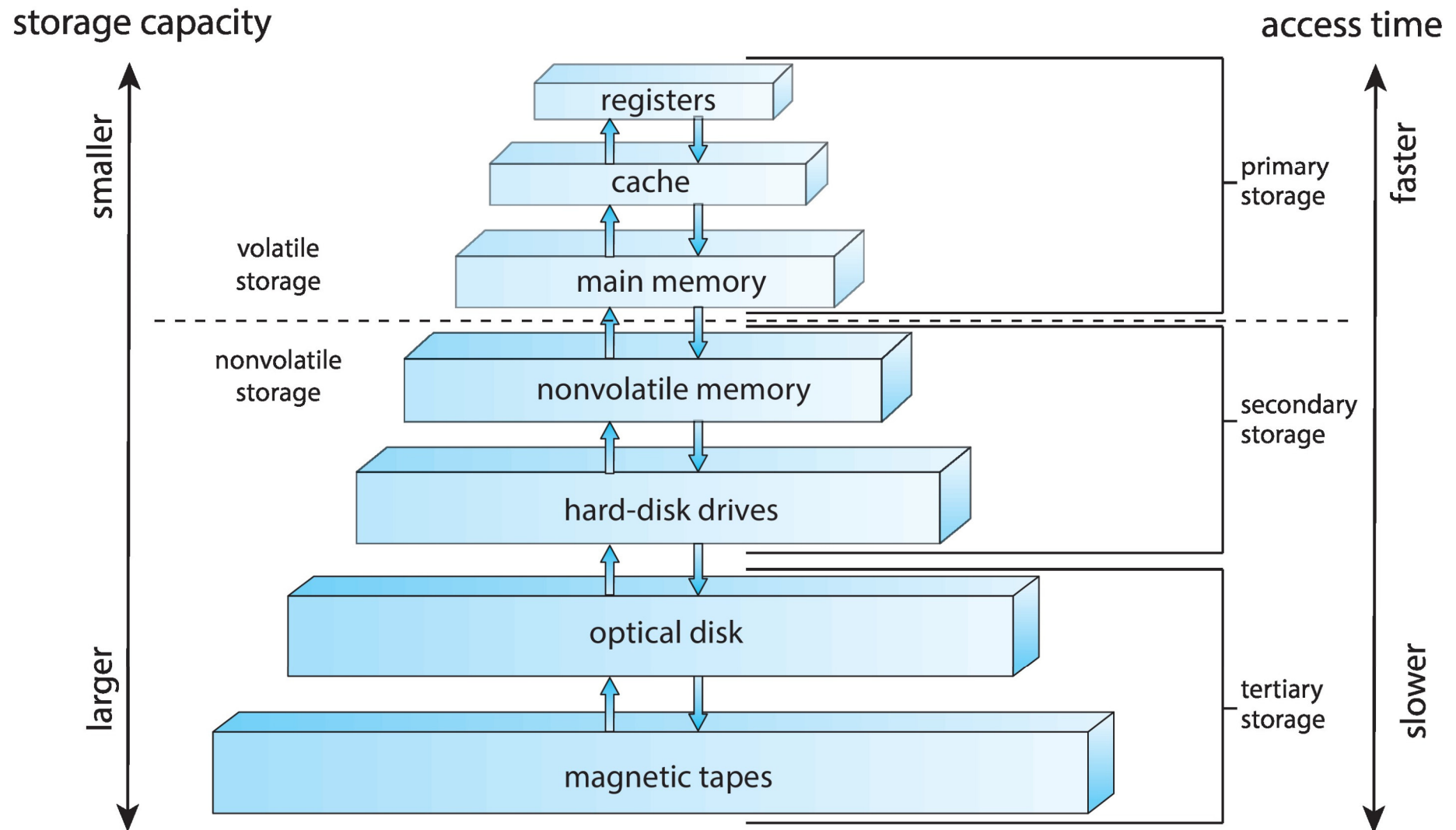
- ✓ CPU e ciclo Fetch-Decode-Execute
- Organizzazione della Memoria
- Organizzazione dei Dispositivi di I/O
- Protezione dell'Hardware

Gerarchia della Memoria

- **Gerarchie delle memorie** - il sistema di memorizzazione è organizzato gerarchicamente sulla base di:
 - Velocità
 - Costo
 - Volatilità
- **Caching** – consiste nel copiare i dati contenuti nella memoria di un certo livello in una memoria di livello superiore (più veloce). Questa serve per tenere i dati che sono stati utilizzati più di recente.
 - Questo richiede una politica di **cache management**.
 - Questo richiede che dati che si trovano simultaneamente in più di un livello devono essere **consistenti** tra loro.

La cache serve per fare il caching della memoria centrale. La memoria centrale può essere vista come la cache della memoria secondaria.

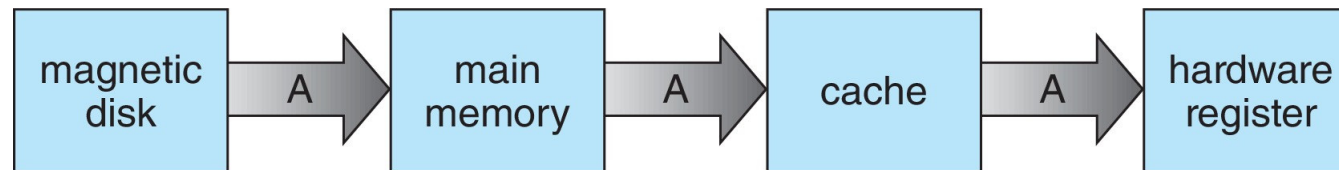
Gerarchia della Memoria



Gerarchia della Memoria

Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

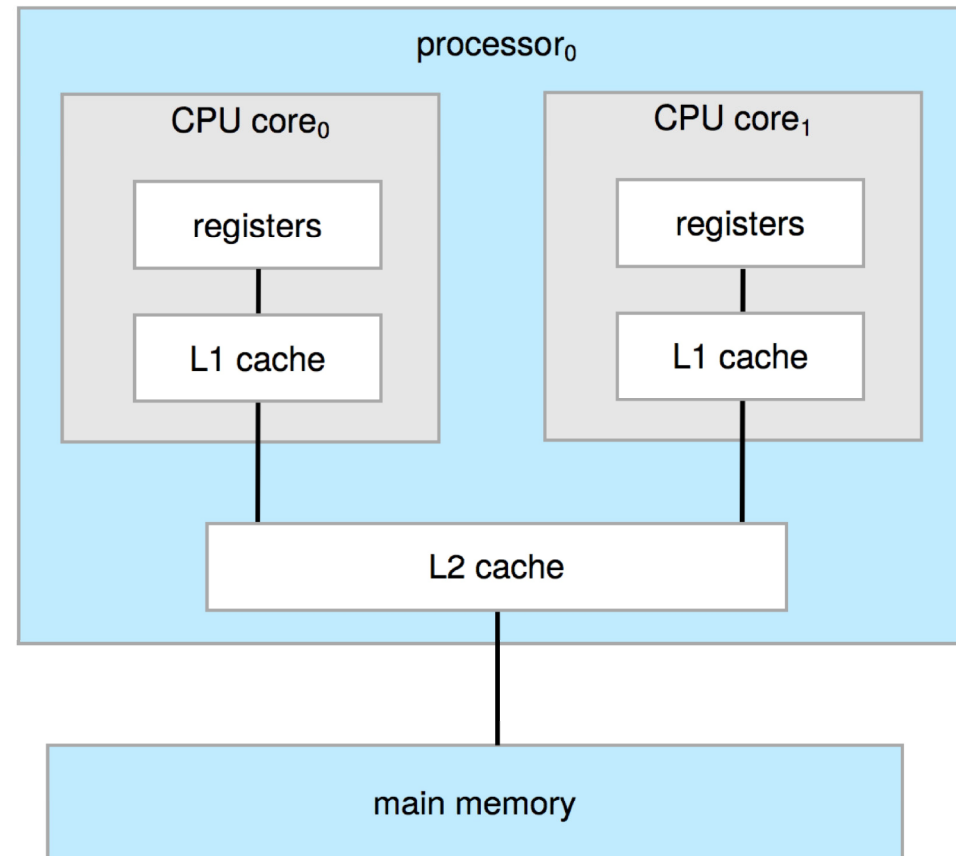
Gerarchia della Memoria



Un po' di chiarezza

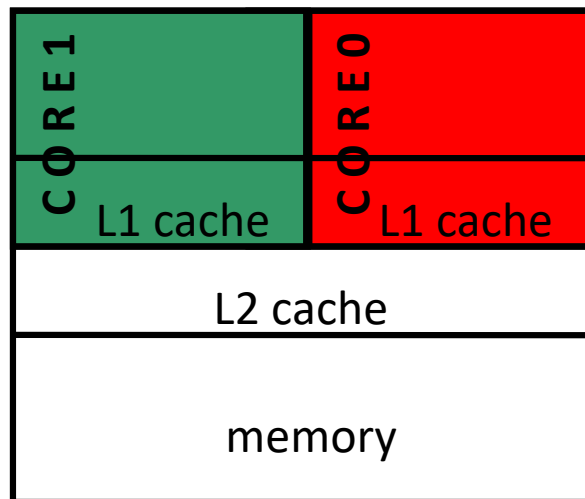
Multipli del byte					
Prefissi SI			Prefissi binari		
Nome	Simbolo	Multiplo	Nome	Simbolo	Multiplo
chilobyte	kB	10^3	kibibyte	KiB	2^{10}
megabyte	MB	10^6	mebibyte	MiB	2^{20}
gigabyte	GB	10^9	gibibyte	GiB	2^{30}
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}
yottabyte	YB	10^{24}	yobibyte	YiB	2^{80}

Multi-Core



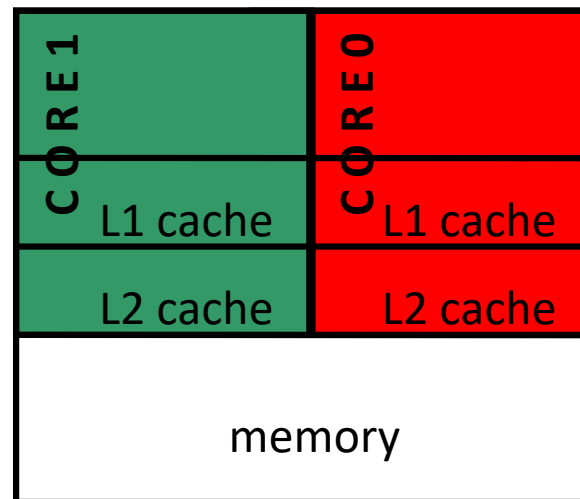
- Cache privata
 - Affinità
 - Minore contesa sul bus
- Cache condivisa
 - Condivisione dati tra core diversi
 - Cache più grandi per thread se ci sono pochi thread

- L1: privata
- L2: condivisa



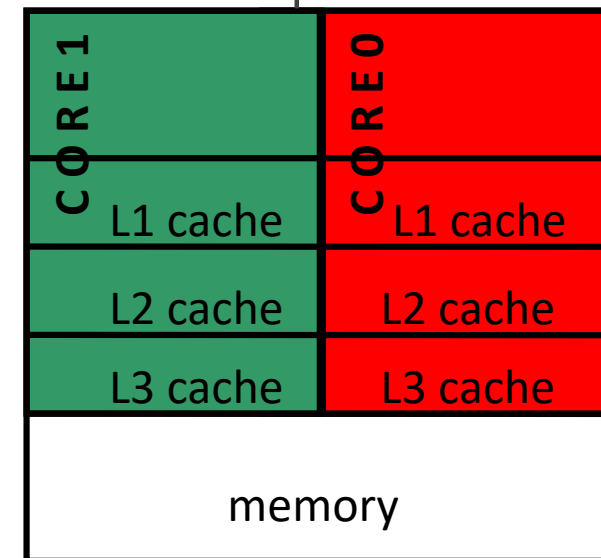
Es. Intel Xeon

- L1: privata
- L2: privata



Es. AMD Opteron

- L1: privata
- L2: privata
- L3: privata

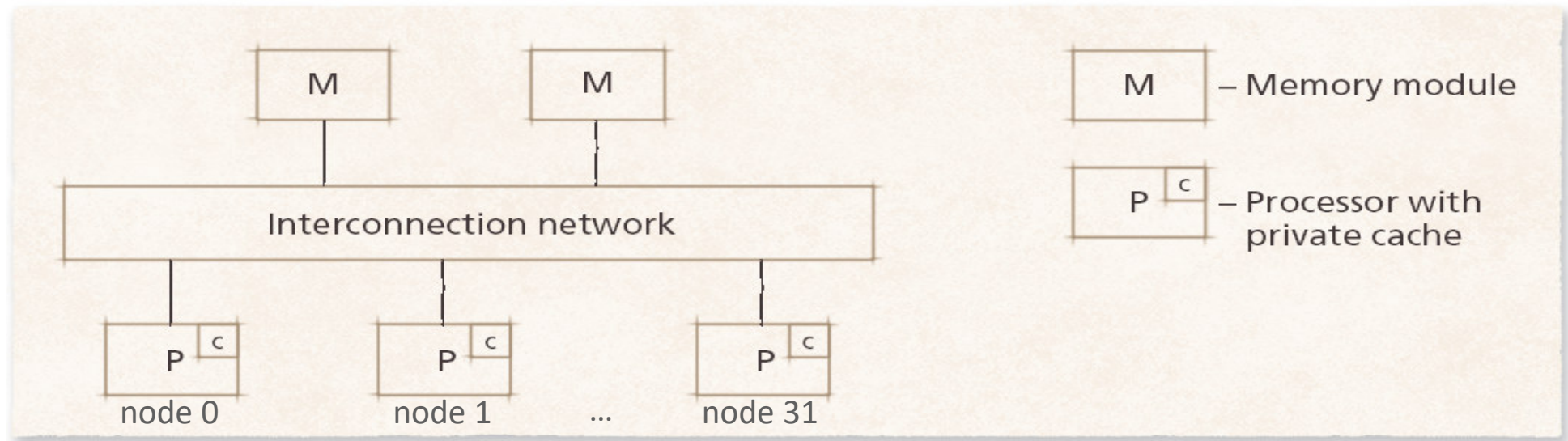


Es. Intel Itanium 2

Multiprocessori: UMA

(Uniform Memory Access)

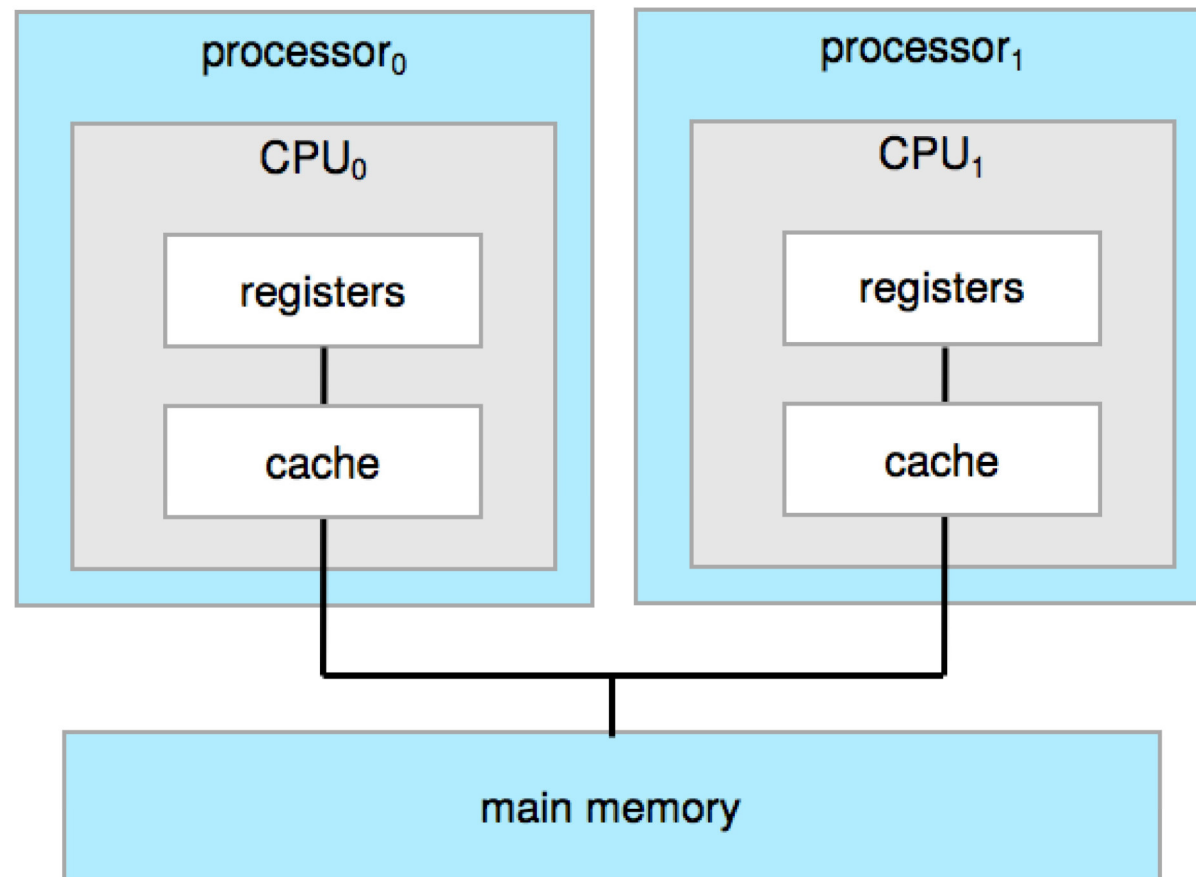
- supportano al max 16-32 CPU



Multiprocessori: UMA

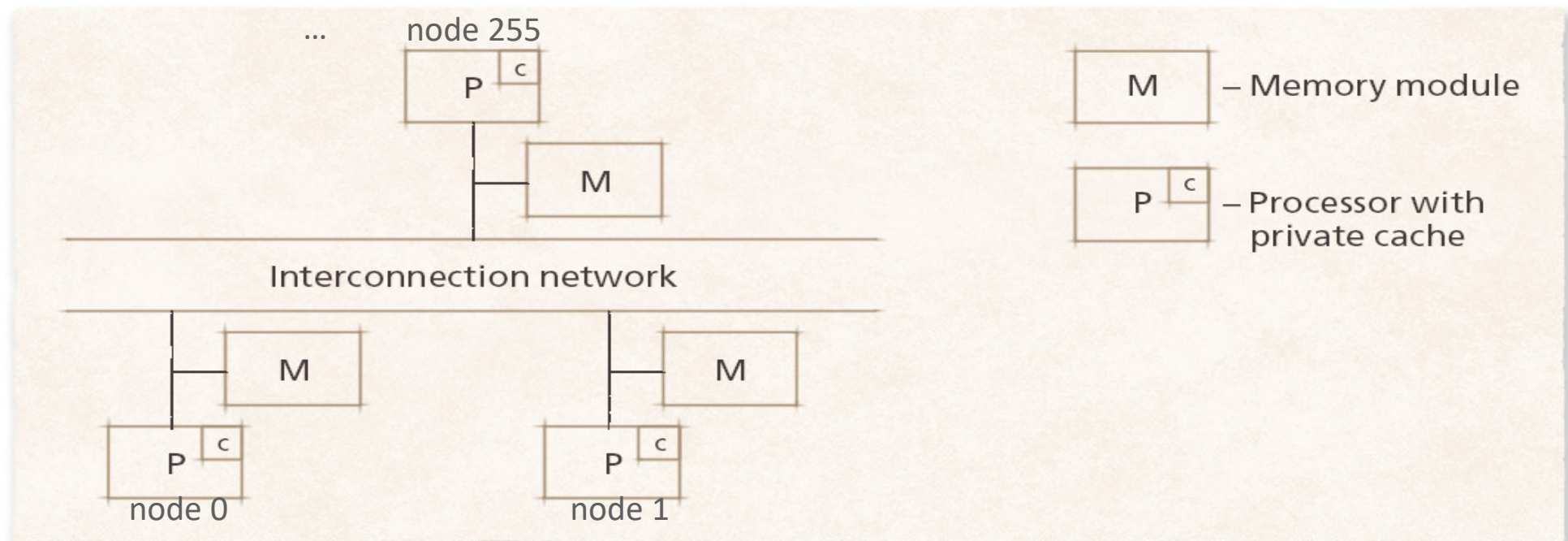
(Uniform Memory Access)

- supportano al max 16-32 CPU



Multiprocessori: NUMA

(Non-Uniform Memory Access)



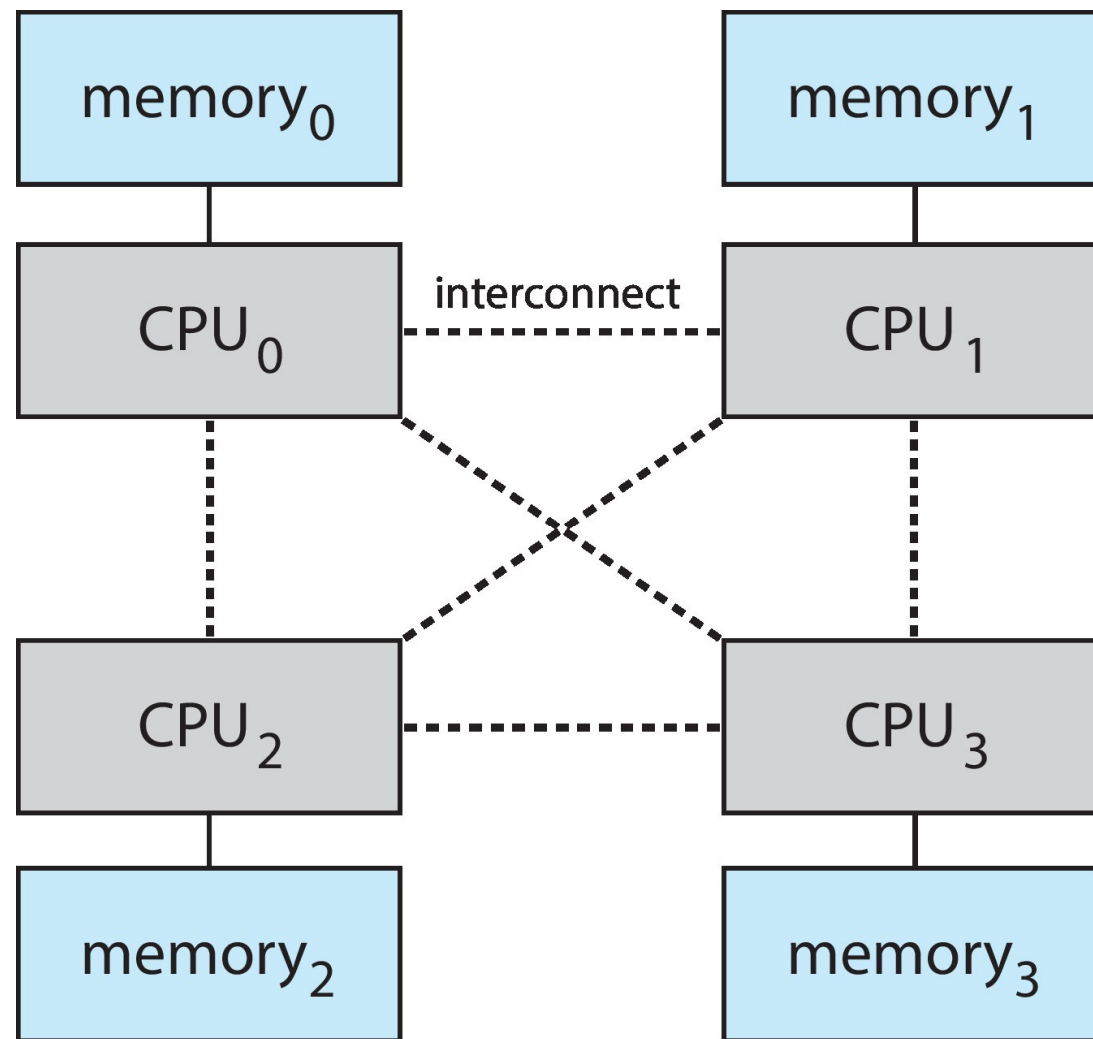
Spazio degli indirizzi unico



Vengono scambiati molti messaggi per mantenere traccia di quale cache contiene un dato blocco.

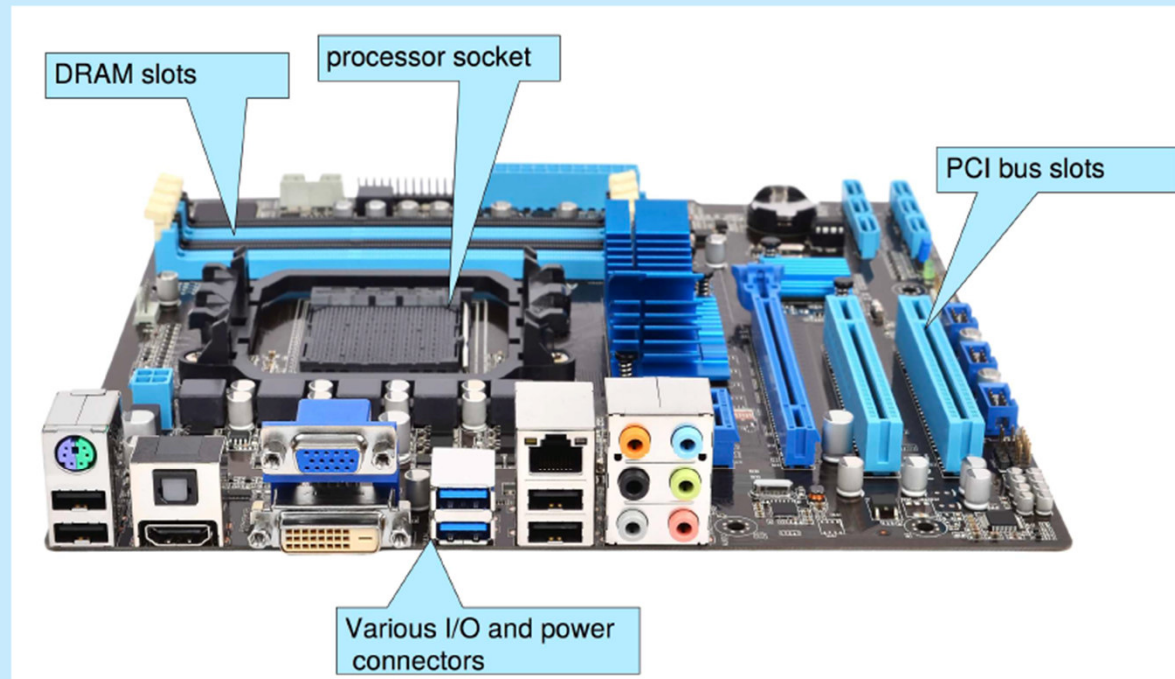
Multiprocessori: NUMA

(Non-Uniform Memory Access)



PC Motherboard

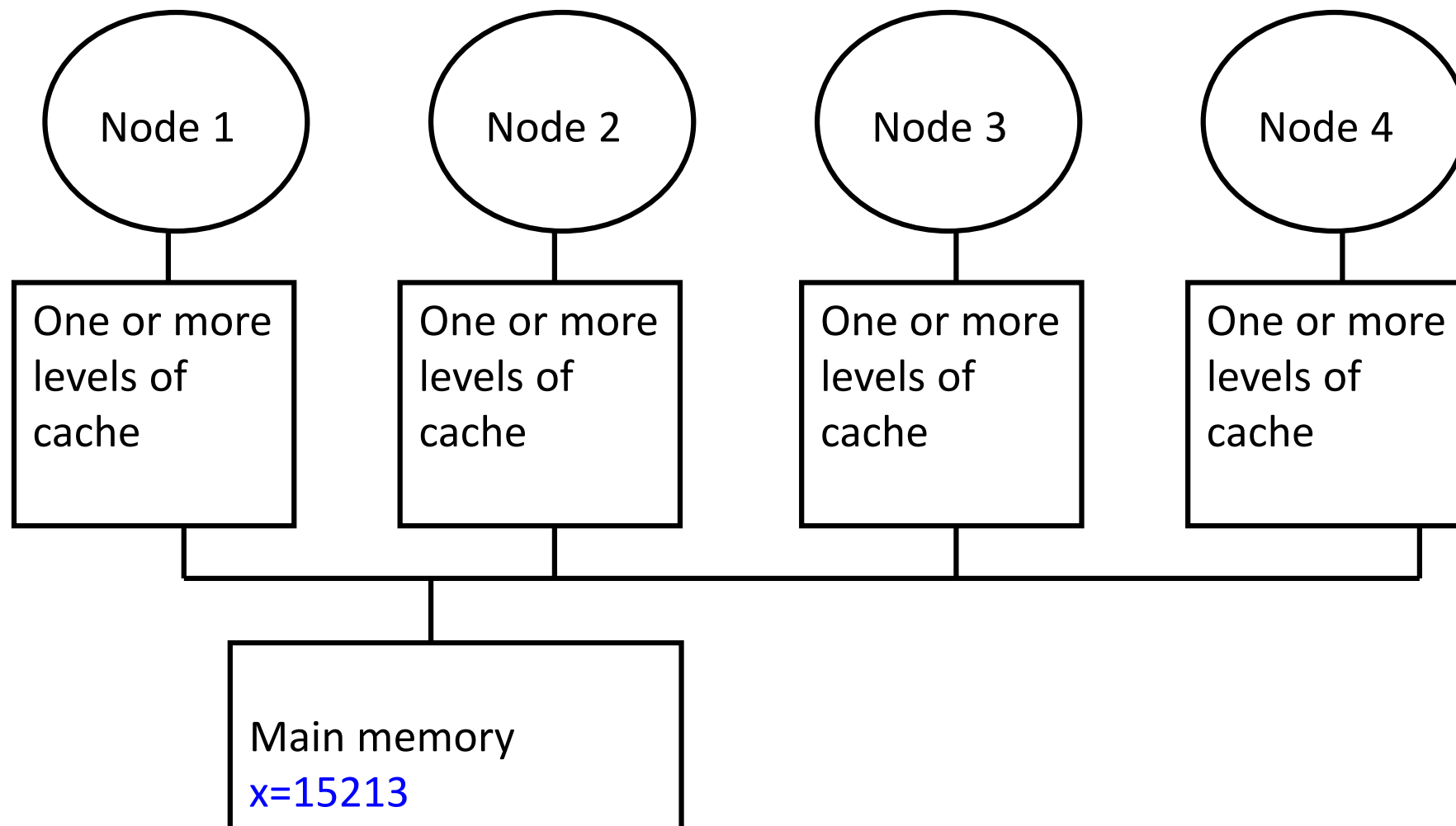
Consider the desktop PC motherboard with a processor socket shown below:



This board is a fully-functioning computer, once its slots are populated. It consists of a processor socket containing a CPU, DRAM sockets, PCIe bus slots, and I/O connectors of various types. Even the lowest-cost general-purpose CPU contains multiple cores. Some motherboards contain multiple processor sockets. More advanced computers allow more than one system board, creating NUMA systems.

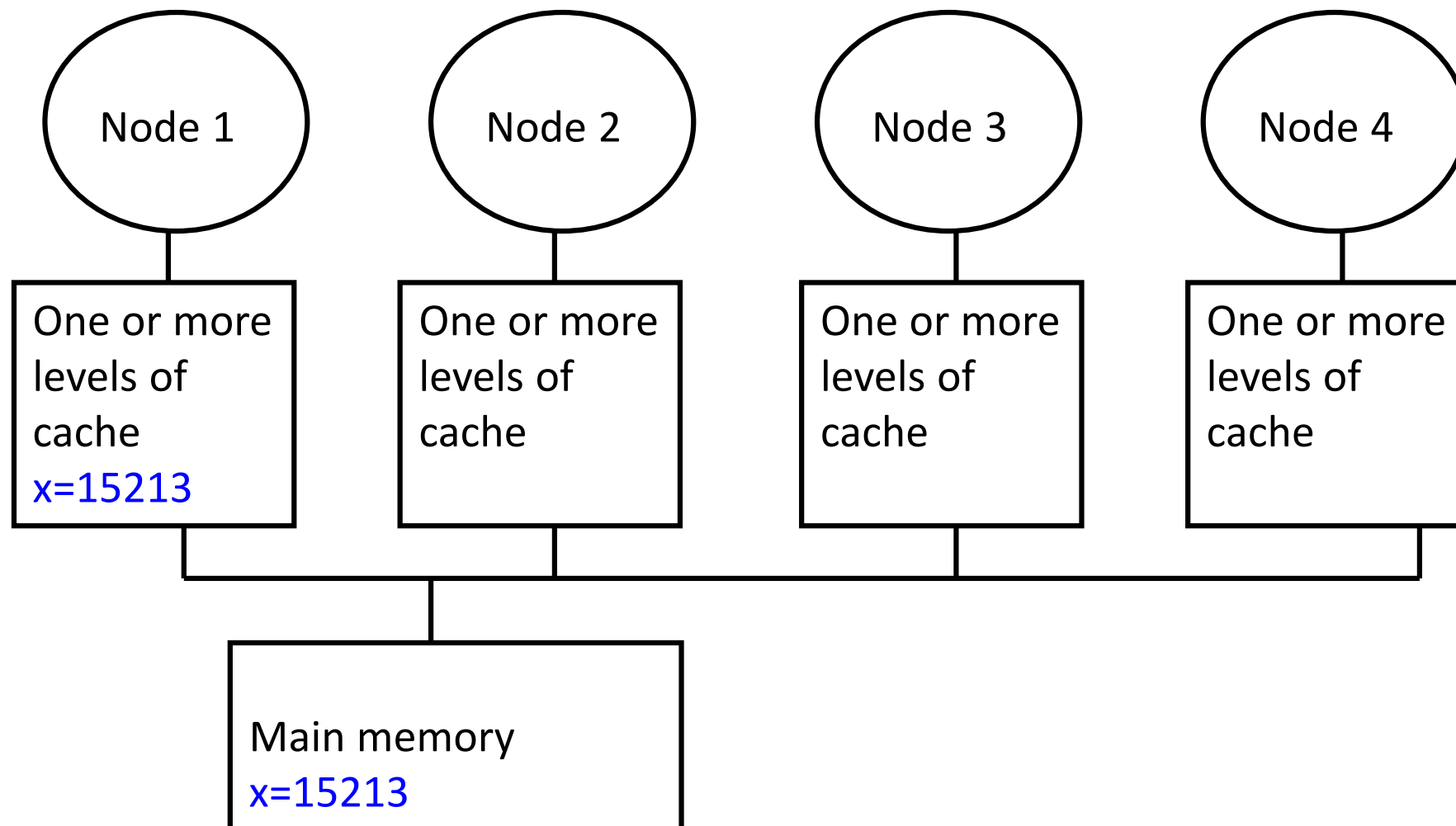
Coerenza della cache: il problema

- Ipotizziamo che x inizialmente contenga 15213



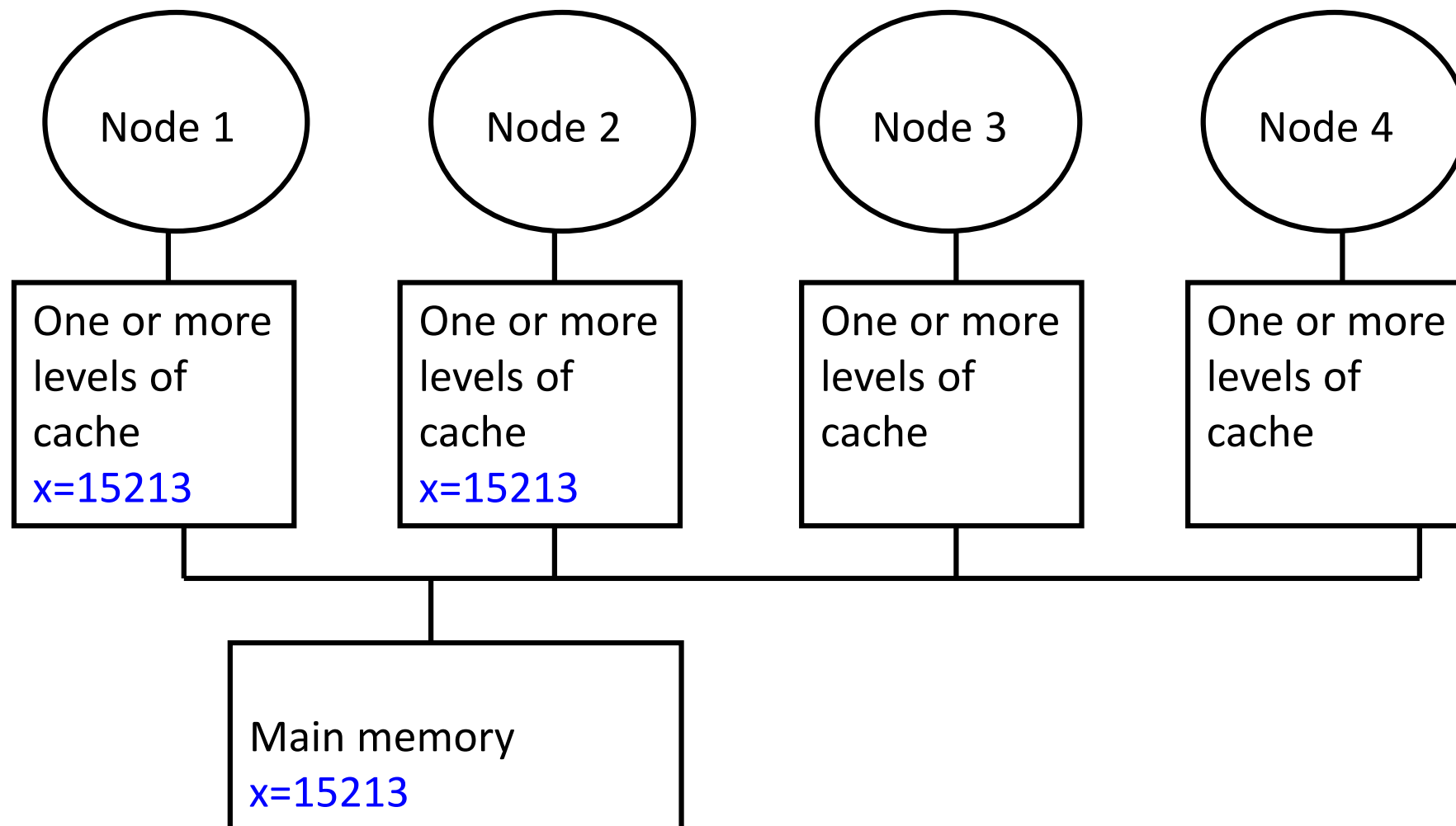
Coerenza della cache: il problema

- Node 1 legge x



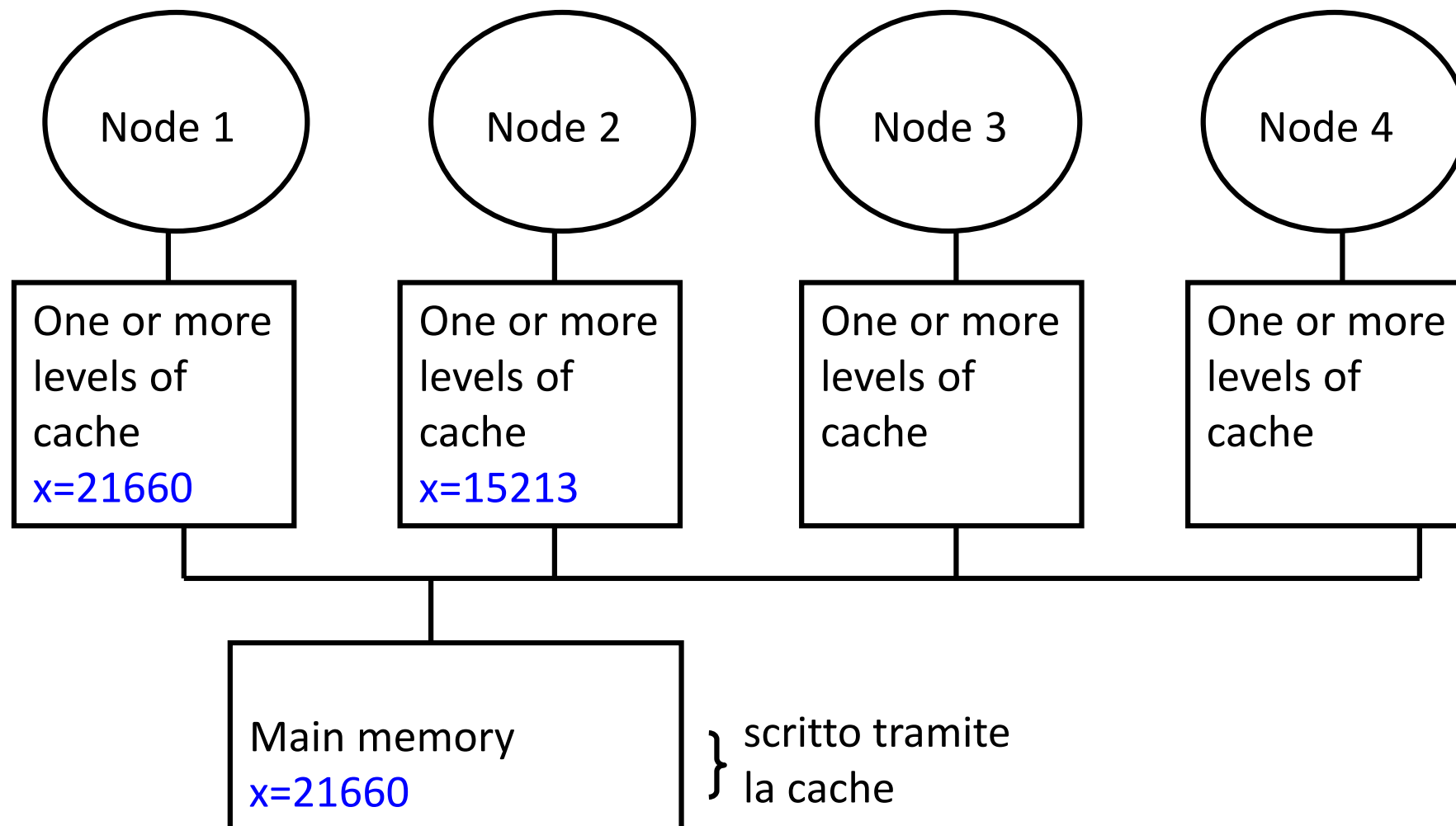
Coerenza della cache: il problema

- Node 2 legge x



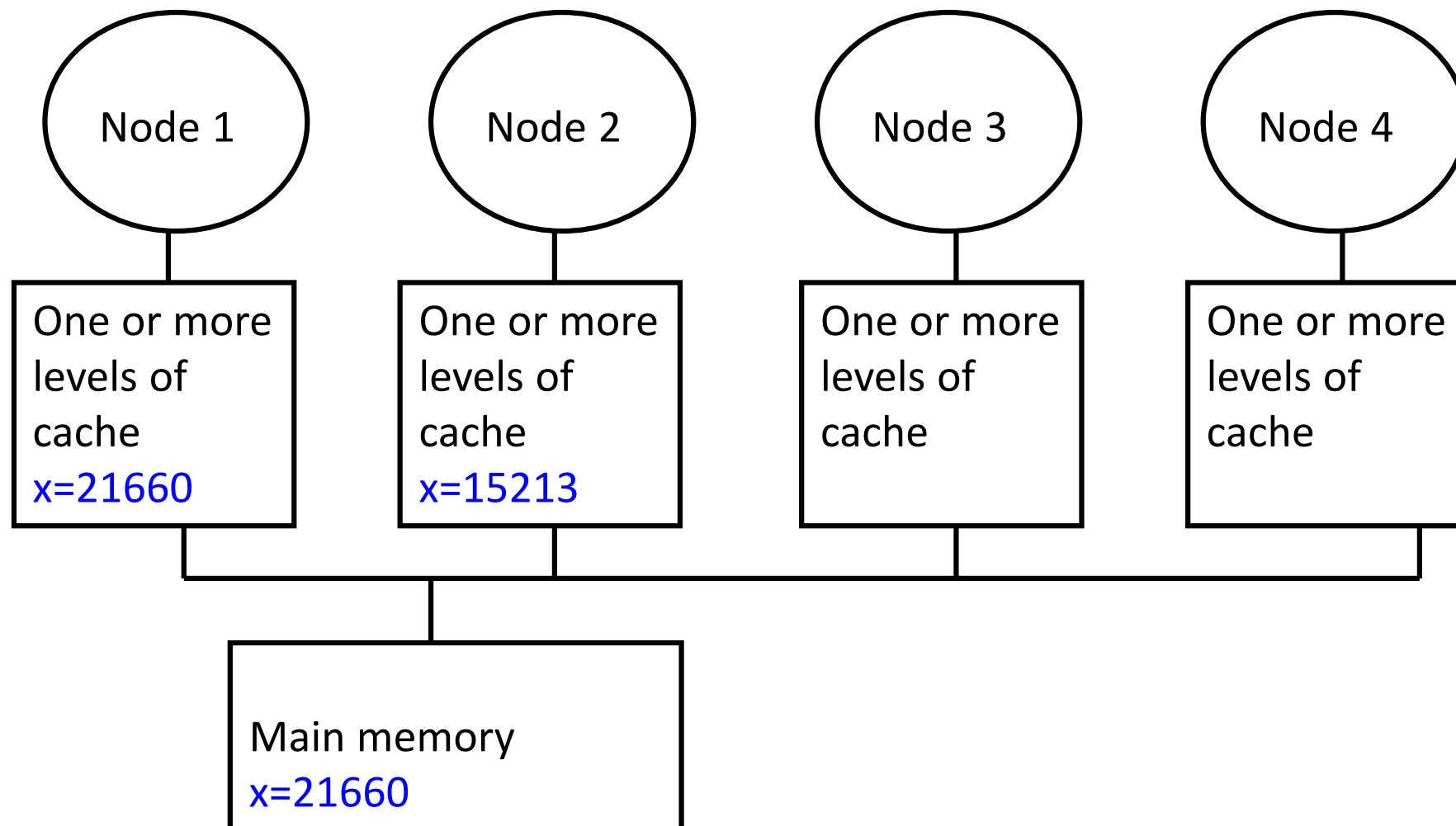
Coerenza della cache: il problema

- Node 1 scrive su x, ponendola a 21660



Coerenza della cache: il problema

- Node 2 legge x ... ma ottiene un valore obsoleto



Coerenza della cache

Meccanismi di coerenza

Indica quali meccanismi adottare per conoscere lo stato di un dato.

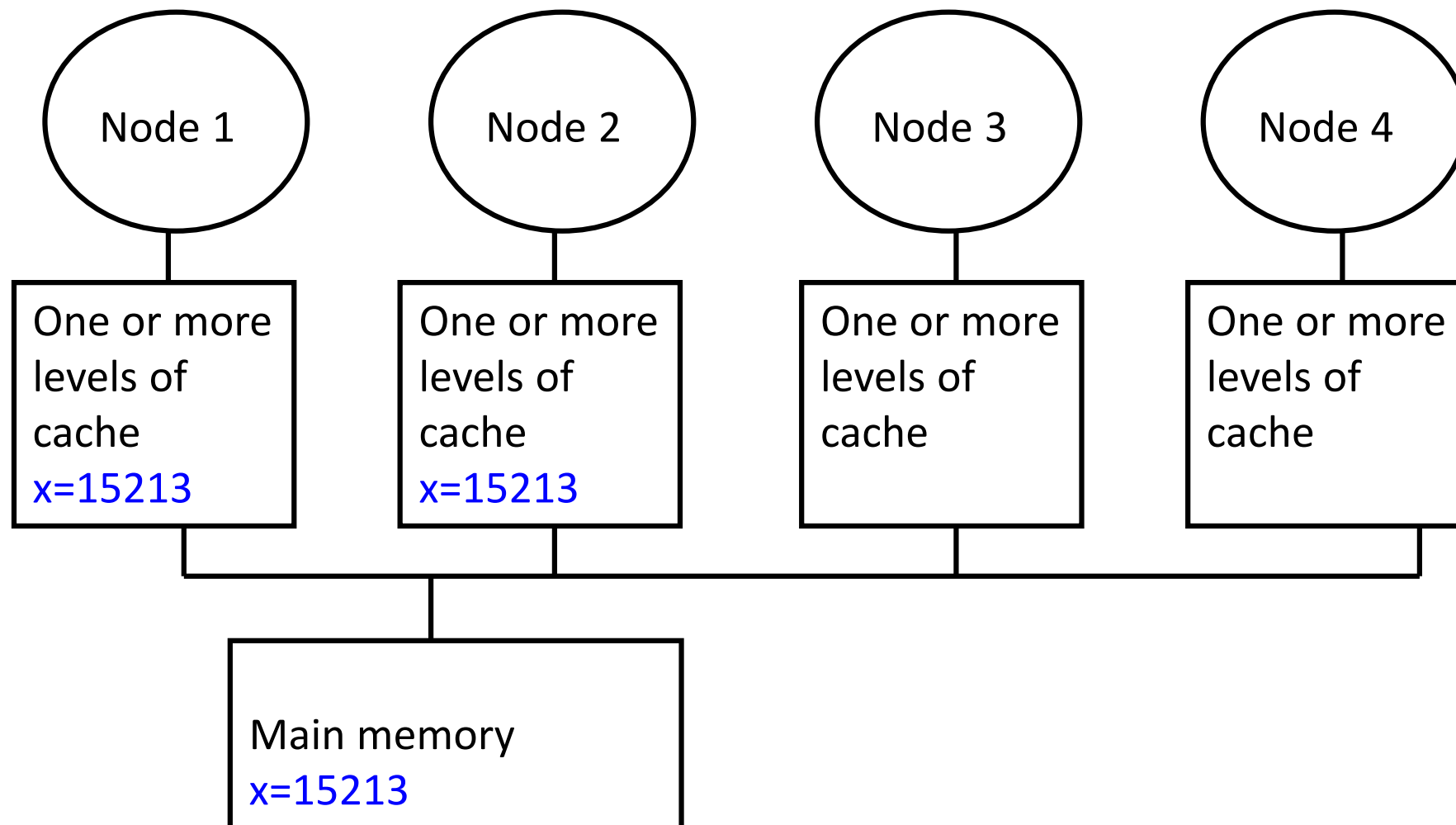
- Una cache per ogni dato ➡ UMA e NUMA
 - Un solo processore alla volta può avere in cache un particolare dato
- Bus snooping ➡ UMA e NUMA
 - Ogni cache controller spia il bus per cambiare lo stato dei dati nella sua cache
- Centralized directory ➡ UMA e NUMA
 - Il sistema mantiene una directory centralizzata dove per ogni dato è indicato in quali cache è copiato e quale è il suo stato
- Home-based directory ➡ NUMA
 - Il sistema mantiene una directory per ogni nodo, dove per ogni dato della memoria locale del nodo è indicato in quali cache è copiato e quale è il suo stato

- Ogni nodo associa ad ogni blocco di memoria uno stato che indica la disponibilità del blocco stesso per quel nodo
 - Il controllore di ogni cache si deve attenere al protocollo scelto:
 - osservando gli **eventi** – richieste che provengono dal proprio processore o richieste che transitano sul bus (richieste che provengono da altri nodi)
 - eseguendo **azioni** sul bus (che verranno osservate dagli altri controllori)
 - cambiando lo **stato** locale del blocco interessato (un blocco può trovarsi in stati diversi su nodi diversi)
- Protocolli basati sull'invalidazione:
 - quando un blocco viene modificato da un nodo, lo stato di quel blocco negli altri nodi è settato ad invalido.
 - Es. MSI e MESI
- Protocolli basati sull'aggiornamento:
 - quando un processore modifica una locazione condivisa il suo valore viene aggiornato nelle cache di tutti i processori che possiedono quel blocco;
 - per questo motivo i protocolli basati sull'aggiornamento hanno un miss-rate più basso ma generano più traffico nel bus, risultando più complessi nella gestione.
 - DRAGON

Coerenza della cache

Protocolli basati sull'Invalidazione

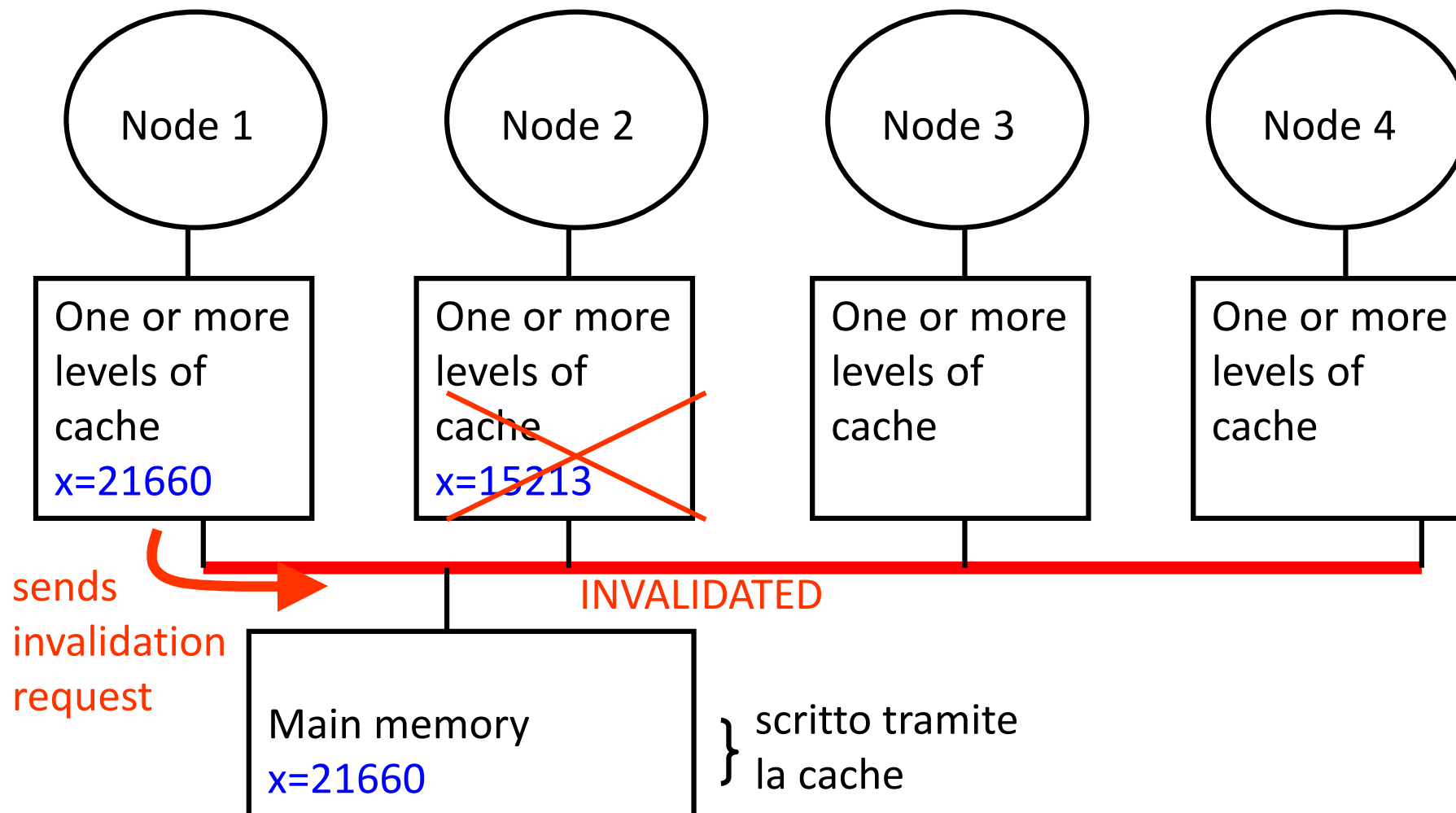
- Ripartiamo da quando sia Node 1 e Node 2 hanno letto x



Coerenza della cache

Protocolli basati sull'Invalidazione

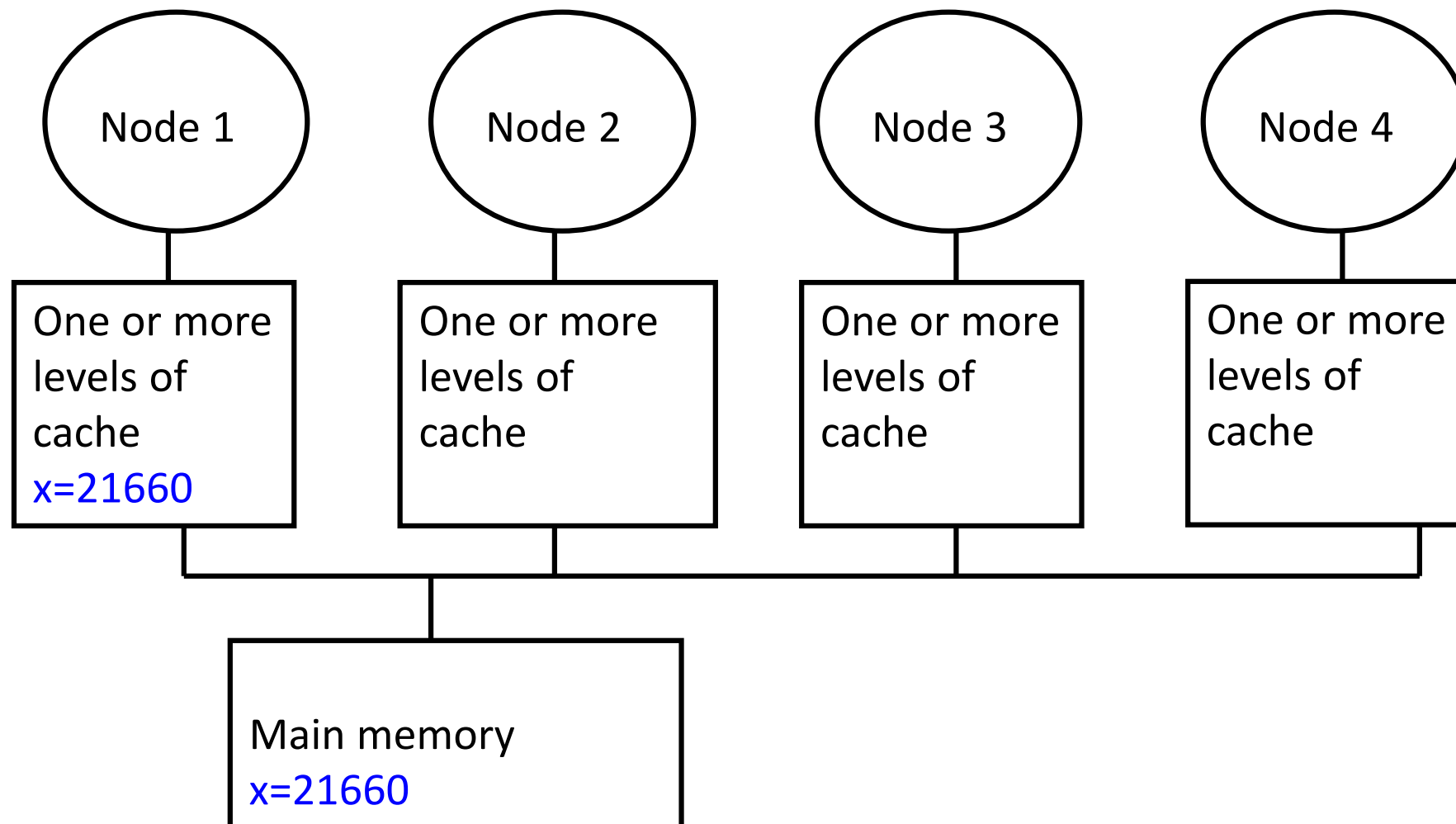
- Node 1 scrive su x, ponendola a 21660



Coerenza della cache

Protocolli basati sull'Invalidazione

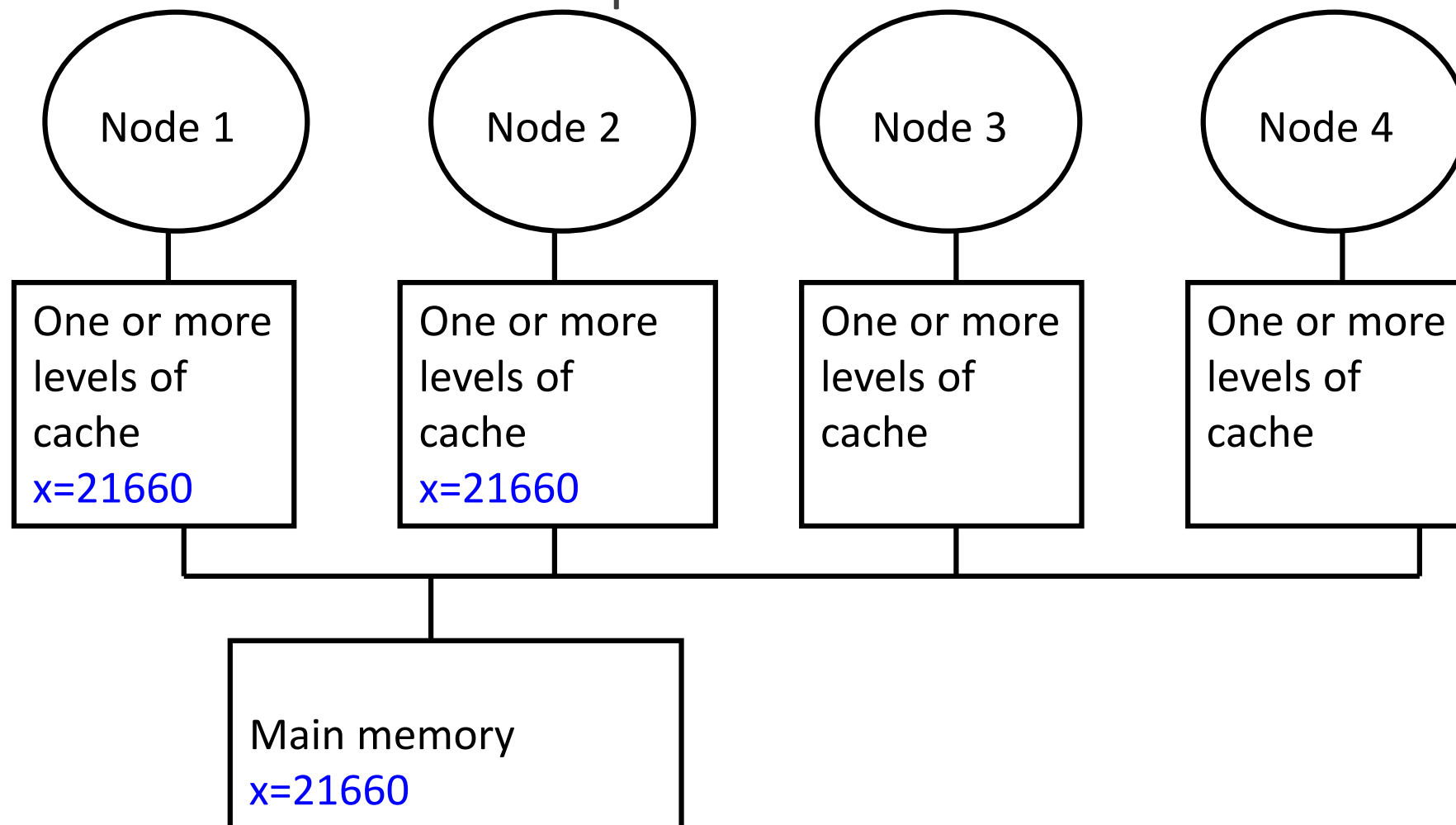
■ Dopo



Coerenza della cache

Protocolli basati sull'Invalidazione

- Nodo 2 legge x. ➡ Cache miss.
- Nodo 2 carica nuova copia.



Diagrammi di stato

I protocolli possono essere formalizzati mediante i diagrammi di stato



Semantica:

Se (è stato osservato l'evento *e*) **e** (la guardia *g* è vera) **allora**

- viene eseguita l'azione *a* ;
- il sistema transita dallo stato s1 allo stato s2 ;

Coerenza della cache

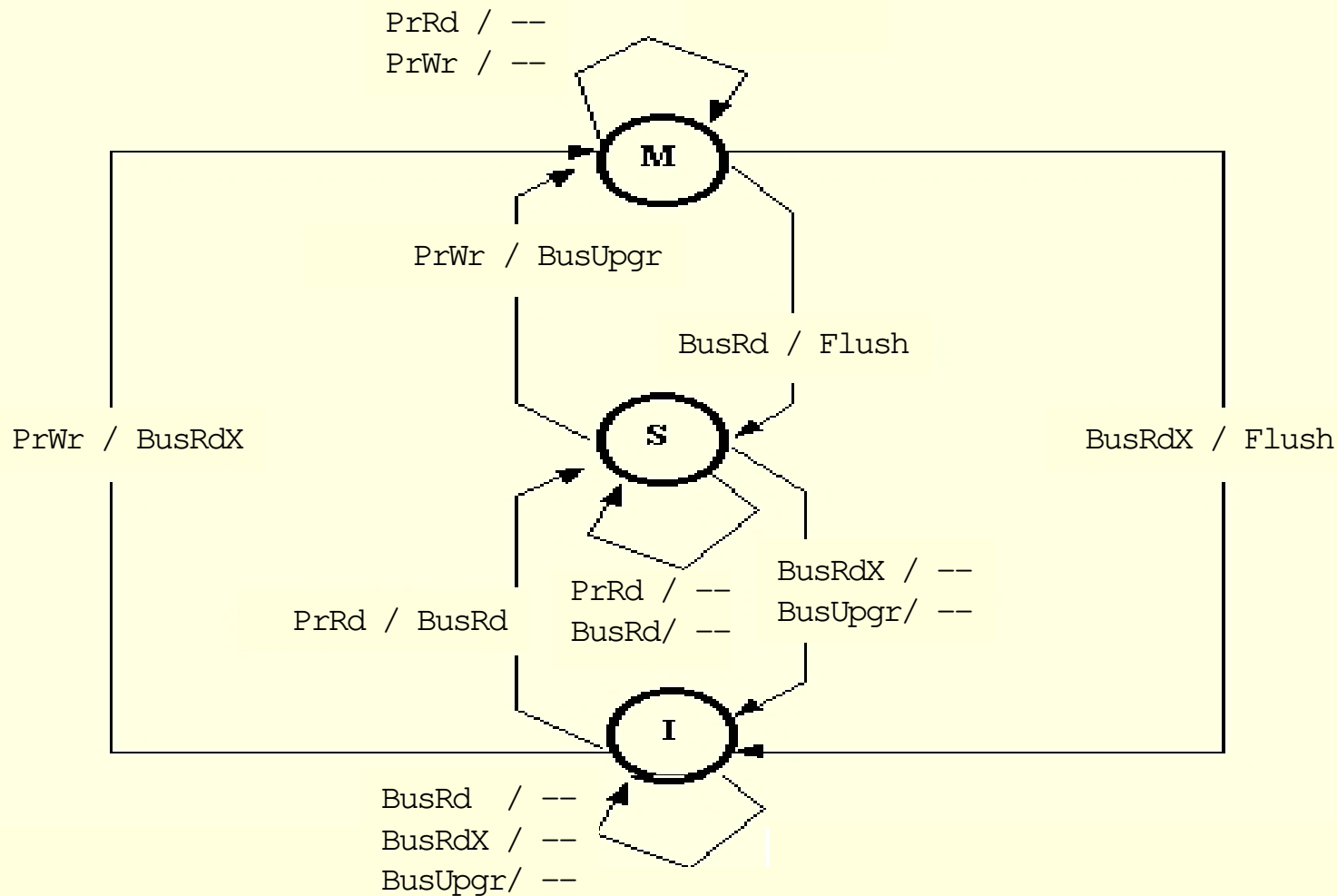
Protocolli basati sull'Invalidazione

- Richieste del processore:
 - PrRd: lettura;
 - PrWr: scrittura.
- Azioni sul bus:
 - BusRd: quando si verifica una lettura mancata nella cache di un processore, il controllore della sua cache invia l'indirizzo al bus e chiede una copia del blocco che non intende modificare;
 - BusRdX: quando si verifica una scrittura mancata nella cache di un processore, il controllore della cache invia l'indirizzo al bus e chiede una copia del blocco che intende modificare, richiesta che invalida il blocco nelle cache degli altri processori;
 - BusUpgr: quando c'è un hit di scrittura nella cache di un processore, invia una richiesta BusUpgr sul bus per invalidare il blocco nelle cache degli altri processori.
 - Flush: un intero blocco della cache viene inviato attraverso il bus, viene ricevuto dal controller della cache che lo ha richiesto e dal controller della memoria centrale

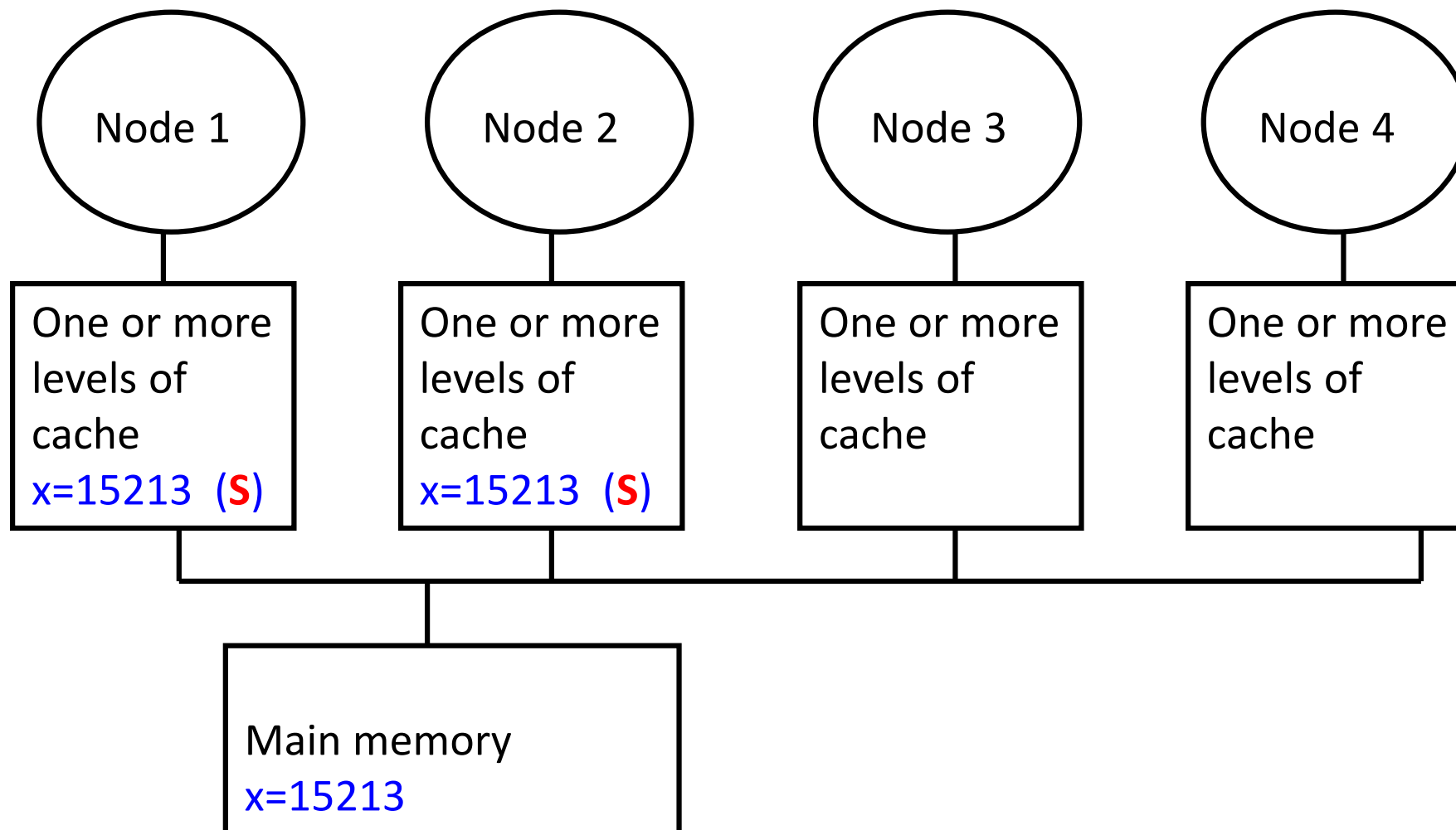
- **Ogni controllore** della cache segue questo protocollo di coerenza
- **Stati**
Il protocollo utilizza tre stati per identificare la situazione di un blocco di memoria in una cache:
 - **I** (invalid): il blocco di memoria non è valido;
 - **S** (shared valid): il blocco è presente in uno stato non modificato in una o più cache; le copie in memoria centrale e nelle altre caches in S sono aggiornate;
 - **M** (modified o dirty): solo un processore ha una copia valida del blocco nella sua cache; la copia in memoria centrale è obsoleta e nessun'altra cache ha una copia valida.
- **Eventi**
 - PrRd, PrWr, BusRd, BusRdX, BusUpgr
- **Guardie**
 - In questo protocollo non vengono utilizzate guardie
- **Azioni**
 - BusRd, BusRdX, BusUpgr, Flush

Coerenza della cache Protocollo MSI

Cache *i*



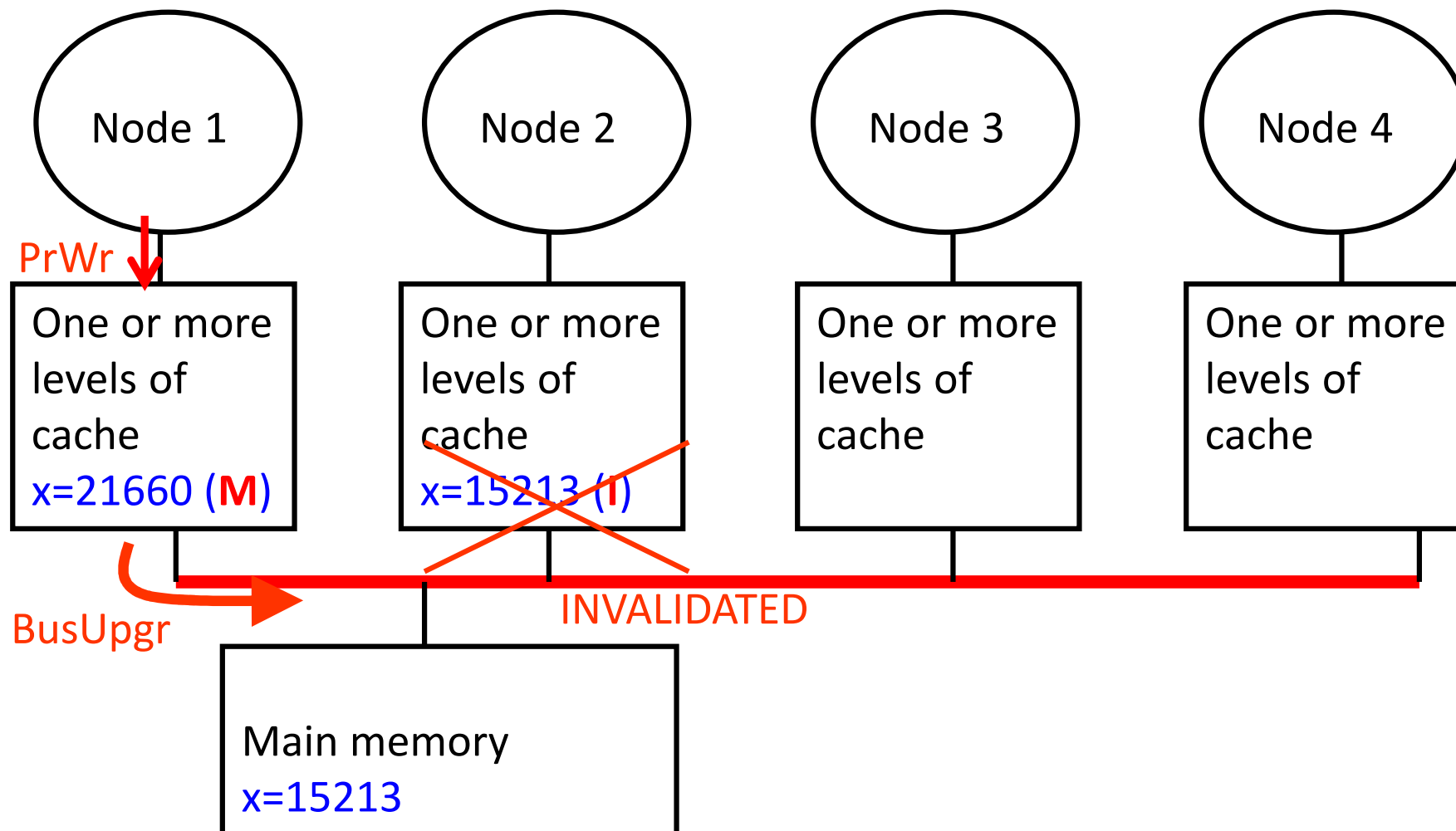
- Ripartiamo da quando sia Node 1 e Node 2 hanno letto x



Coerenza della cache

Protocollo MSI

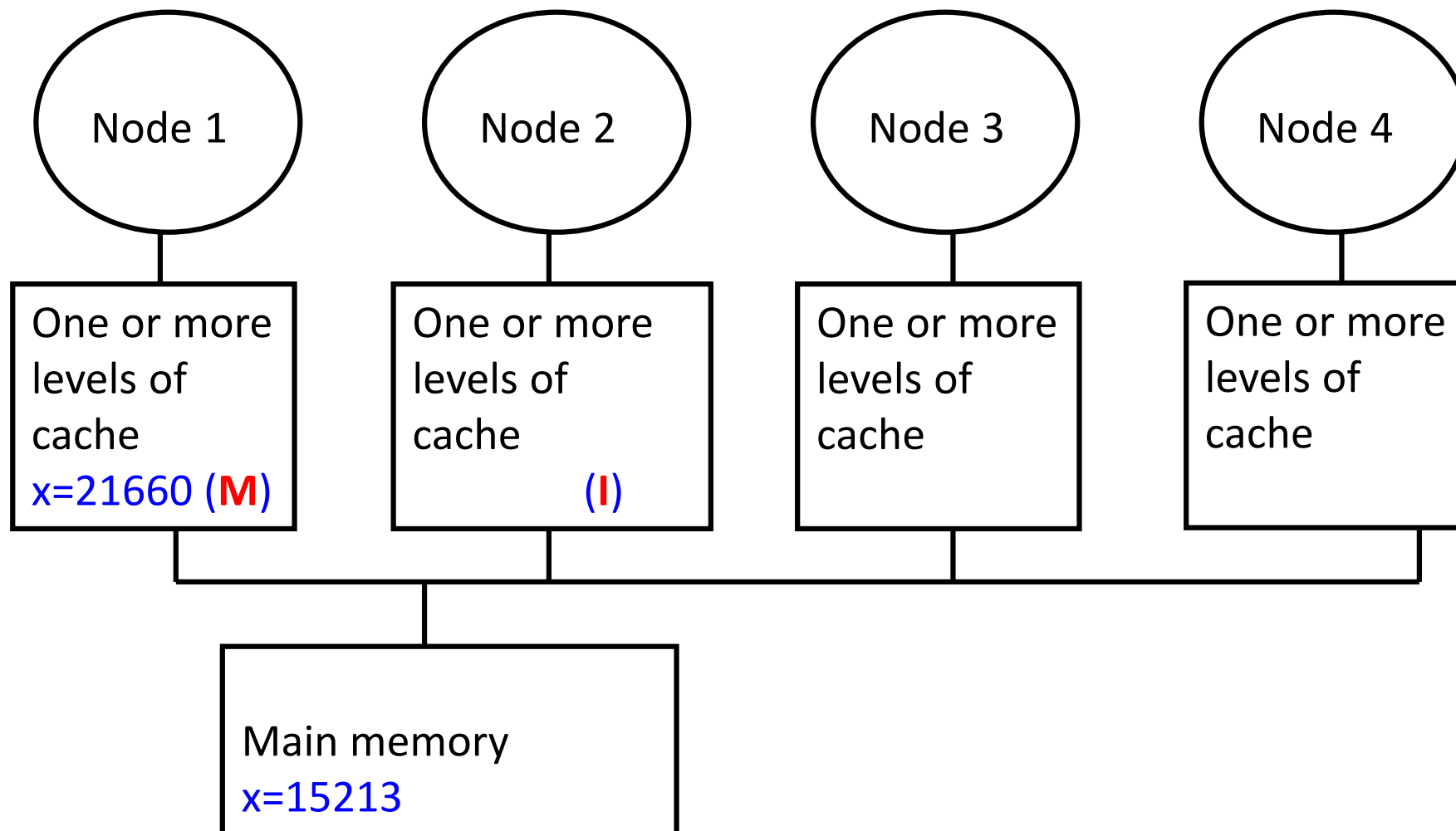
- Node 1 scrive su x, ponendola a 21660



Coerenza della cache

Protocollo MSI

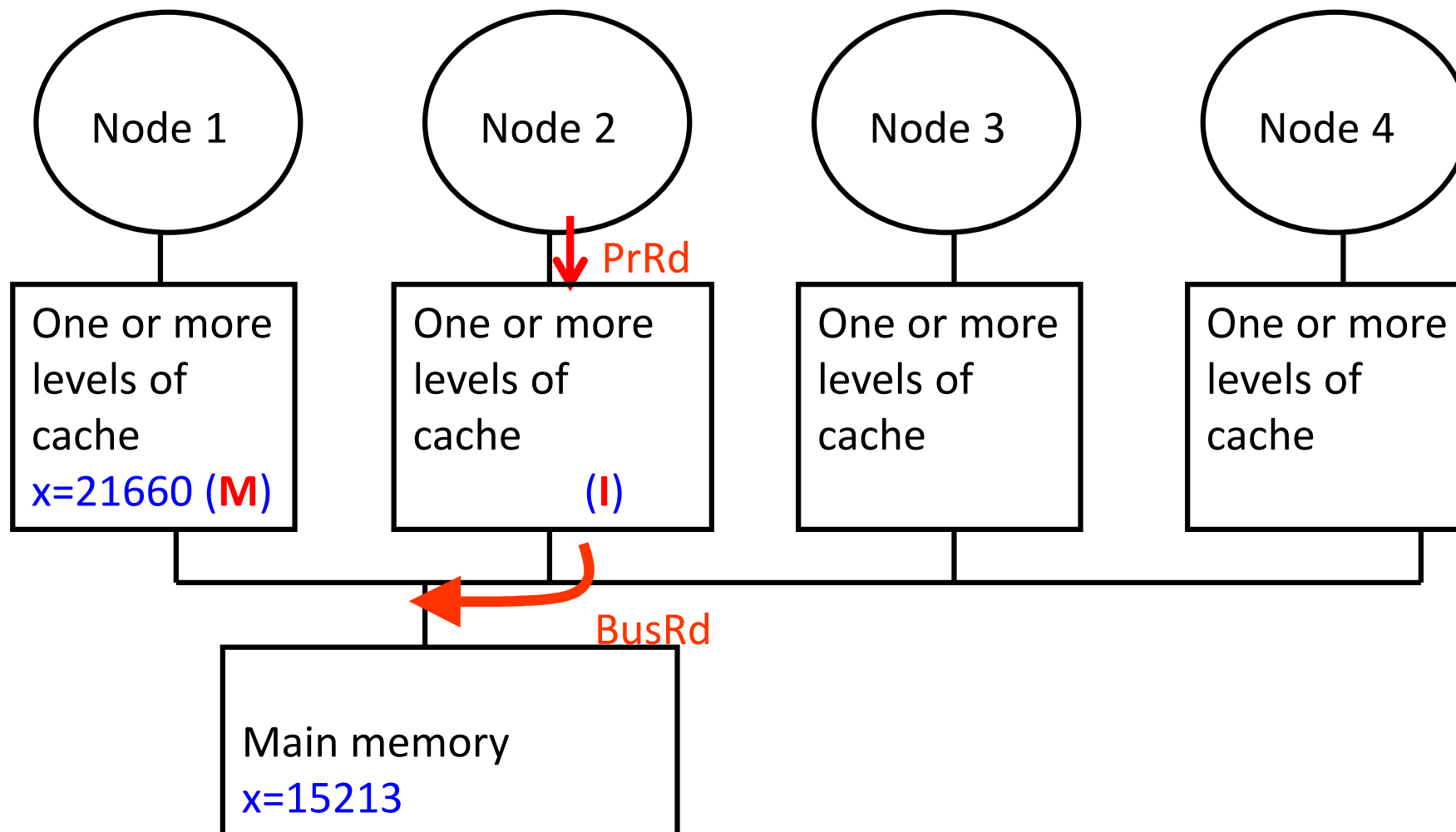
■ Dopo



Coerenza della cache

Protocollo MSI

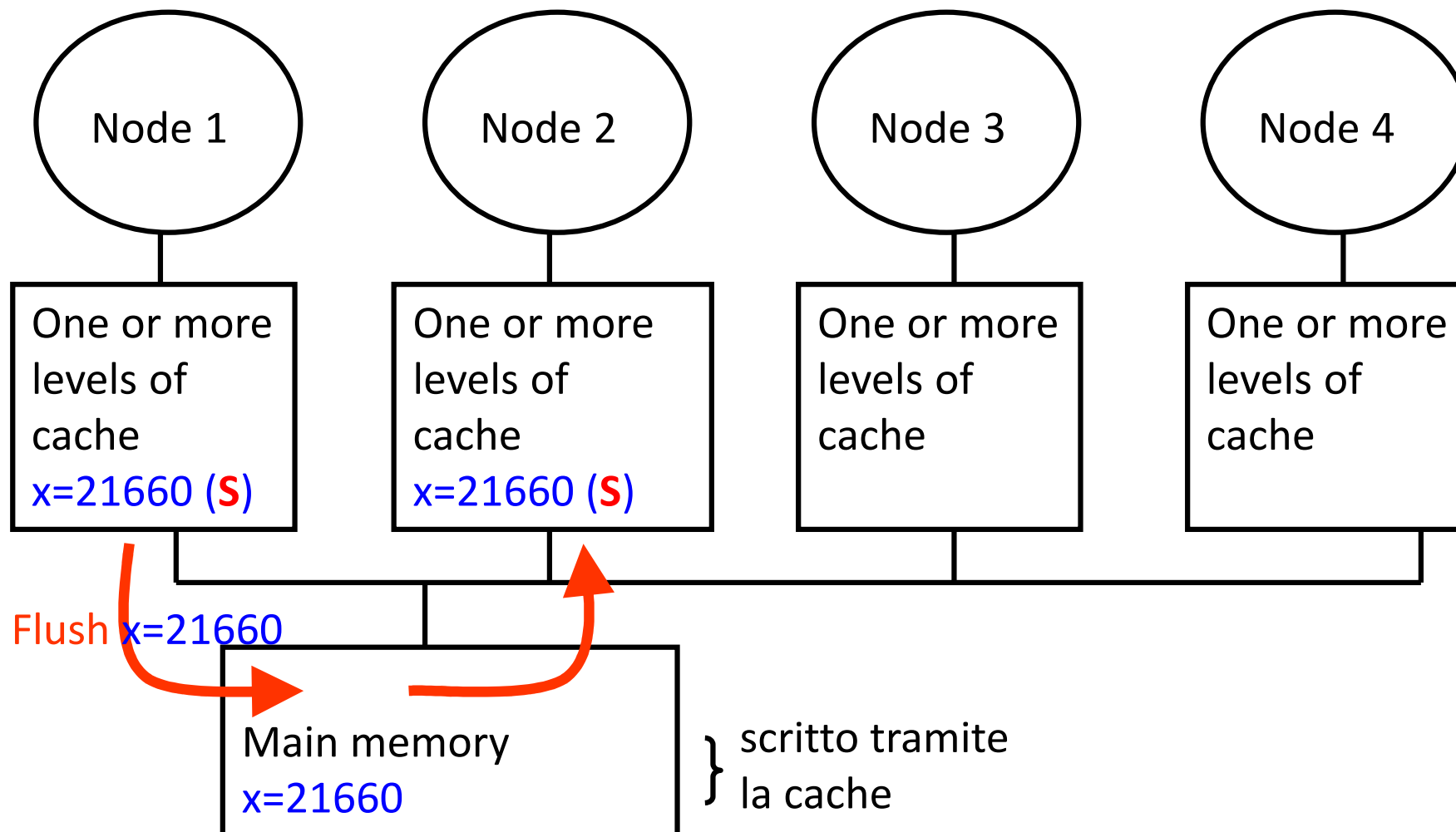
- Nodo 2 legge x. ➡ Cache miss.



Coerenza della cache

Protocollo MSI

- Nodo 2 carica nuova copia.

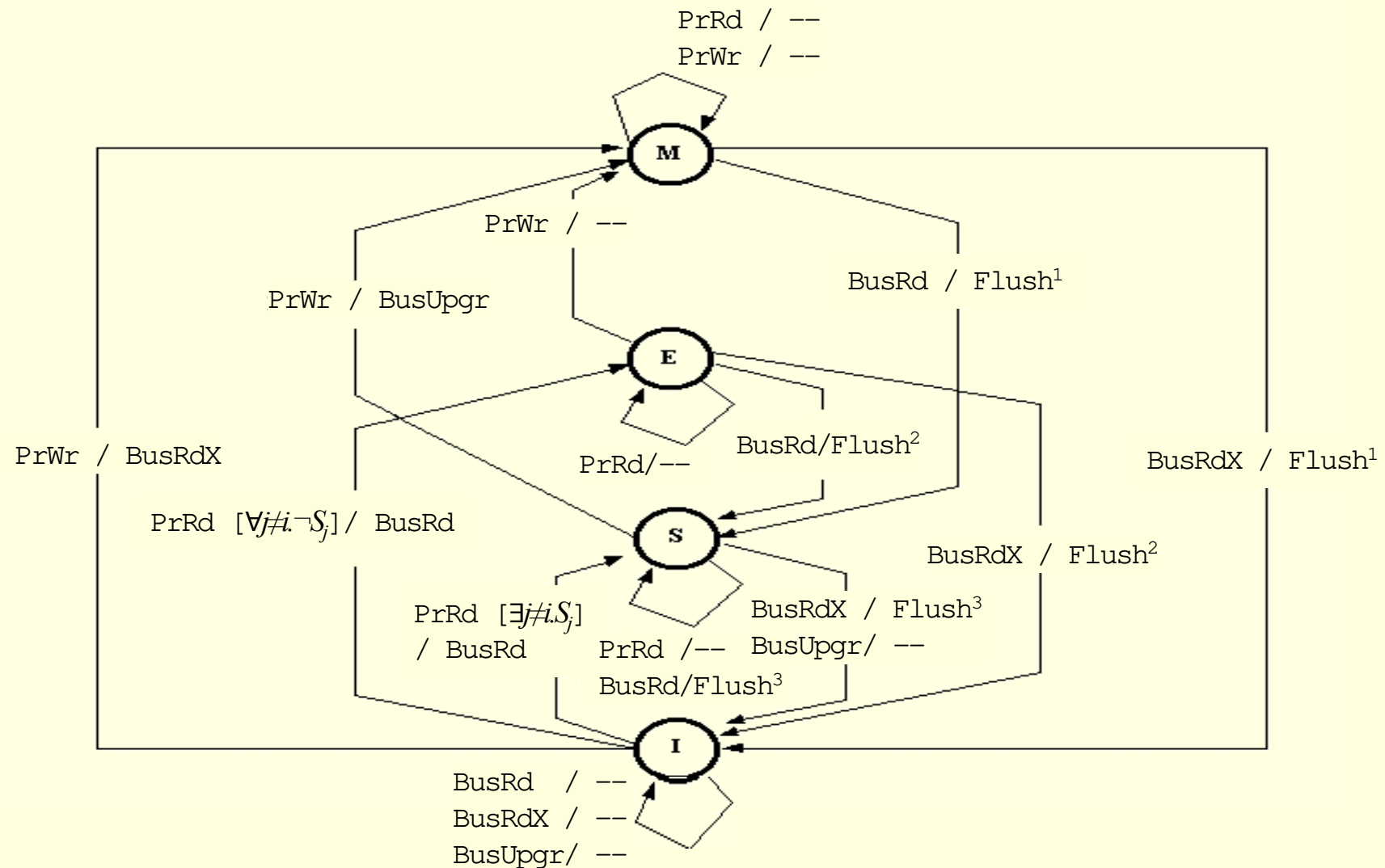


- Il protocollo MESI utilizza quattro stati:
 - **I** (invalid): come per MSI, ovvero il blocco di memoria non è valido.
 - **S** (shared o valid): due o più processori hanno questo blocco nella loro cache in uno stato non modificato. La memoria centrale è aggiornata.
 - **E** (exclusive): solo una cache ha una copia del blocco ed essa non è stata modificata. La memoria centrale è aggiornata; quindi se il controllore della cache riceve una richiesta di scrittura dal proprio processore (PrWr) non deve invalidare nessuno (non deve eseguire BusUpgr) **risparmiando tempo**.
 - **M** (modified o dirty): come per MSI, ovvero solo un processore ha una copia valida del blocco nella sua cache; la copia in memoria centrale è obsoleta e nessun'altra cache ha una copia valida.
- Eventi
 - PrRd, PrWr, BusRd, BusRdX, BusUpgr
- Guardie
 - $[\forall j \neq i. \neg S_j]$ – tutte le altre cache non hanno una copia aggiornata del blocco
 - $[\exists j \neq i. S_j]$ – esiste almeno un'altra cache con una copia aggiornata del blocco
- Azioni
 - BusRd, BusRdX, BusUpgr, Flush

Coerenza della cache

Protocollo MESI

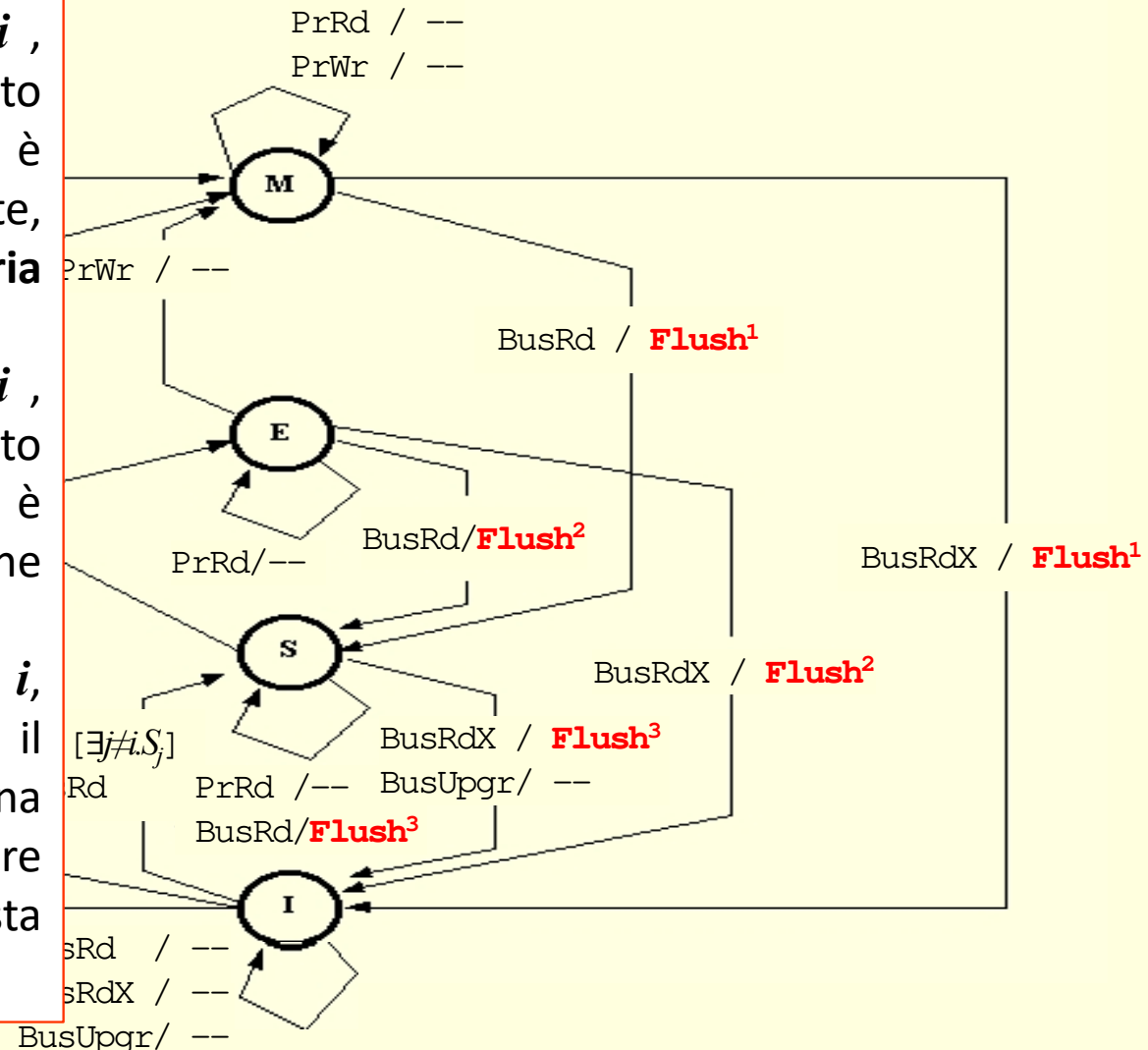
Cache i



Coerenza della cache

Protocollo MESI

- **Flush¹** - il controller della cache i , che è in stato M , mette il contenuto del blocco sul bus. Il blocco è ricevuto sia dalla cache richiedente, che è in stato I , sia dalla **memoria centrale**.
- **Flush²** - il controller della cache i , che è in stato E , mette il contenuto del blocco sul bus. Il blocco è ricevuto dalla cache richiedente, che è in stato I ,
- **Flush³** - il controller della cache i , che è in stato S , può mettere il contenuto del blocco sul bus (è una scelta di progettazione stabilire quale cache con stato S fa questa operazione).

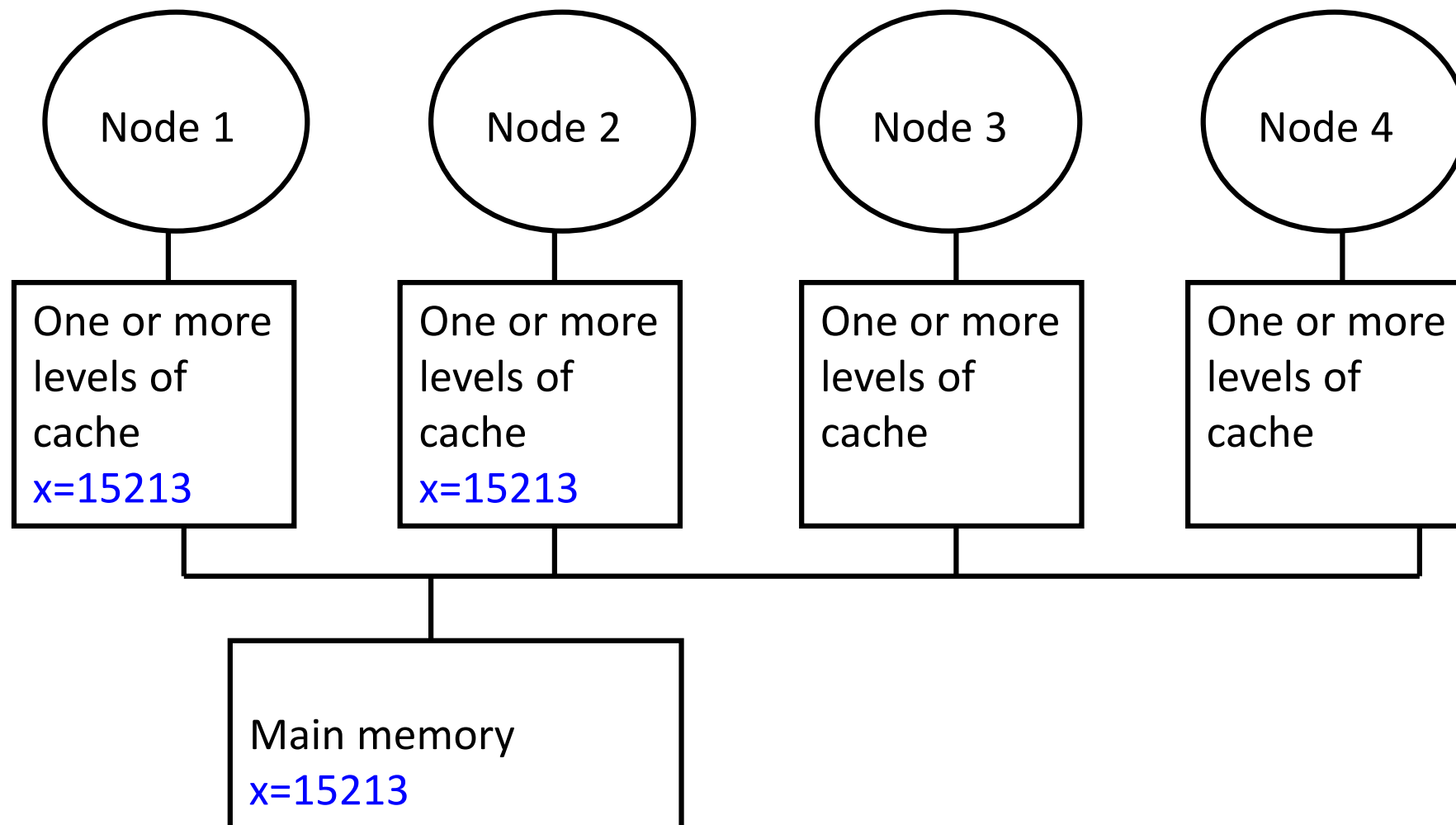


- Supponiamo di avere 3 nodi: n_1 , n_2 , n_3 le cui memorie di cache sono inizialmente vuote.
- Supponiamo di avere la seguente sequenza di richieste:
 1. n_1 – PrWr Blocco 0
 2. n_2 – PrRd Blocco 1
 3. n_3 – PrRd Blocco 0
 4. n_1 – PrWr Blocco 1

Coerenza della cache

Protocolli basati sull'Aggiornamento

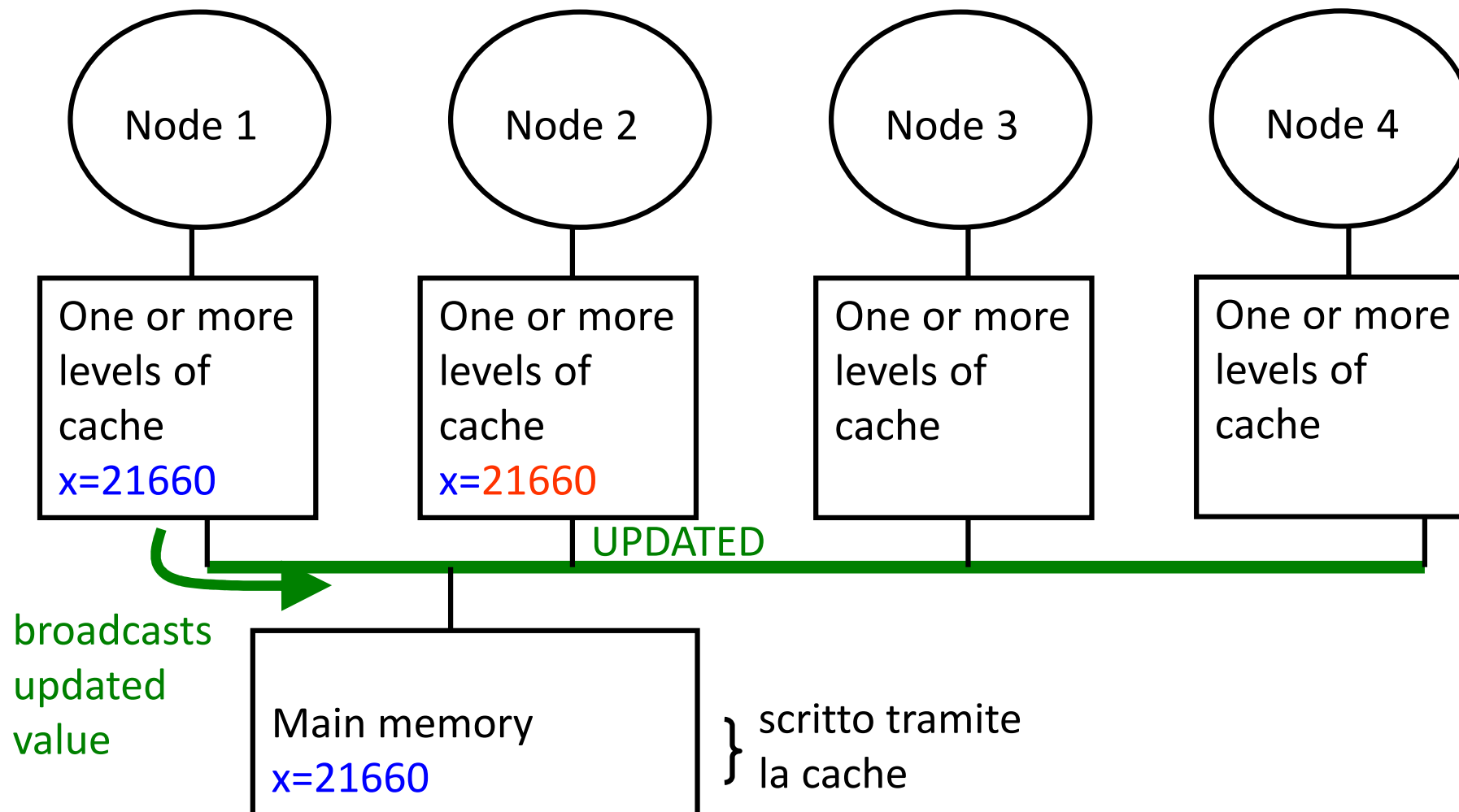
- Ripartiamo da quando sia Node 1 e Node 2 hanno letto x



Coerenza della cache

Protocolli basati sull'Aggiornamento

- Nodo 1 pone $x=21660$:



Coerenza della cache

Protocolli basati sull'Aggiornamento

- **Richieste del processore:**
 - PrRd: lettura; **PrRdMiss**: lettura di un nuovo blocco (non in cache)
 - PrWr: scrittura; **PrWrMiss**: scrittura di un nuovo blocco (non in cache)

- **Azioni sul bus:**
 - BusRd: il controllore della cache invia l'indirizzo al bus e chiede una copia del blocco che non intende modificare (in caso di ricerca fallita nella cache);
 - **BusUpd**: è una nuova transazione che prende una parola specifica scritta dal processore e la trasmette al bus così che tutte le altre cache si possano aggiornare. Ogni BusUpd è seguito da un'azione di **Update** eseguita dalle cache che devono aggiornare il blocco.
 - Flush: il controllore della cache può mettere il contenuto del blocco referenziato sul bus, piuttosto che permettere alla memoria centrale di fornire i dati.

Coerenza della cache

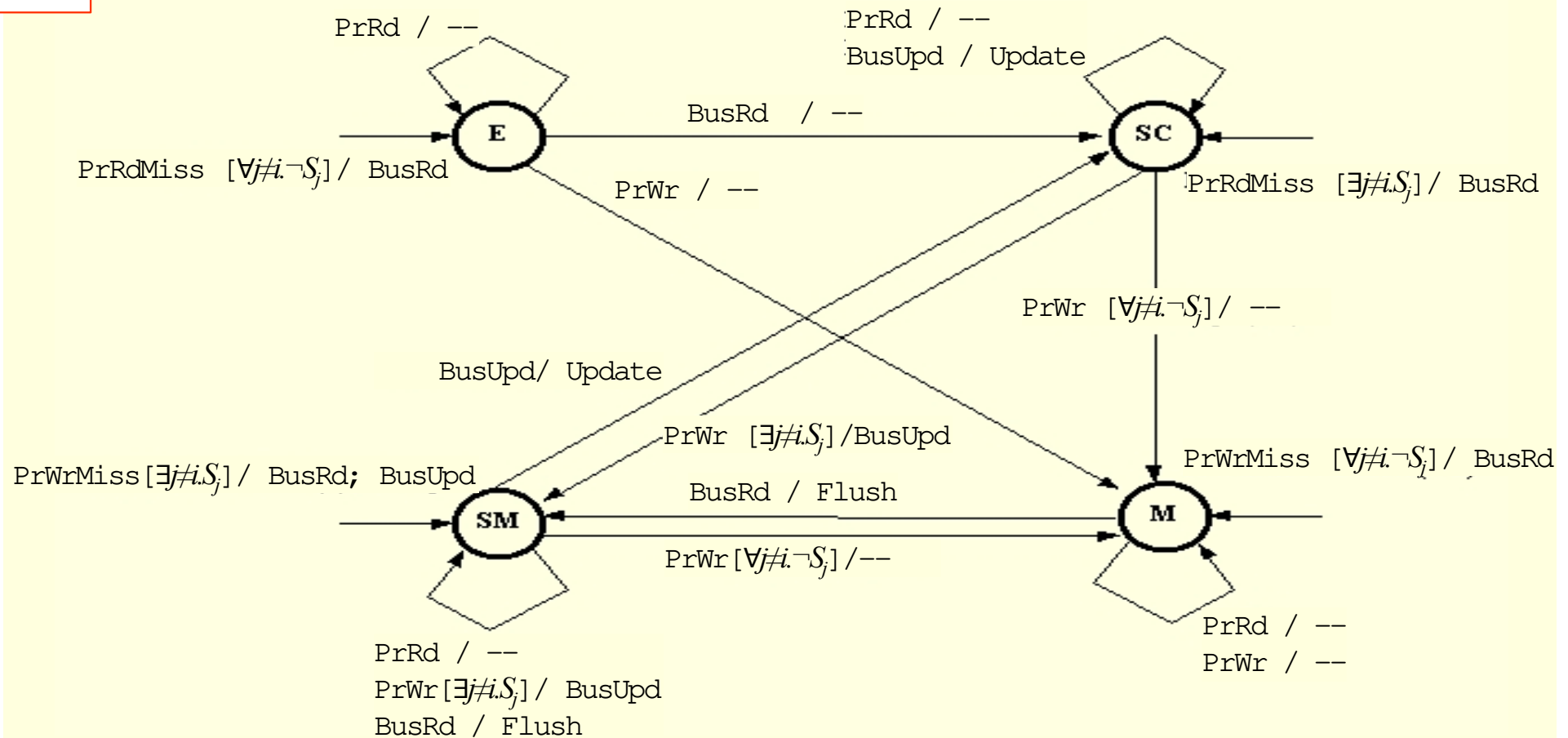
Protocollo Dragon

- Questo protocollo usa quattro stati:
 - **E** (exclusive): come per MESI;
 - **SC** (shared-clean): potenzialmente due o più processori hanno questo blocco nella cache. La memoria centrale può essere o non essere aggiornata.
 - **SM** (shared-modified): potenzialmente due o più processori hanno questo blocco nella loro cache. La memoria centrale non è aggiornata ed è responsabilità di tale processore ad aggiornare la memoria centrale quando il blocco è rimpiazzato.
 - **M** (modified): come per MESI.
- Eventi
 - PrRdMiss, PrWrMiss, PrRd, PrWr, BusRd, BusUpd
- Guardie
 - $[\forall j \neq i. \neg S_j]$ – tutte le altre cache non hanno una copia aggiornata del blocco
 - $[\exists j \neq i. S_j]$ – esiste almeno un'altra cache con una copia aggiornata del blocco
- Azioni
 - BusRd, BusUpd, Flush, Update

Coerenza della cache

Protocollo Dragon

Cache i

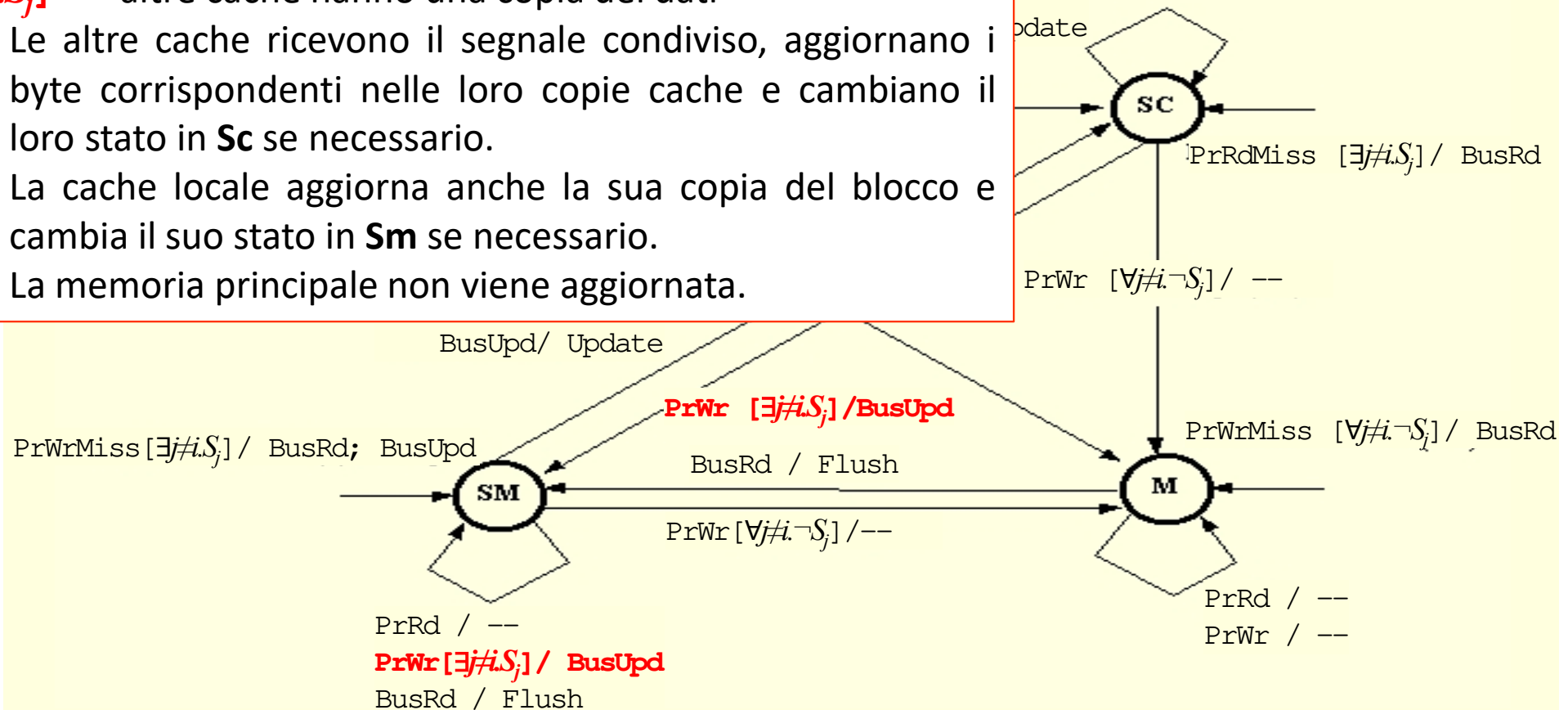


Coerenza della cache

Protocollo Dragon

PrWr - se il blocco è nello stato **Sc** o **Sm** → transazione **BusUpd**.

- **$[\exists j \neq i. S_j]$** – altre cache hanno una copia dei dati
 - Le altre cache ricevono il segnale condiviso, aggiornano i byte corrispondenti nelle loro copie cache e cambiano il loro stato in **Sc** se necessario.
 - La cache locale aggiorna anche la sua copia del blocco e cambia il suo stato in **Sm** se necessario.
 - La memoria principale non viene aggiornata.

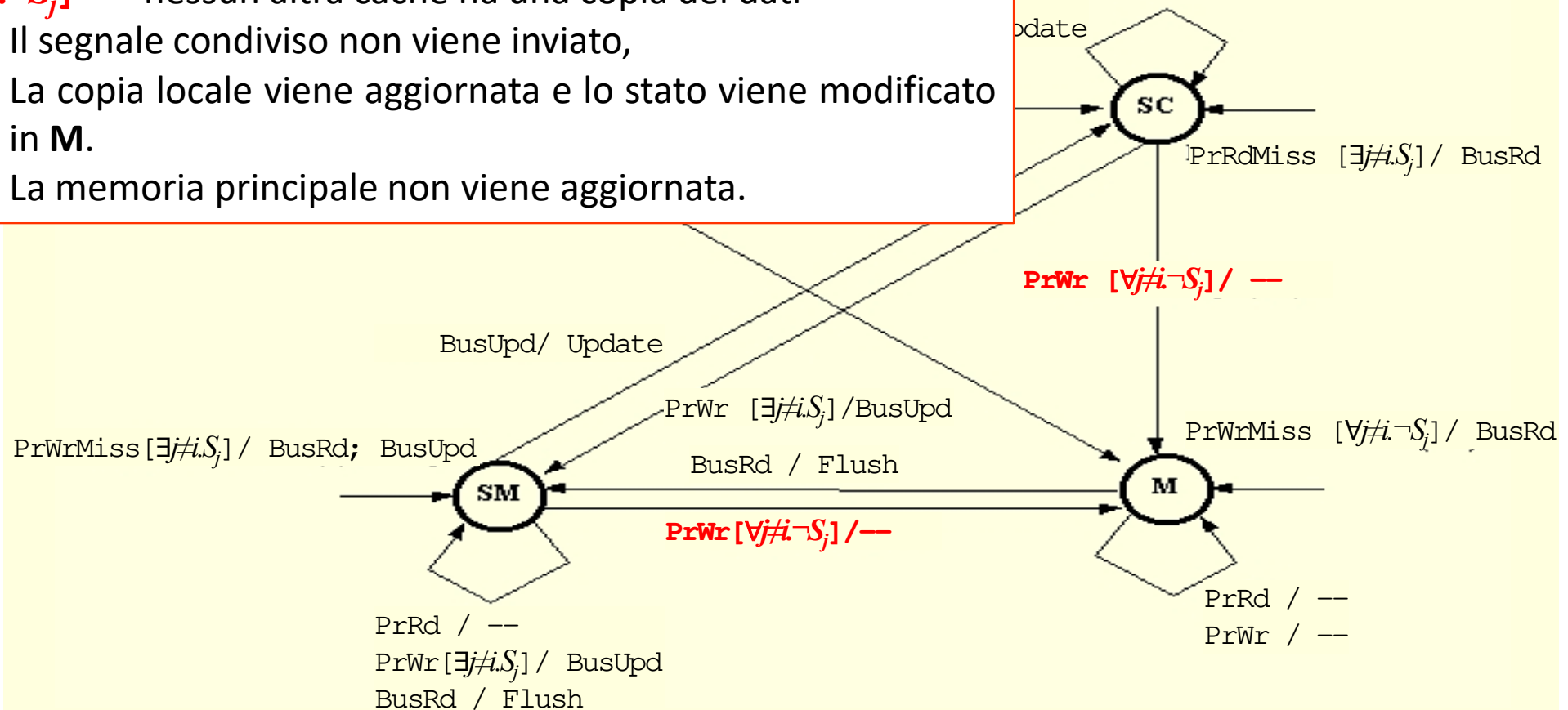


Coerenza della cache

Protocollo Dragon

PrWr - se il blocco è nello stato **Sc** o **Sm** → transazione **BusUpd**.

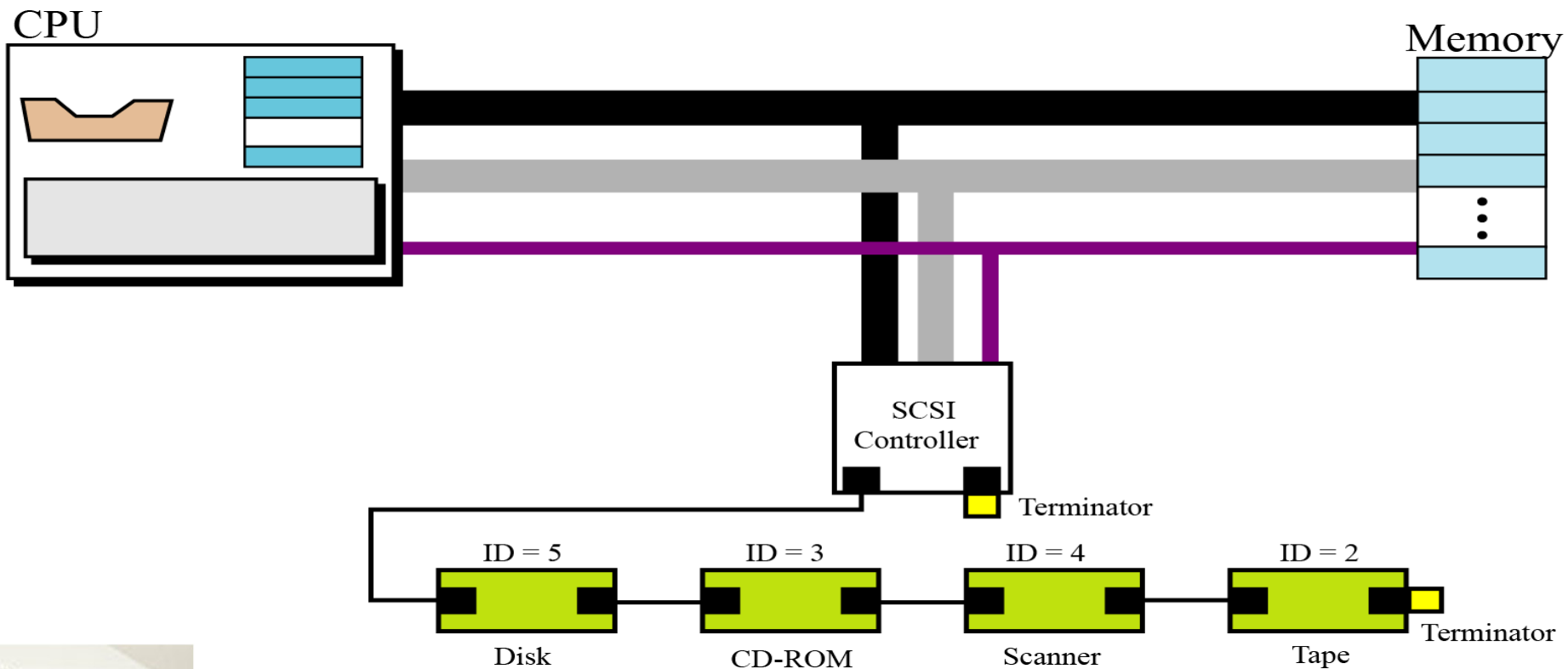
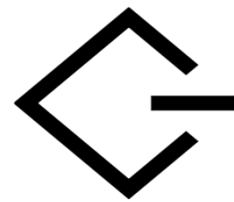
- $[\forall j \neq i. \neg S_j]$ – nessun'altra cache ha una copia dei dati
 - Il segnale condiviso non viene inviato,
 - La copia locale viene aggiornata e lo stato viene modificato in **M**.
 - La memoria principale non viene aggiornata.



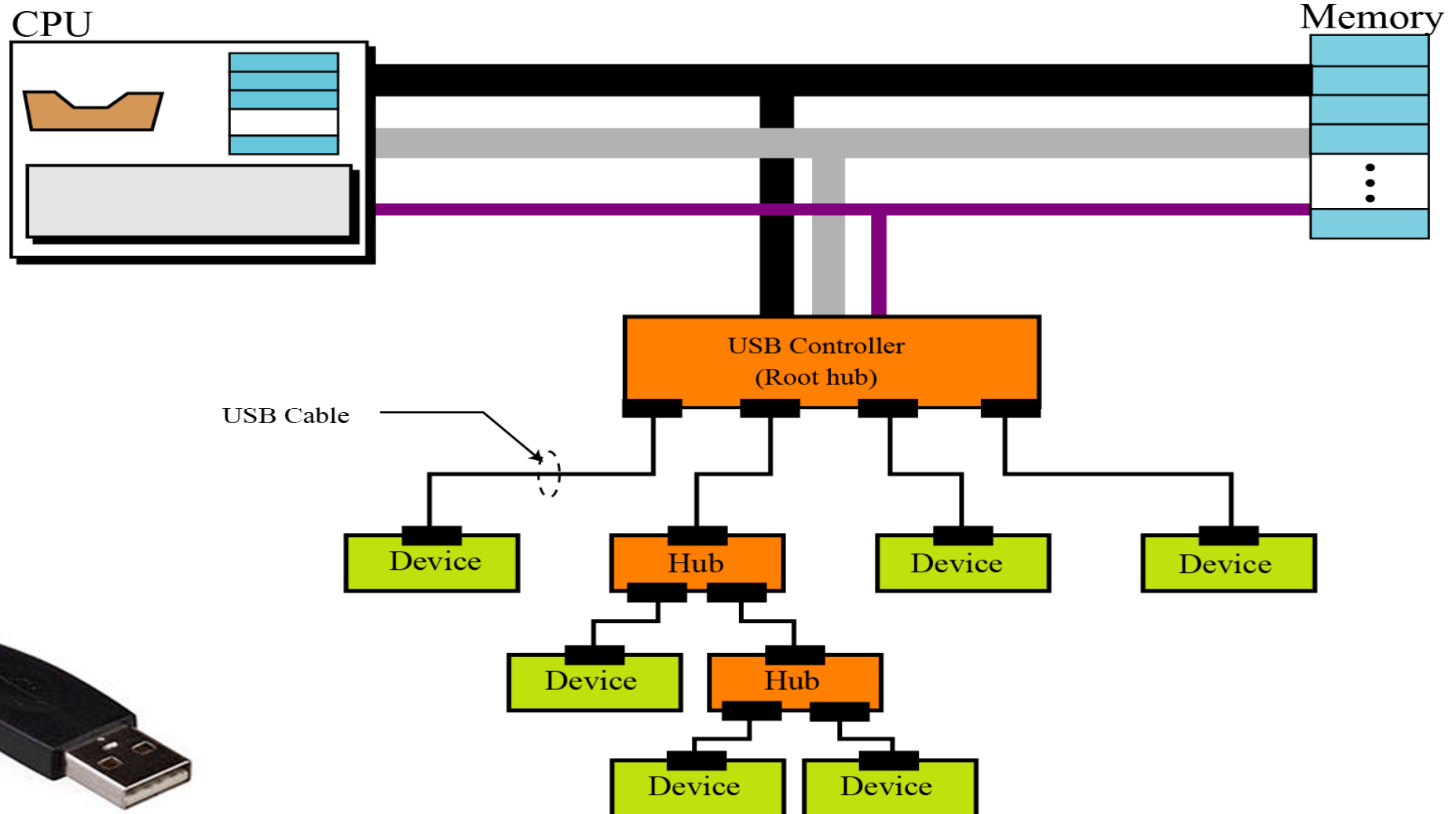


- ✓ CPU e ciclo Fetch-Decode-Execute
- ✓ Organizzazione della Memoria
- Organizzazione dei Dispositivi di I/O
- Protezione dell'Hardware

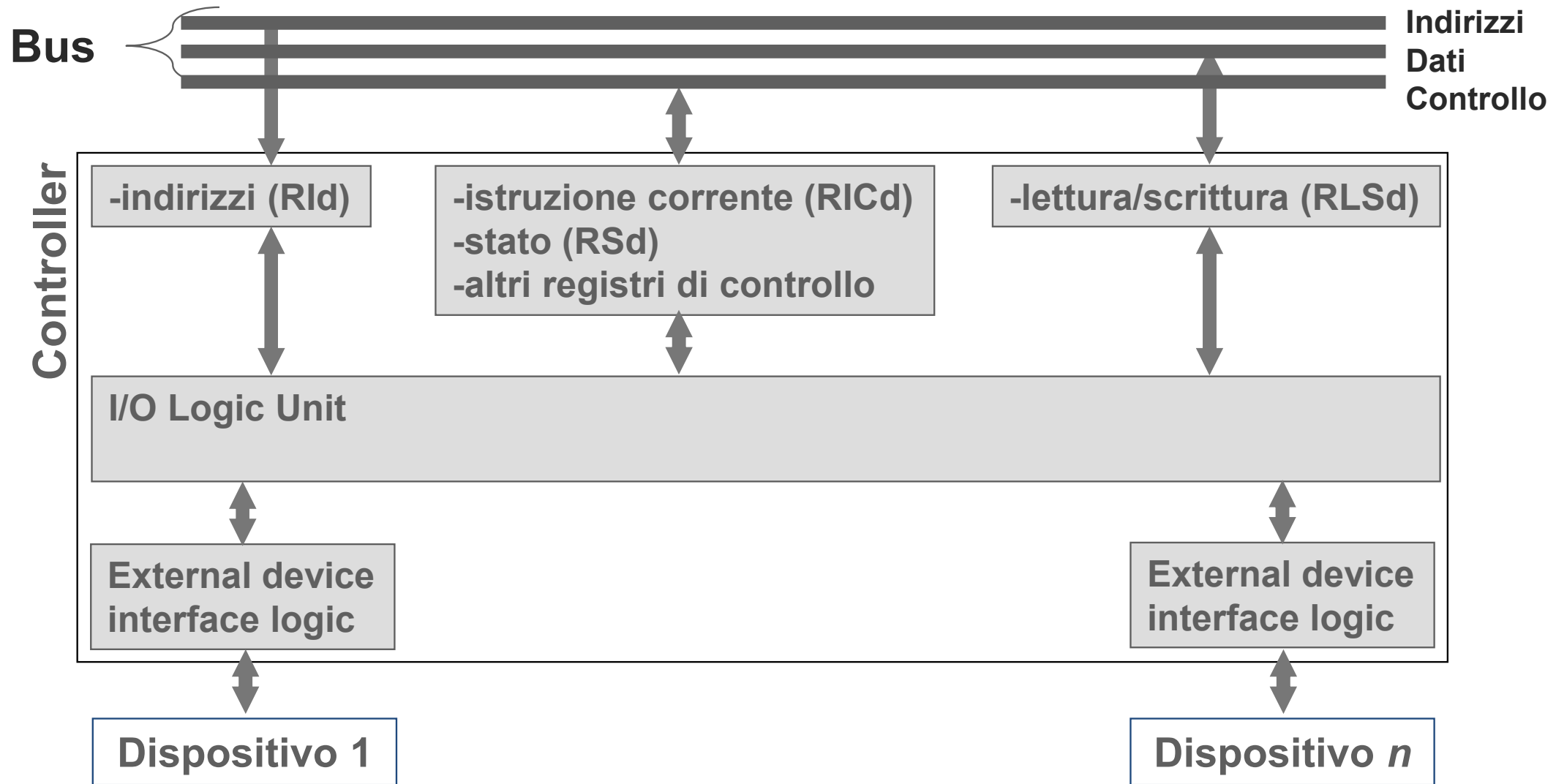
SCSI Controller



USB Controller



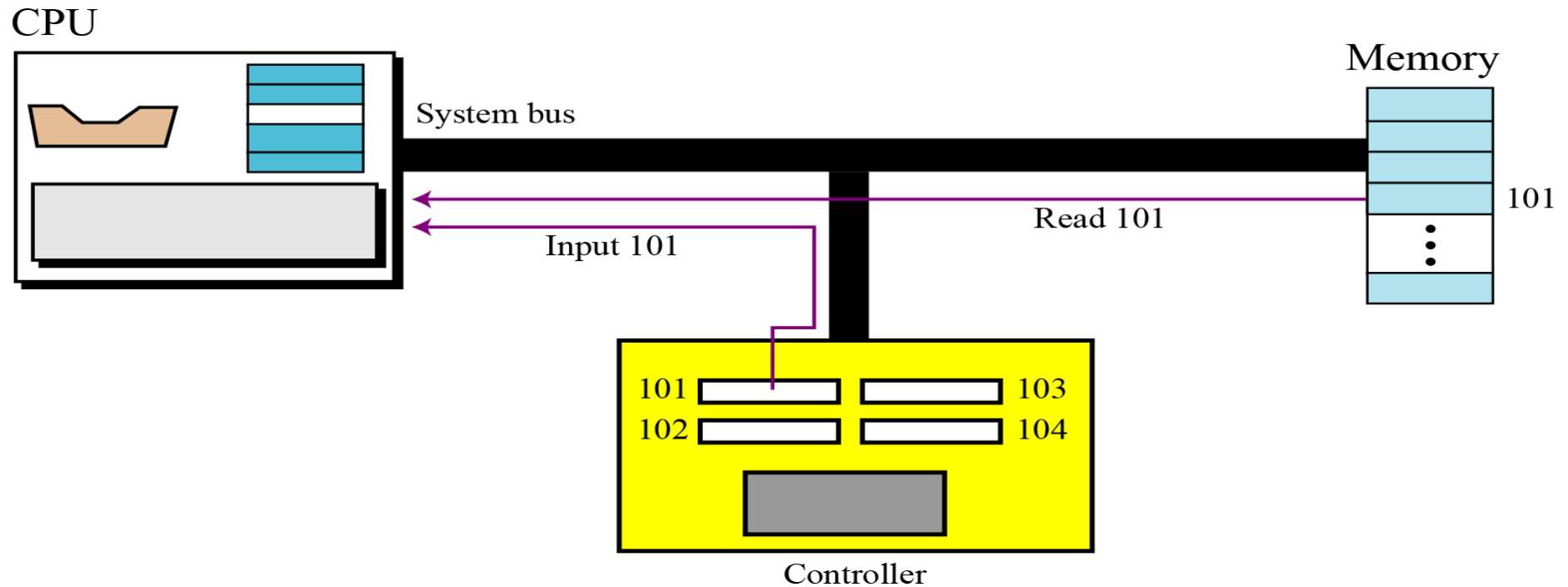
Architettura dei Dispositivi di I/O



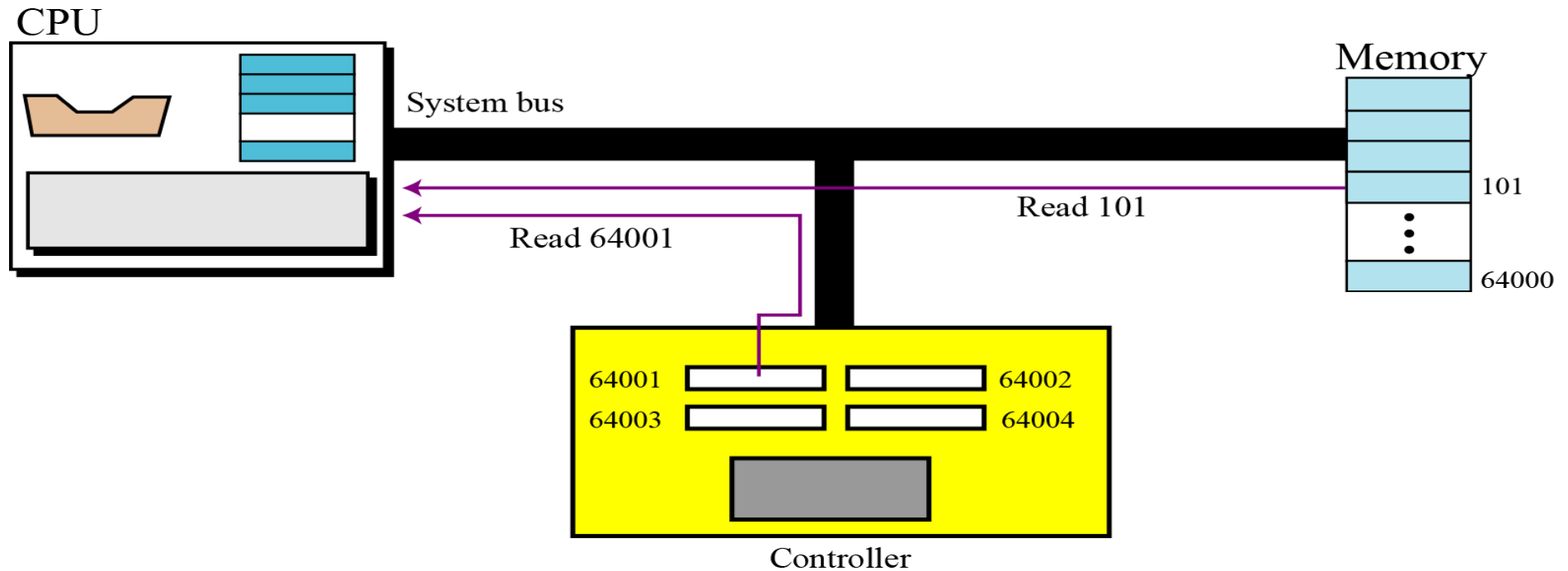
Architettura dei Dispositivi di I/O

- Ogni dispositivo è collegato al bus di sistema tramite un'interfaccia dove risiedono alcuni registri (di controllo, per permettere la sincronizzazione CPU-Dispositivo, buffer per contenere i dati in ingresso/uscita). Questa interfaccia è chiamata **controller**.
- La CPU colloquia con il controller scrivendo/leggendolo i suoi registri:
- utilizzando apposite istruzioni (**IN / OUT**) che avranno come parametro un "indirizzo" che identifica un particolare controller
- utilizzando normali istruzioni **LOAD/STORE** a indirizzi associati ai registri del controller: questa tecnica si chiama **Memory Mapped I/O**

Isolated I/O Addressing

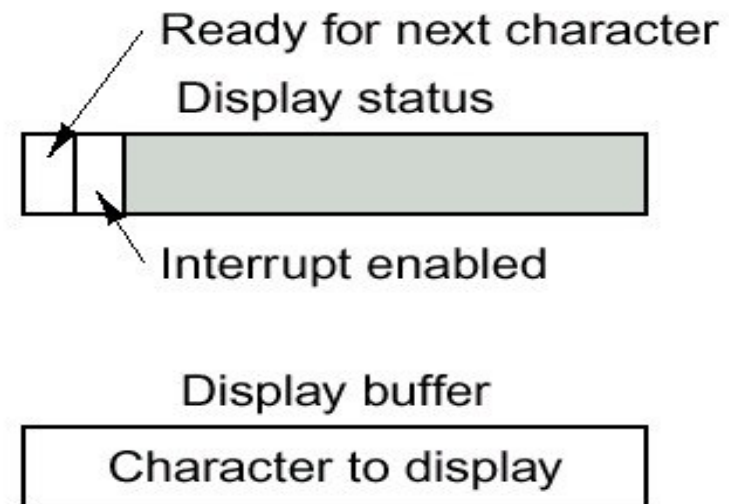
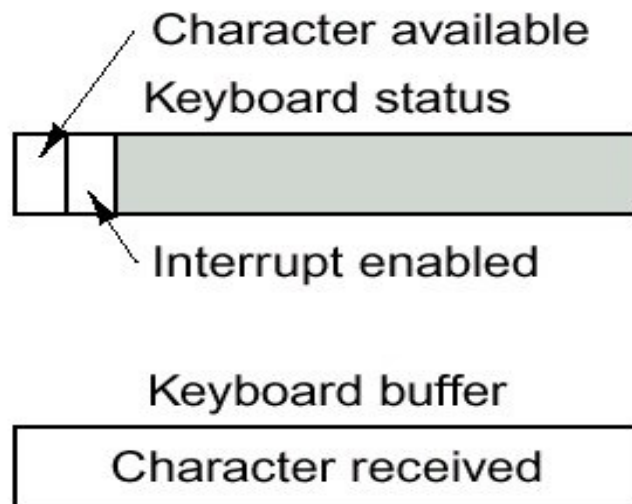


Memory Mapped I/O Addressing



- Nelle architetture più semplici si trovano normalmente tre tipi di schemi per eseguire le azioni di I/O
- I/O programmato con Busy-Waiting
- I/O guidato dalle interruzioni (Interrupt)
- I/O con accesso diretto alla memoria (DMA)

I/O Programmato (con busy waiting)



I/O Programmato (con busy waiting)

- Pseudocodice per il controllo dell'output di una stringa di caratteri sul display

```
public static void output_buffer(int buf[ ], int count) {  
    // Output a block of data to the device  
    int status, i, ready;  
    for (i = 0; i < count; i++) {  
        do {  
            status = in(display_status_reg); // get status  
            ready = (status << 7) & 0x01; // isolate ready bit  
        } while (ready == 1);  
        out(display_buffer_reg, buf[i]);  
    }  
}
```

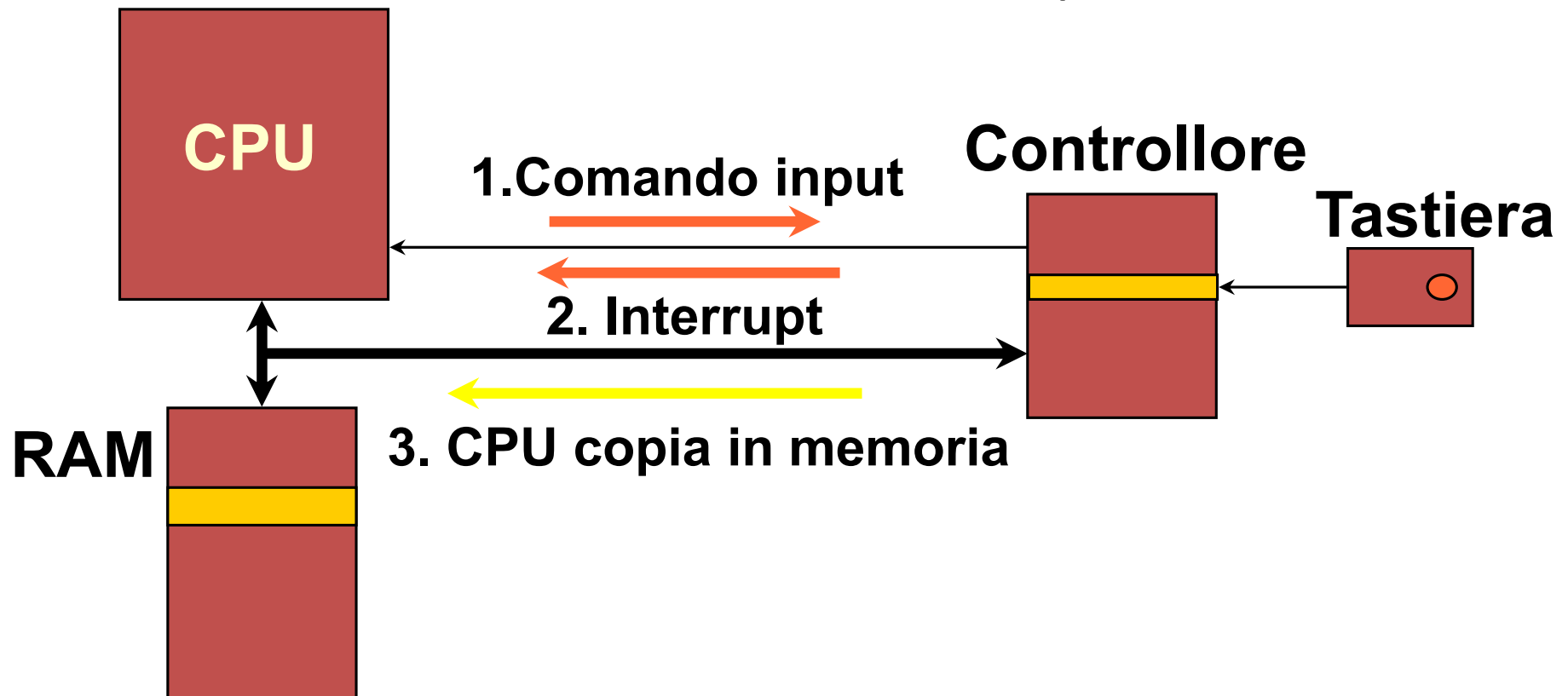
Busy Waiting

I/O Guidato da Interrupt

- Dato che CPU e dispositivi di I/O hanno velocità molto diverse, è opportuno non tenere la CPU bloccata in un ciclo di controllo dei bit di stato in attesa che termini l'operazione richiesta. Se si potesse evitare il busy waiting, mentre i dispositivi di I/O lavorano, la CPU potrebbe essere impiegata in compiti più utili (per esempio eseguire un altro programma).

I/O Guidato da Interrupt

- Interazione CPU/tastiera basata su interrupt



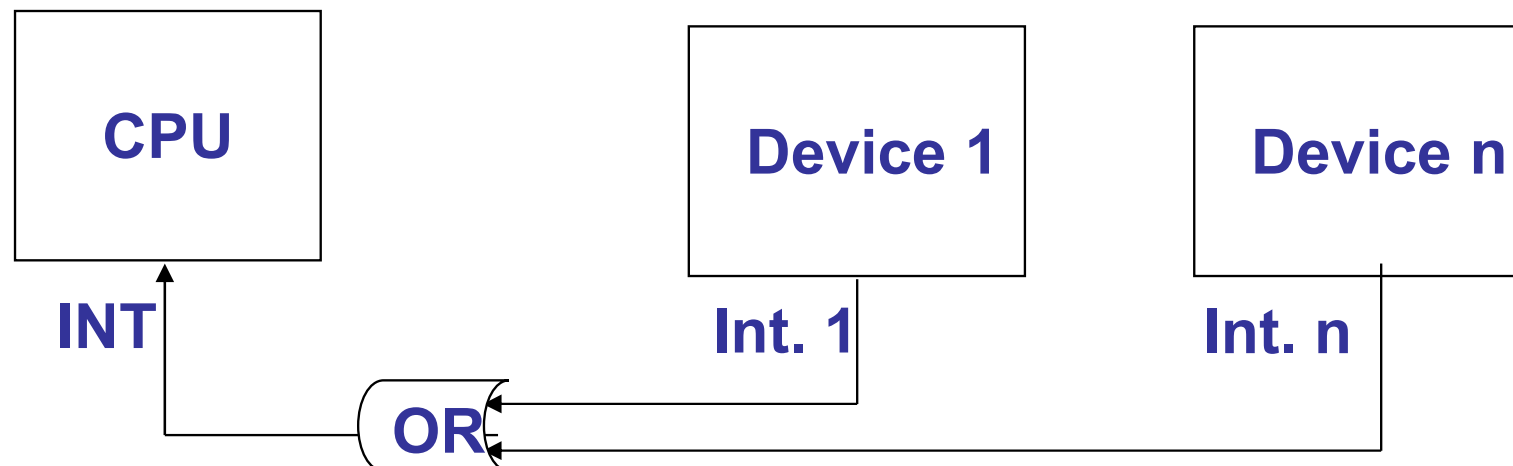
- La CPU viene interrotta prima del trasferimento in memoria (che viene effettuato dalla CPU) del contenuto del buffer.

I/O Guidato da Interrupt

Problemi:

Come funziona nel caso ci siano più dispositivi che possono inviare segnali di interrupt ?

1. i segnali provenienti dai diversi dispositivi vengono messi in OR
2. il dispositivo che ha causato interrupt viene individuato tramite Polling oppure utilizzando la tecnica dell'Interrupt Vettorizzato



I/O Guidato da Interrupt

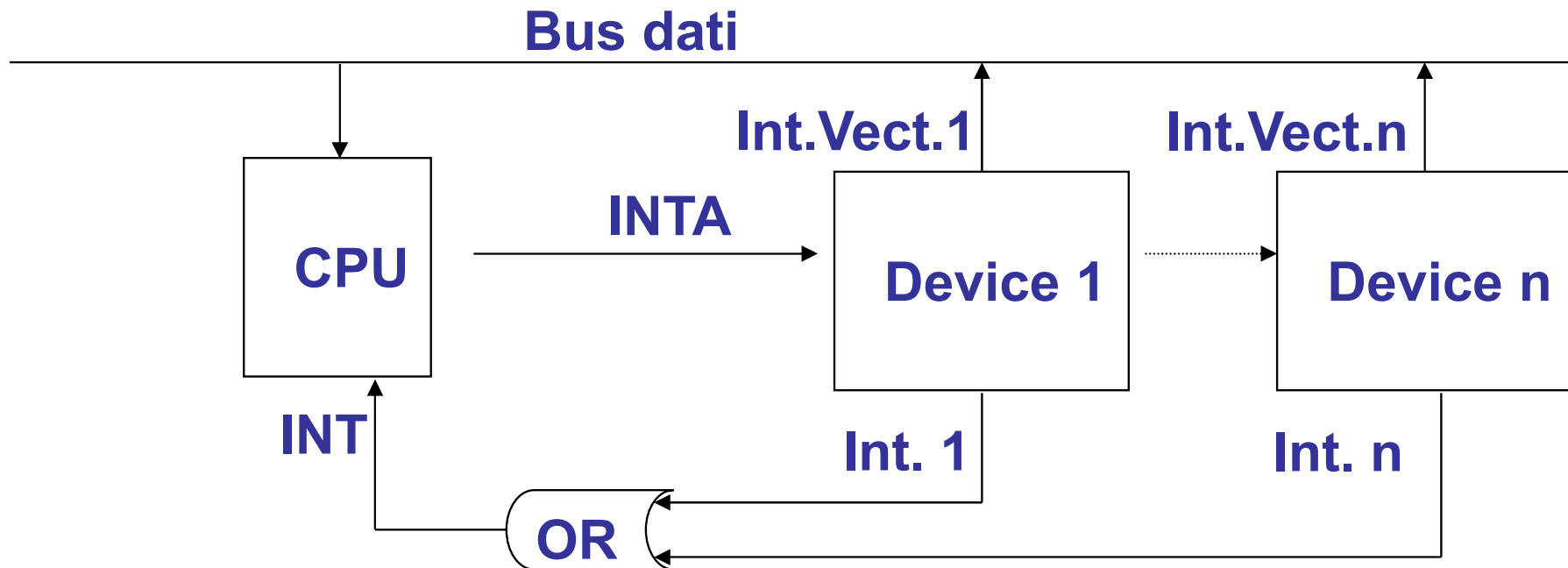
Problemi:

Cosa accade se arrivano piu' interrupt contemporaneamente?

la CPU avverte un solo segnale (OR di tutti i segnali di int.)

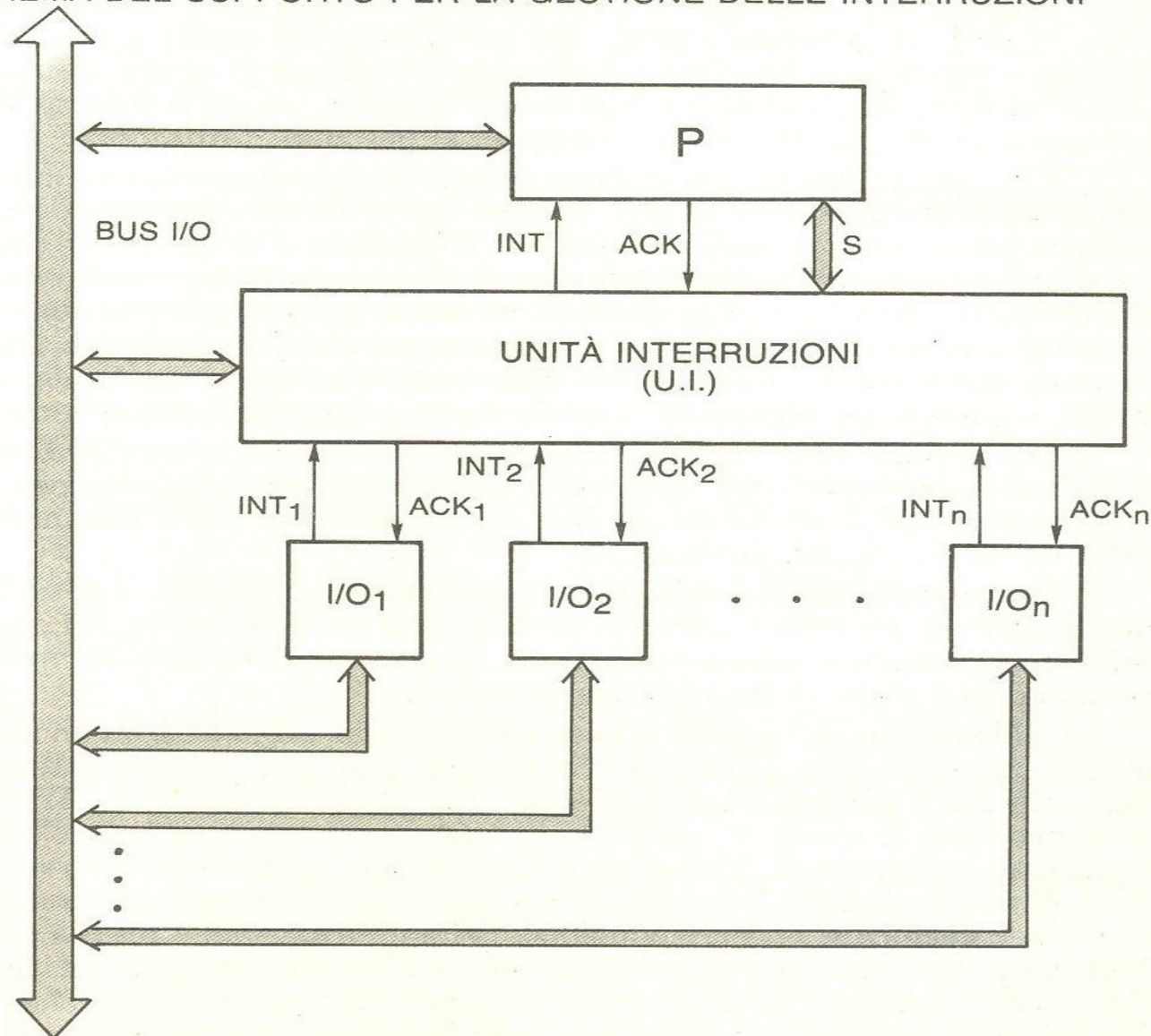
- 1) se si usa il Polling, l'ordine di scansione determina un ordinamento implicito tra i dispositivi
- 2) se si usa l'Interrupt vettorizzato, occorre impostare un protocollo di comunicazione tra CPU e dispositivi: quando la CPU ha ricevuto l'interrupt ed e' pronta a gestirlo, invia un segnale di Acknowledge (INTA). Quando il controller riceve il segnale invia l'identificatore del dispositivo (vettore di interrupt) sul bus dati. Se il segnale di INTA viene propagato sequenzialmente dai controllori dei dispositivi (collegamento daisy chain) e assorbito dal primo controller che aveva un interrupt pendente, si ha un ordinamento implicito tra i dispositivi determinato dal collegamento in daisy chain degli stessi

I/O Guidato da Interrupt



I/O guidato da Interrupt

SCHEMA DEL SUPPORTO PER LA GESTIONE DELLE INTERRUZIONI



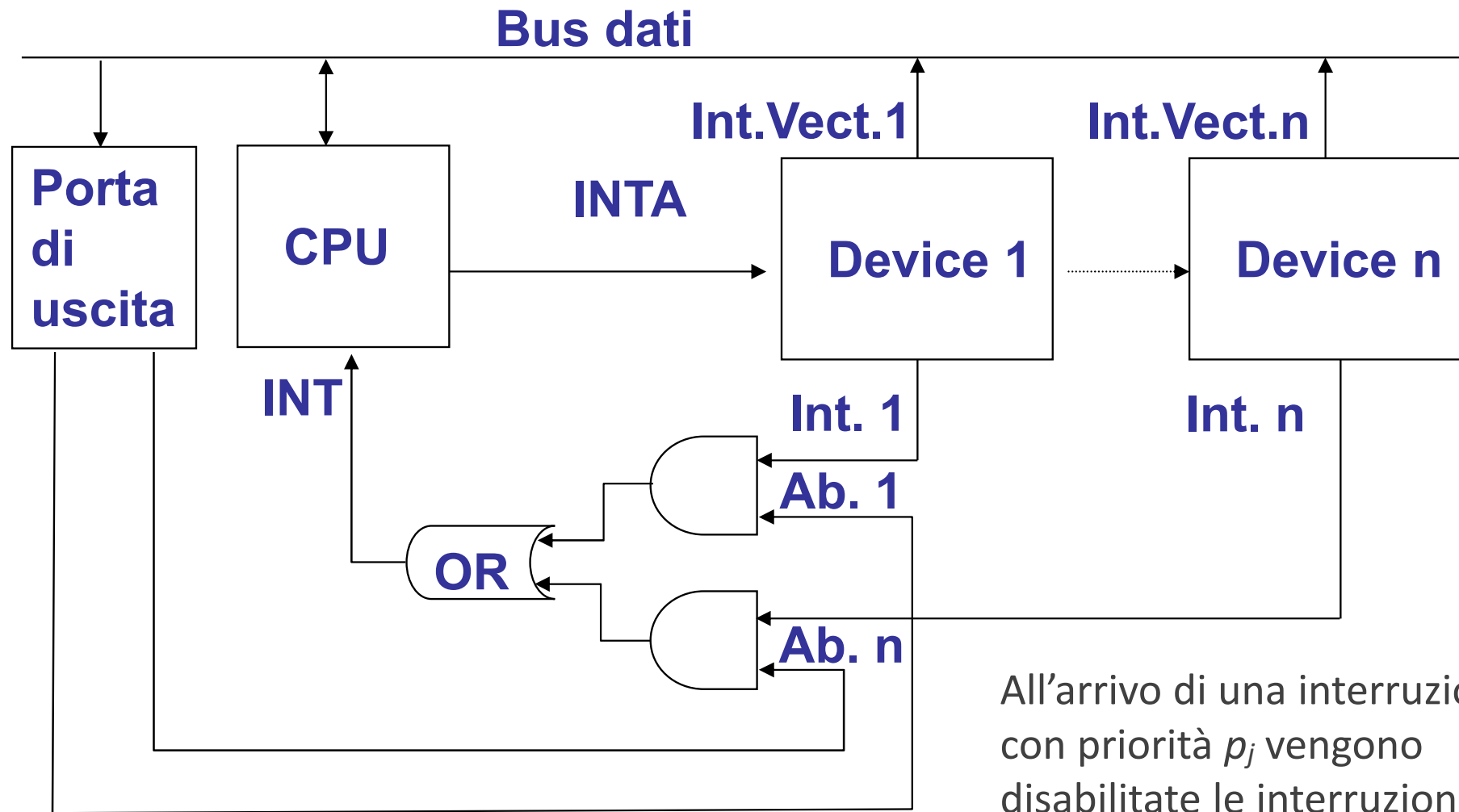
I/O Guidato da Interrupt

Problemi:

Cosa accade se arriva un secondo interrupt mentre è in esecuzione la procedura di trattamento di un (primo) interrupt?

- 1) Deve essere possibile **disabilitare** gli interrupt per prevenire la perdita di interrupt e più in generale quando deve essere eseguita una porzione di codice senza interruzioni. In questo caso gli interrupt pendenti vengono ignorati fino alla riabilitazione degli interrupt. Non si possono avere interrupt annidati.
- 2) Se vi sono segnali di interrupt con vincoli stringenti sui tempi di risposta, è opportuno poter definire diversi **livelli di priorità** e permettere ad un interrupt ad alta priorità di essere gestito anche se è in esecuzione una procedura di gestione di un altro interrupt (a più bassa priorità) – si possono quindi avere **interrupt annidati**

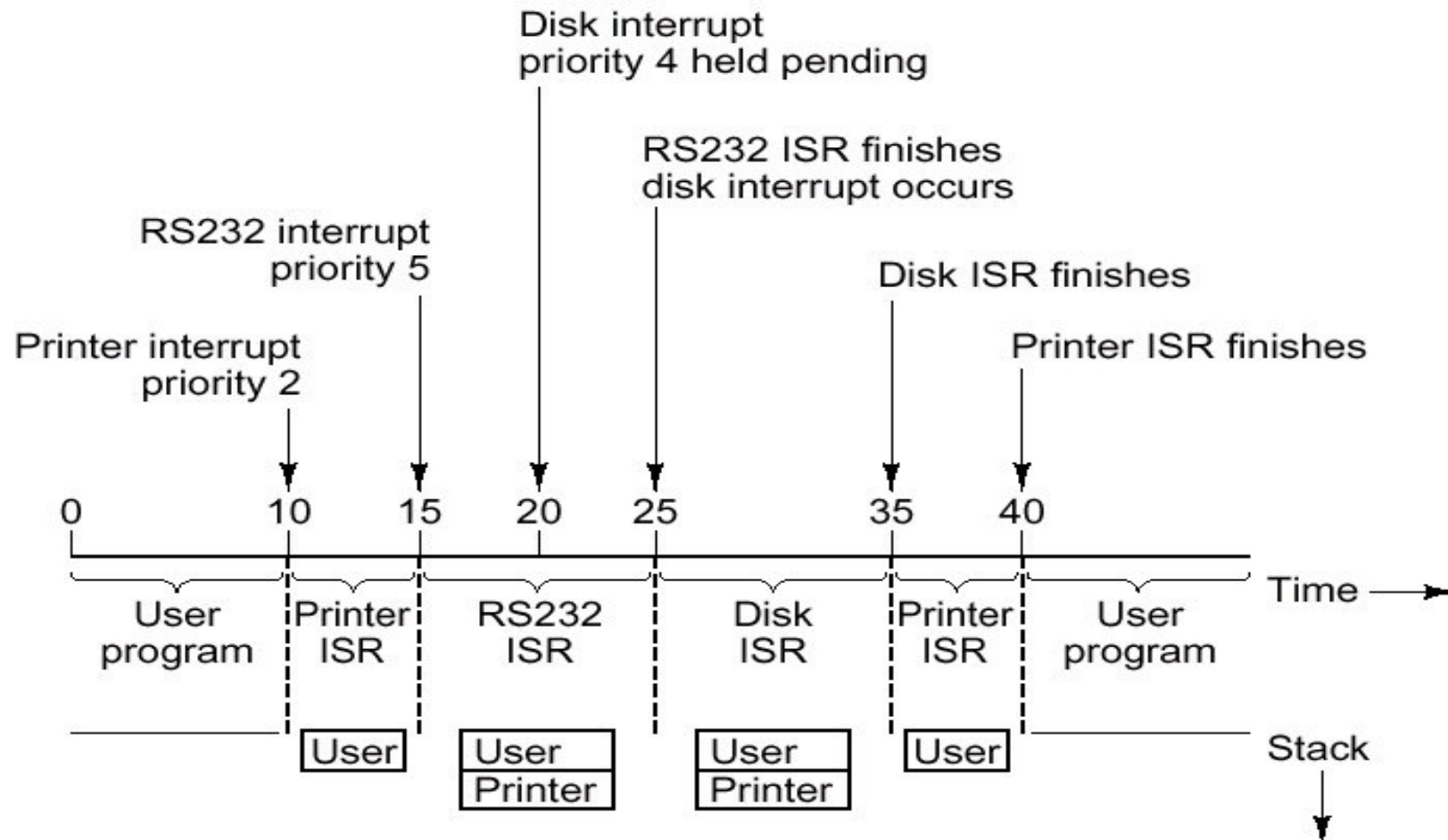
I/O Guidato da Interrupt



All'arrivo di una interruzione con priorità p_j vengono disabilitate le interruzioni da dispositivi con priorità inferiore a p_j

I/O Guidato da Interrupt

Esempio di utilizzo di diversi *livelli di priorit * per gli interrupt



Stampante: Livello 2, RS232: Livello 5, Disco: livello 4

I/O Guidato da Interrupt

Cosa succede quando la CPU riceve un interrupt:

1. L'istruzione macchina in corso viene completata.
2. Si salva in memoria lo stato della computazione interrotta (registri e program counter) nel descrittore di processo.
3. Si determina il tipo di interruzione che è avvenuta (tramite polling o tecnica dell'interrupt vettorizzato).
4. Si salta ad una posizione in memoria dove è contenuto l'interrupt handler ovvero il gestore di quel tipo di interruzioni (es. driver del disco). Infatti segmenti di codice separati determinano quale azione deve essere eseguita per ogni tipo di interrupt.
5. Dopo aver gestito l'interrupt, viene ripristinato lo stato del programma interrotto e la sua esecuzione

**Tutto ciò deve essere TRASPARENTE rispetto
a chi viene interrotto**

I/O Guidato da Interrupt

Alcuni esempi di interrupt handler

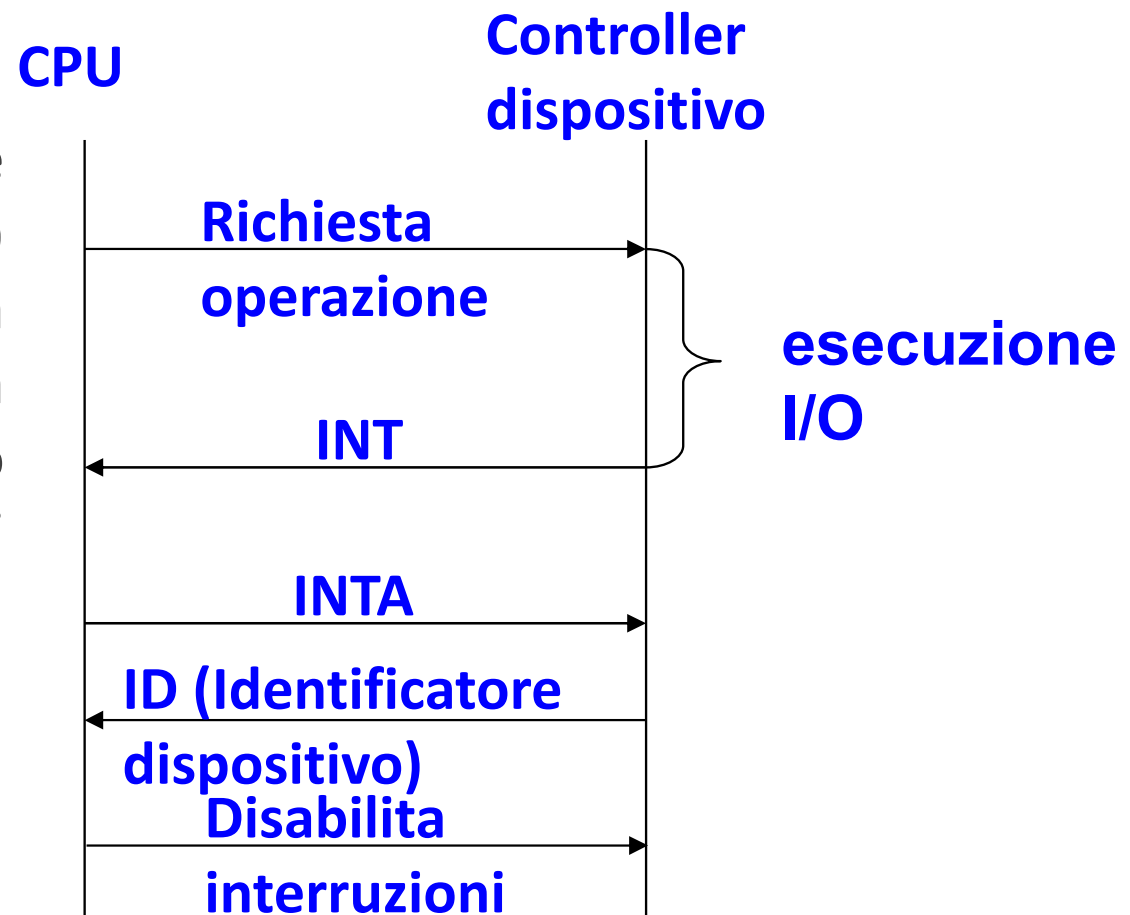
- L'interrupt handler per un **interrupt da disco** trasferisce dati dal buffer del controller a memoria o viceversa, e se esistono richieste di I/O in attesa del disco una di queste viene inoltrata al controller. Inoltre il programma che era in attesa dell'I/O appena terminato, viene riportato nell'insieme dei programmi a cui deve essere assegnata la CPU.
- L'interrupt handler per un **timer interrupt** si occupa dell'assegnazione della CPU ad un altro programma, fra quelli pronti per l'esecuzione.
- L'interrupt handler per un **interrupt da tastiera** trasferisce il carattere appena ricevuto dalla tastiera in memoria e riporta il programma che era in attesa dell'I/O appena terminato nell'insieme dei programmi a cui deve essere assegnata la CPU.

I/O Guidato da Interrupt

Gestione dell'Interrupt

La CPU deve poter sapere quale dispositivo ha inviato l'interrupt:

Salta alla procedura di gestione dell'interrupt (*interrupt handler*) il cui indirizzo di partenza si trova nell'*interrupt vector*, una tabella del sistema operativo indicizzato in base al (tipo di) dispositivo, qui indicato da ID.



***Esempio di
microinterprete senza
gestione degli interrupt:***

```
while (not halt ) do
{ fetch istruzione;
  decodifica istruzione;
  esegui istruzione;
}
```

***Esempio di microinterprete con
gestione degli interrupt:***

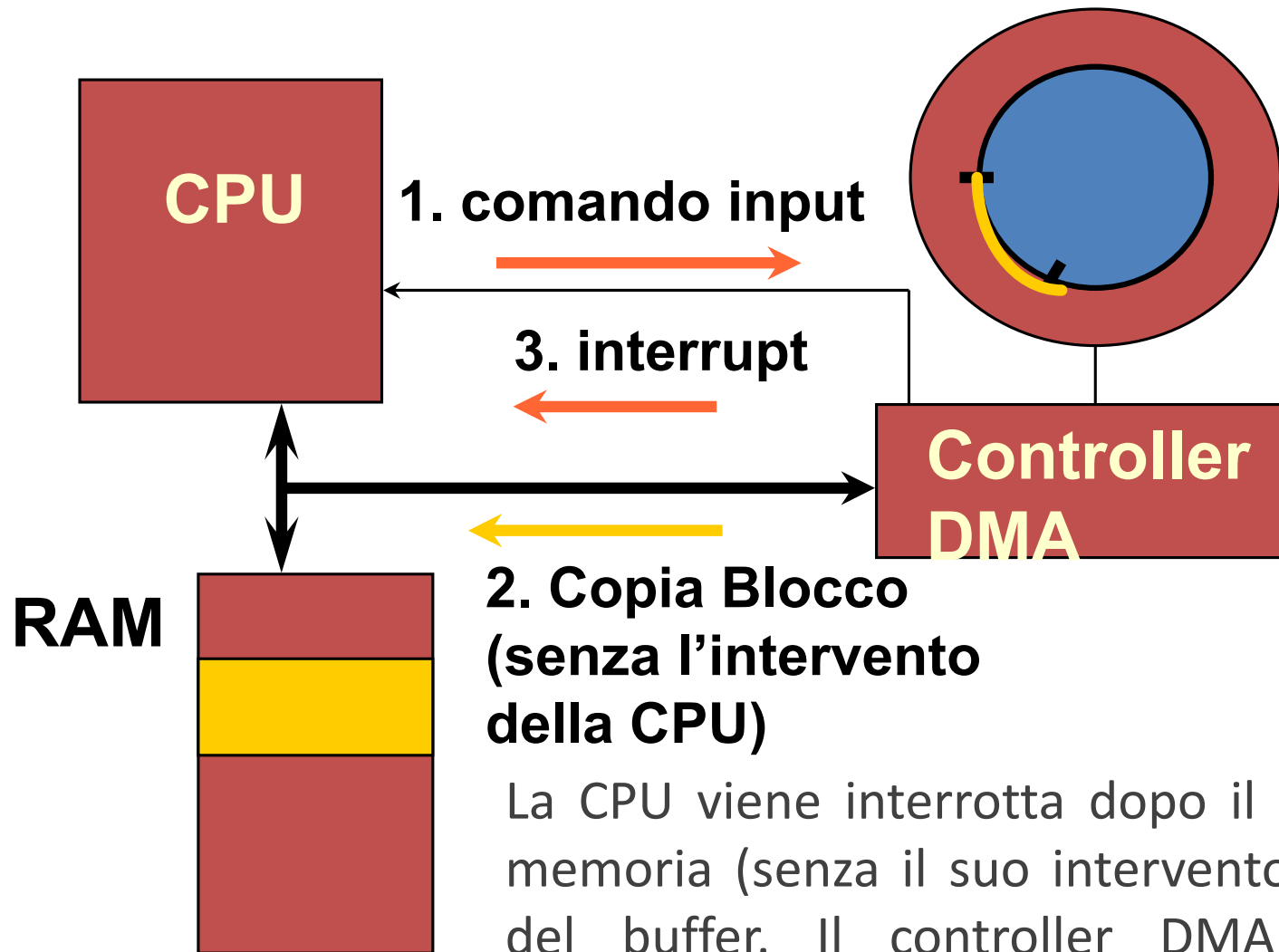
```
while (not halt ) do
{ if (interrupt & not int_disable)
  {salva stato;
   invia INTA;
   leggi device_id;
   int_disable = true;
   PC = int_vector[device_id];}
  fetch istruzione;
  decodifica istruzione;
  esegui istruzione; }
```

Direct Memory Access Structure

- Utilizzato per dispositivi di I/O ad alta velocità in grado di trasferire dati ad una velocità vicina a quella della memoria.
- Il controller del dispositivo trasferisce blocchi di dati dal buffer direttamente alla memoria centrale senza l'intervento della CPU.
- Viene generato un solo interrupt per blocco piuttosto che un interrupt per byte.

I/O con Accesso Diretto alla Memoria

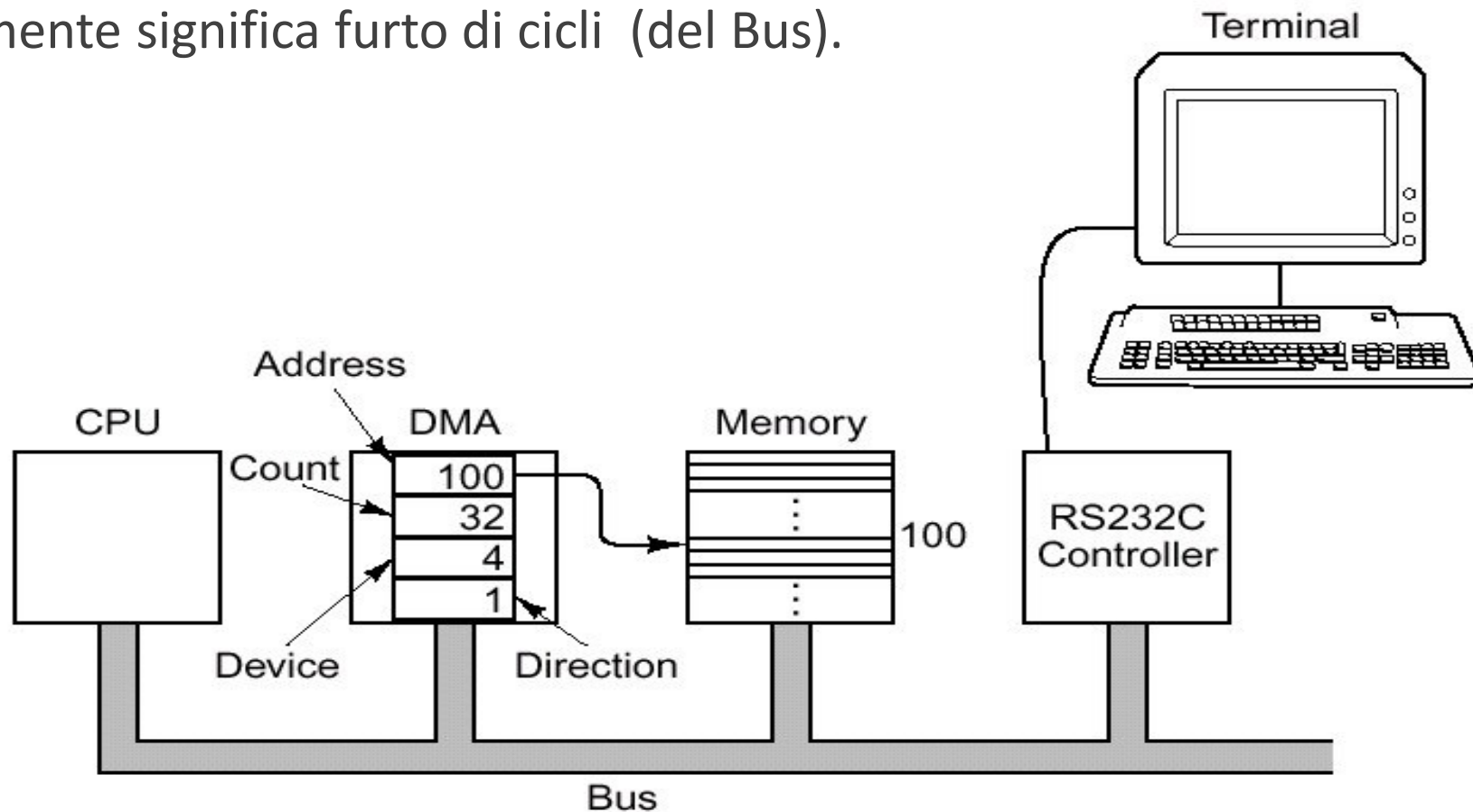
Interazione CPU/controller DMA



La CPU viene interrotta dopo il trasferimento in memoria (senza il suo intervento) del contenuto del buffer. Il controller DMA e la CPU si contendono il bus.

I/O con Accesso Diretto alla Memoria

Poiche' il Bus di collegamento alla memoria e' condiviso occorre un arbitro che gestisca i conflitti. Fenomeno del *Cycle Stealing* che letteralmente significa furto di cicli (del Bus).

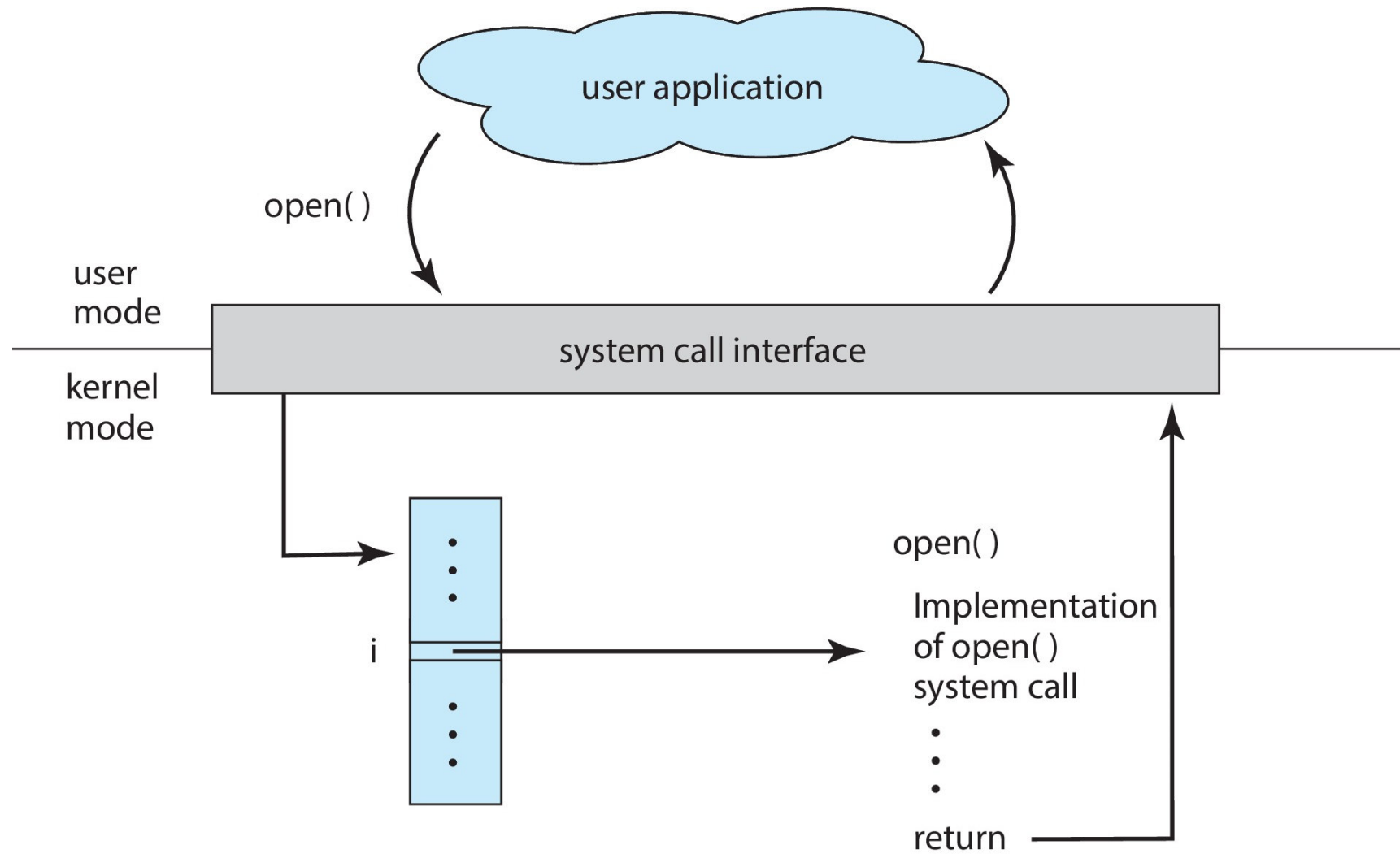


Interruzioni Software (Trap)

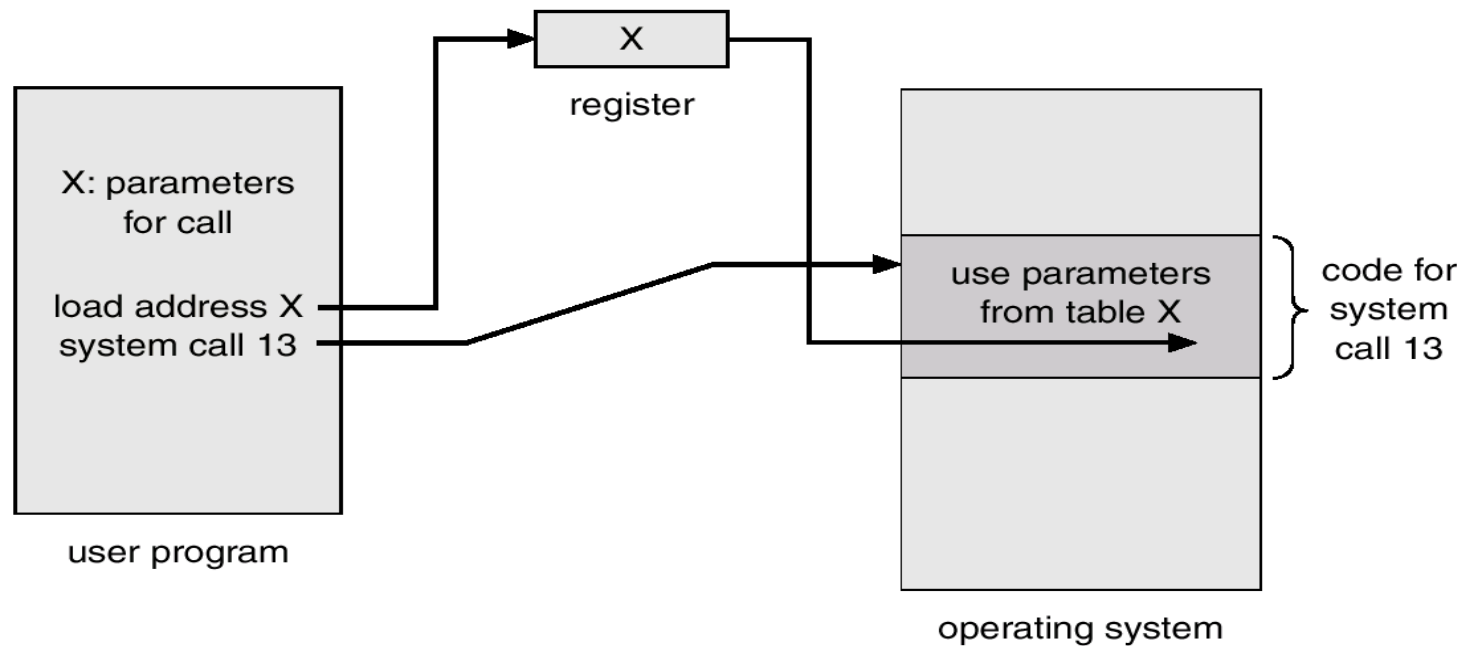
- Un altro tipo di interrupt, provocato dall'esecuzione di una determinata istruzione del programma in esecuzione:
 - istruzioni con errori (divisione per zero, accesso in memoria non valido, ...)
 - istruzioni esplicite di “trap” al SO (un programma utente richiede un servizio del SO – system call)
- Le trap vengono trattate in modo del tutto analogo agli interrupt: ciascun tipo di trap ha un identificatore che viene usato per cercare nell'interrupt vector l'indirizzo della prima istruzione del corrispondente interrupt handler.
- Gli **Interrupt** sono **ASINCRONI** rispetto al programma in esecuzione
- mentre le **Trap** sono **SINCRONE**: causate dal programma in esecuzione.

- Le **system call** passano attraverso l'interfaccia tra un programma in esecuzione ed i servizi di un SO (ovvero l'Application Binary Interface – ABI).
- Le system call sono realizzate tramite **trap**.
- Esistono tre metodi per il passaggio dei parametri:
 - I valori dei parametri vengono memorizzati nei **registri**.
 - I valori dei parametri vengono memorizzati in una **tabella** in memoria e l'indirizzo della tabella viene memorizzato in un registro.
 - I valori dei parametri vengono memorizzati dal programma con una operazione di *push* nello **stack** e letti dal SO con una operazione di *pop*.

System Call



Passaggio parametri attraverso Tabella





Alcune System Call di Windows e Unix

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

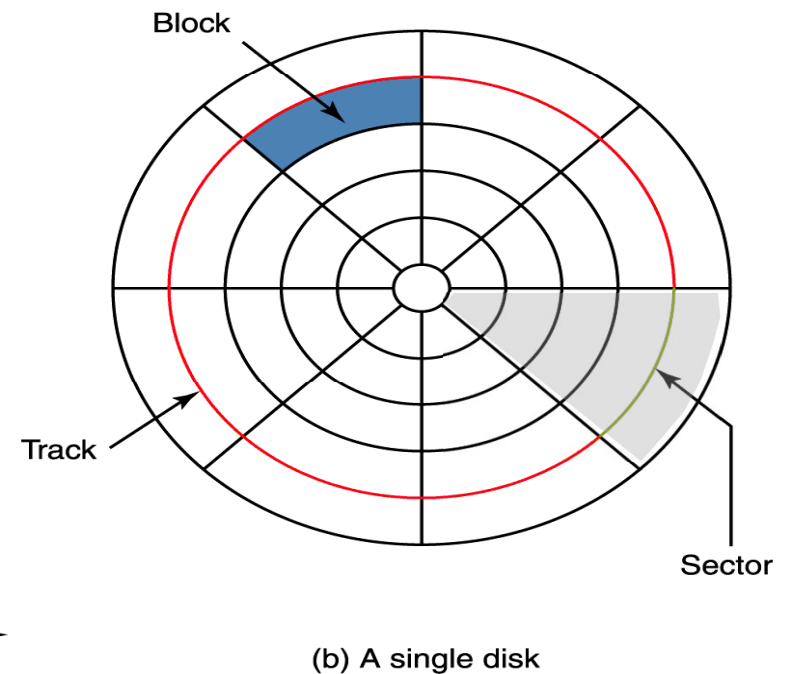
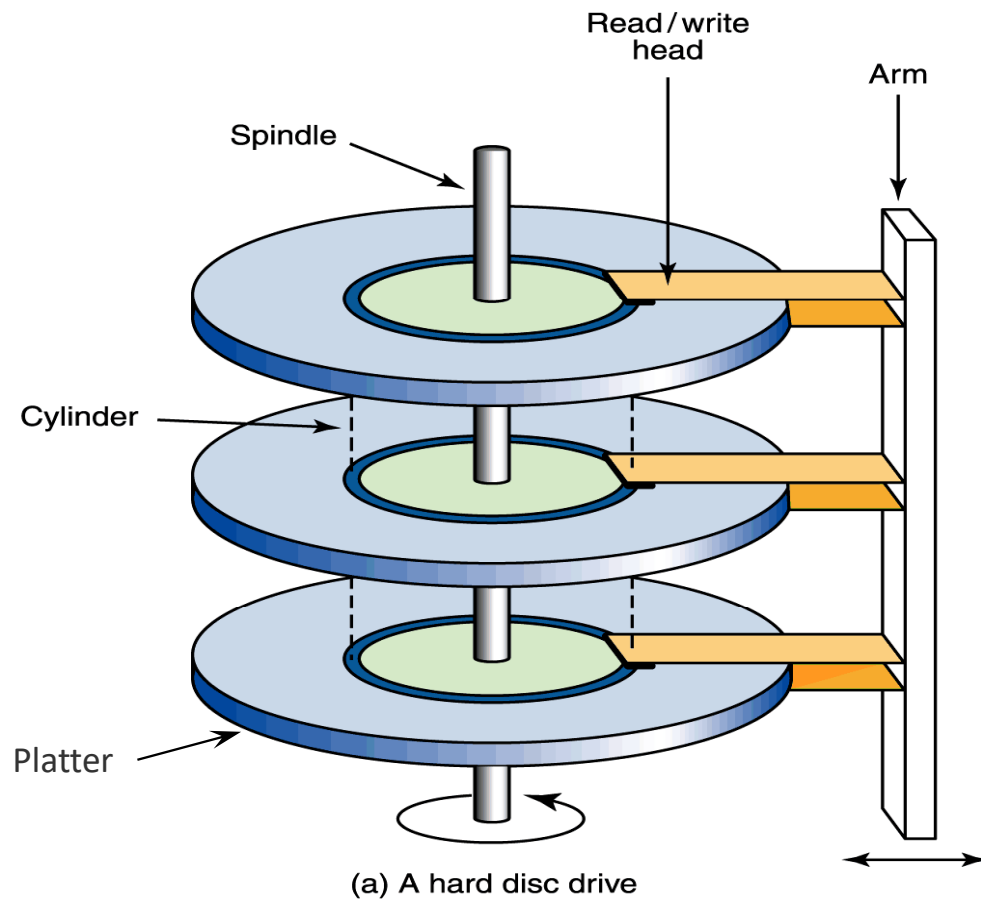
The following illustrates various equivalent system calls for Windows and UNIX operating systems.

	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Memoria Secondaria

- La **memoria centrale** è un supporto di memorizzazione (volatile) che può essere acceduto direttamente dalla CPU.
- La **memoria secondaria** è una estensione della memoria centrale, in grado di fornire un supporto di memorizzazione non volatile e di maggiori dimensioni.
 - Un disco magnetico è un insieme di piatti ricoperti di materiale magnetico.

Disco Magnetico





- ✓ CPU e ciclo Fetch-Decode-Execute
- ✓ Organizzazione della Memoria
- ✓ Organizzazione dei Dispositivi di I/O
- Protezione dell'Hardware



Protezioni Hardware

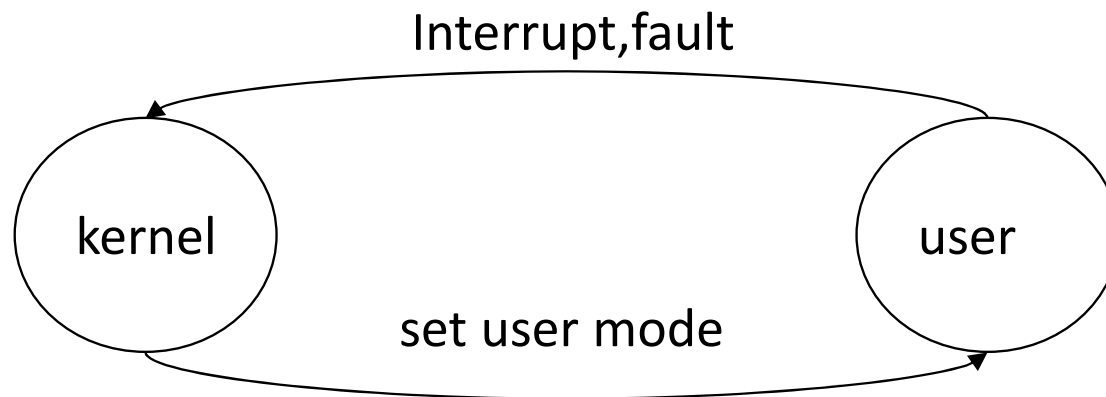
- Funzionamento in Dual-Mode
- Protezione dell'I/O
- Protezione della Memoria
- Protezione della CPU

Funzionamento in Dual-Mode

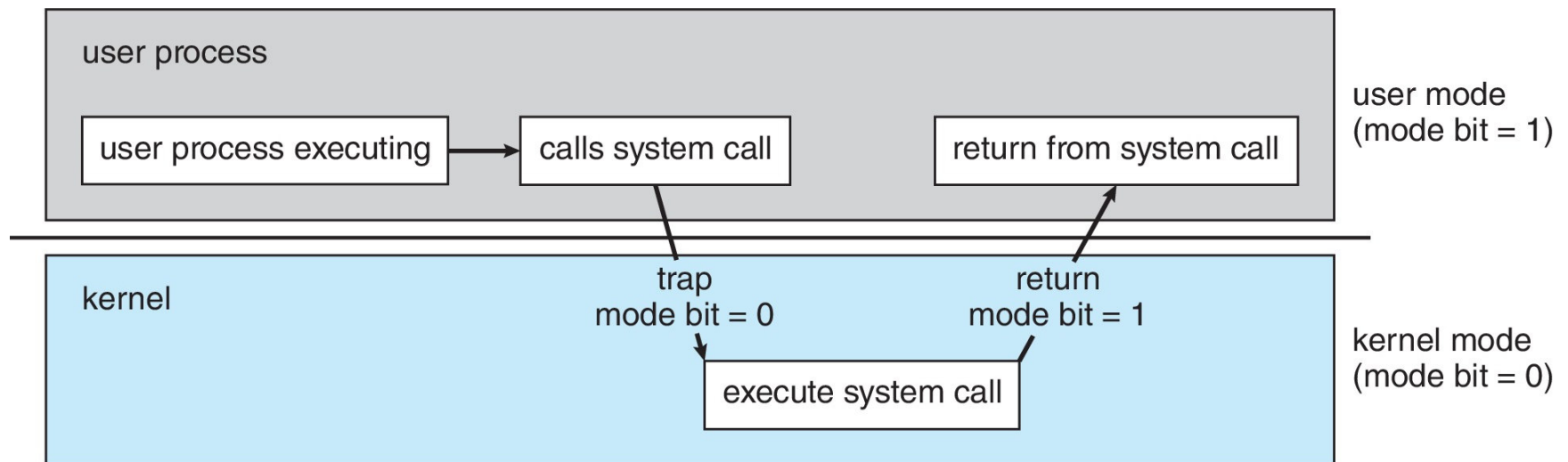
- La condivisione delle risorse di sistema richiede al SO di assicurare che un programma non corretto non danneggi gli altri programmi (SO incluso).
 - I sistemi sono quindi forniti di hardware di supporto per differenziare come minimo due modi di funzionamento:
 1. **User mode** – il sistema sta eseguendo istruzioni per conto di programma utente.
 2. **Kernel mode** (anche detto *monitor mode* o *system mode*) – il sistema sta eseguendo istruzioni per conto del SO.
 - Anche le istruzioni vengono suddivise in
 - **istruzioni privilegiate** – le istruzioni macchina in grado di causare danni allo stato del sistema
 - **istruzioni non privilegiate** – tutte le altre istruzioni
- Le istruzioni privilegiate possono essere eseguite solo in modalità kernel.

Funzionamento in Dual-Mode (Cont.)

- Viene aggiunto un *bit di modo* all'hardware del computer per indicare la modalità di funzionamento: kernel (0) o user (1).
- Quando c'è una interruzione o un trap, l'hardware passa in modalità kernel. In questo modo l'interrupt handler (che è un componente del software di sistema operativo) esegue in *modalità kernel*. Alla istruzione di ritorno da interrupt, viene ripristinata la *modalità utente*.



Funzionamento in Dual-Mode (Cont.)



- La fase di esecuzione di una qualsiasi istruzione privilegiata inizierà con un test sul bit di modo: se questo è impostato a “kernel” l’esecuzione prosegue, altrimenti viene generata una eccezione (trap) e il programma che ha tentato di eseguire l’istruzione privilegiata viene interrotto.



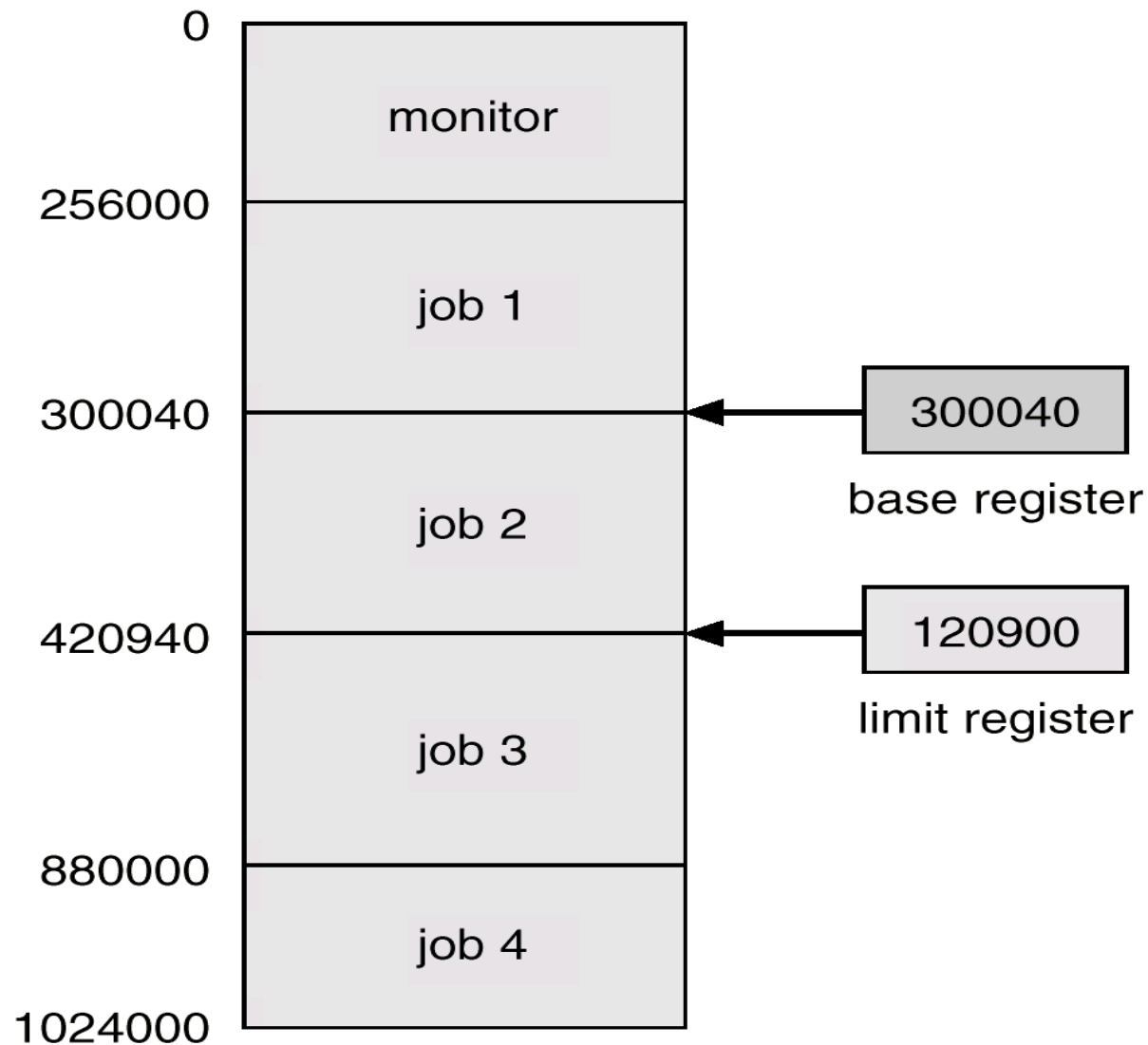
Protezione dell'I/O

- Tutte le istruzioni di I/O sono istruzioni privilegiate.
- Questo è il motivo per cui per eseguire operazioni di I/O bisogna eseguire delle system call.

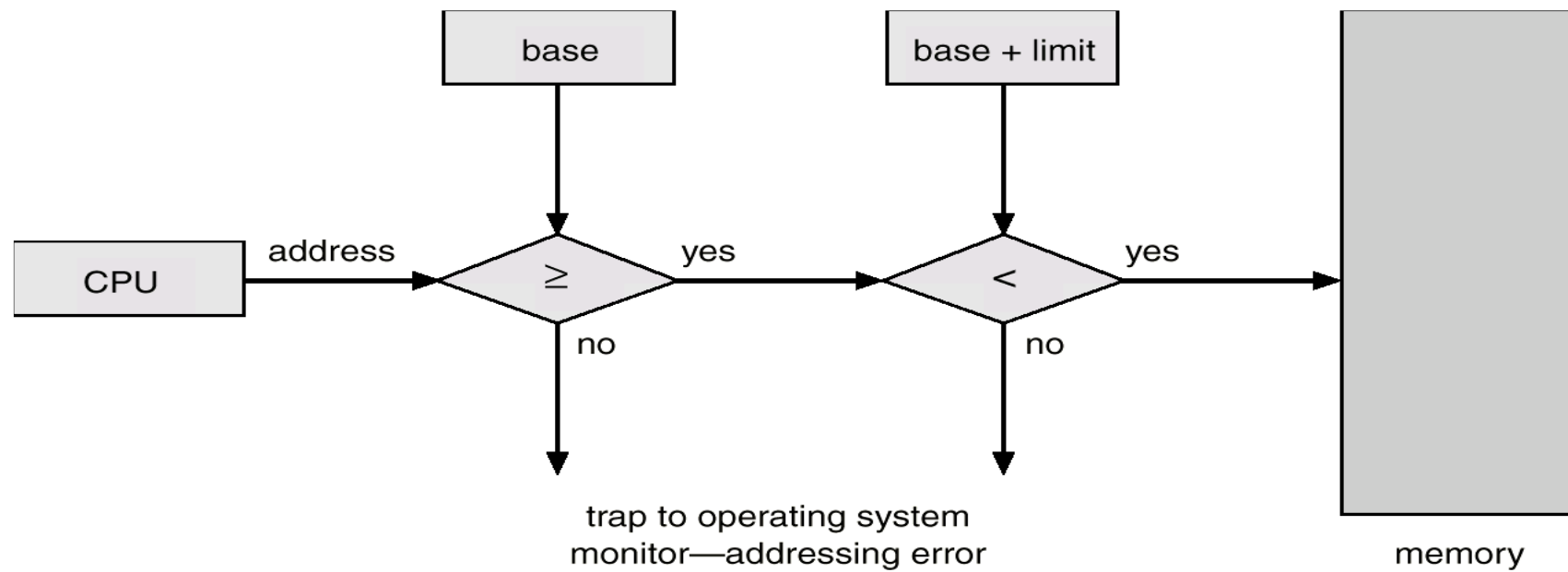
Protezione della Memoria

- Le protezioni viste fino ad ora non sono sufficienti, infatti un programma utente potrebbe modificare il vettore di interrupt mettendo un indirizzo relativo al codice del programma utente stesso, in questo modo il programma utente riuscirebbe ad eseguire le proprie istruzioni in modalità utente.
- Serve quindi proteggere la memoria da accessi indesiderati, al meno per il vettore di interrupt e gli interrupt handler.
- Per avere la protezione della memoria bisogna aggiungere due registri che determinano l'intervallo nello spazio degli indirizzi che un programma può accedere legalmente:
 - **Registro Base** – contiene l'indirizzo fisico più basso al quale un programma può accedere legalmente.
 - **Registro Limite** – contiene la dimensione dell'intervallo
- La memoria al di fuori di tale intervallo è protetta.

Protezione della Memoria



Protezione della Memoria





Protezione della Memoria

- Quando il SO sta funzionando in modalità kernel, ha accesso illimitato sia alla memoria del monitor sia alla memoria dei programmi utente.
- Le istruzioni di load per i registri base e limite sono istruzioni privilegiate.

Protezione della CPU

- Bisogna evitare che un programma mantenga il controllo della CPU per un tempo illimitato, il SO deve essere sempre in grado di riprendere il controllo della CPU.
- Questo può essere ottenuto con un **timer**.
- *Timer* – interrompe il computer dopo uno specifico periodo di tempo:
 - Timer è un contatore che viene decrementato dopo ogni clock tick.
 - Quando raggiunge il valore 0, viene generato un interrupt.
- Timer sono comunemente usati per realizzare il time sharing.
- Timer sono anche usati per calcolare l'ora.
- Load-timer è una istruzione privilegiata.